

Sequence Modeling: RNN, LSTM, GRU, Deep RNN and Bidirectional RNN

Md. Shaon Khan

সেপ্টেম্বর ২০২৬

Table of Contents

০. Introduction (ভূমিকা).....	8
০.১ Sequence Modeling কী?.....	8
০.২ কেন ordinary Feedforward Network sequence-এর জন্য যথেষ্ট নয়?.....	8
০.৩ Vanilla RNN-এর মূল ধারণা (Foundation Recap).....	9
০.৪ Vanilla RNN-এর প্রধান Limitation	10
০.৪.১ Gradient-এর Chain: সমস্যার উৎস.....	10
০.৪.২ Vanishing Gradient (গ্রেডিয়েন্ট বিলুপ্তি).....	11
০.৪.৩ Exploding Gradient (গ্রেডিয়েন্ট বিস্ফোরণ).....	11
০.৪.৪ Long-Term Dependency Problem.....	11
০.৫ এই Document-এর কেন্দ্রীয় Progression	12
১. PART 1 — Long Short-Term Memory (LSTM).....	13
১.১ LSTM কেন তৈরি করা হলো?	13
১.২ LSTM-এর Core Idea	13
(ক) Cell State (C_t) — দীর্ঘমেয়াদি স্মৃতি	13
(খ) Hidden State (h_t) — স্বল্পমেয়াদি/প্রকাশযোগ্য স্মৃতি.....	13
(গ) Gates — নিয়ন্ত্রণ ব্যবস্থা.....	14
(ঘ) Candidate Cell State (C_t) — সম্ভাব্য নতুন তথ্য.....	14
১.৩ LSTM Architecture.....	14
[DIAGRAM 1: Full LSTM Cell Architecture]	14
[DIAGRAM 2: LSTM Cell State and Hidden State Flow Across Time]	15
১.৪ LSTM-এর প্রথম Gate — Forget Gate.....	16
১.৪.১ What is it? (এটি কী?).....	16
১.৪.২ Why is it needed? (কেন প্রয়োজন?)	16
১.৪.৩ Mathematical Equation	16
১.৪.৪ কেন Sigmoid, কেন tanh নয়?	17
১.৪.৫ Dimension Tracking.....	17
১.৪.৬ Numerical Example (হাতে-কলমে).....	17
১.৪.৭ Interpretation (ফলাফলের ব্যাখ্যা)	18
১.৫ Input Gate এবং Candidate Cell State.....	18

১.৫.১ দুটি আলাদা অংশ কেন?	18
১.৫.২ Equations	19
১.৫.৩ Numerical Example	19
১.৫.৪ Interpretation	20
১.৬.১ Equation	20
১.৬.২ কেন এই গঠনটি এত গুরুত্বপূর্ণ?	20
১.৬.৩ Complete Numerical Example	20
১.৬.৪ Interpretation	21
১.৭ Output Gate এবং Hidden State	21
১.৭.১ Why is it needed?	21
১.৭.২ Equations	21
১.৭.৩ Dimension Tracking	22
১.৭.৪ Numerical Example	22
১.৭.৫ Interpretation	22
১.৮ Complete LSTM Forward Pass — একত্রে	22
১.৯ LSTM Backward Propagation	23
১.৯.১ প্রয়োজনীয় Derivative-সমূহ	23
১.৯.২ Computational Graph	24
১.৯.৩ ধাপ ১ — Output Gate-এর দিকে	24
১.৯.৪ ধাপ ২ — Cell State-এর দিকে (সবচেয়ে গুরুত্বপূর্ণ)	24
১.৯.৫ ধাপ ৩ — তিন Gate ও Candidate-এর দিকে	24
১.৯.৬ ধাপ ৪ — Parameter Gradient	25
১.৯.৭ ধাপ ৫ — পূর্ববর্তী state-এর দিকে gradient	25
১.৯.৮ Numerical Backward Example	25
১.৯.৯ কেন LSTM Vanishing Gradient কিছুটা মোকাবিলা করতে পারে?	26
১.১০ LSTM Parameter Calculation	27
১.১০.১ Derivation (উৎপত্তি)	27
১.১০.২ Numerical Example ১ (আমাদের ছোট উদাহরণ)	28
১.১০.৩ Numerical Example ২ (বাস্তবসম্মত আকার)	28
১.১০.৪ Vanilla RNN-এর সঙ্গে তুলনা	28

২. PART 2 — Gated Recurrent Unit (GRU).....	29
২.১ কেন GRU?.....	29
২.২ GRU Core Idea.....	29
(ক) Single Hidden State (ht).....	29
(খ) Reset Gate (rt).....	30
(গ) Update Gate (zt).....	30
(ঘ) Candidate Hidden State (ht).....	30
২.৩ GRU Architecture.....	30
[DIAGRAM 3: Complete GRU Cell Architecture]	30
[DIAGRAM 4: GRU Gate-by-Gate Information Flow]	Error! Bookmark not defined.
২.৪ Reset Gate.....	32
২.৪.১ What & Why	32
২.৪.২ Dimension Tracking (বিস্তারিত)	33
২.৫ Reset Filtering — $rt \odot ht - 1$	33
২.৬ Candidate Hidden State এবং সম্পূর্ণ Numerical Example	34
২.৬.১ প্রাথমিক মান.....	34
২.৬.২ Reset Gate গণনা	34
২.৬.৩ Update Gate গণনা.....	34
২.৬.৪ Candidate Hidden State গণনা.....	34
২.৬.৫ Dimension যাচাই.....	35
২.৭ Update Gate-এর ভূমিকা.....	35
২.৮ Final Hidden State — সম্পূর্ণ Numerical Example.....	36
Interpretation.....	36
২.৯ GRU Backward Propagation	37
২.৯.১ Computational Graph.....	37
২.৯.২ ধাপে ধাপে Derivation	37
২.৯.৩ Numerical Backward Example.....	38
২.৯.৪ GRU-তে Gradient Flow-এর বিশ্লেষণ.....	39
২.১০ GRU Parameter Calculation	39
২.১০.১ Derivation	39

২.১০.২ Numerical Examples.....	39
২.১০.৩ LSTM-এর সঙ্গে গাণিতিক তুলনা	40
৩. PART 3 — LSTM বনাম GRU: বিস্তারিত তুলনা.....	41
8. PART 4 — Deep RNN / Stacked RNN	43
8.১ Deep RNN কী?.....	43
8.২ Temporal Depth বনাম Spatial Depth	43
[DIAGRAM 5: Deep RNN — Stacked and Unfolded in Time]	44
8.৩ Mathematical Notation	45
8.৩.১ $h_{t,l}$ -এর অর্থ.....	45
8.৩.২ Master Equation	45
8.৪ W এবং U -এর মৌলিক পার্থক্য.....	46
[DIAGRAM 6: Deep RNN — Horizontal vs Vertical Information Flow]	47
8.৫ Dimension Tracking	47
8.৬ Deep RNN Forward Pass — Numerical Example.....	48
ধাপ ১ — Layer 1, timestep t (উল্লম্ব প্রবাহের শুরু).....	49
ধাপ ২ — Layer 2, timestep t	49
ধাপ ৩ — পরবর্তী timestep $t + 1$ (অনুভূমিক প্রবাহ).....	49
গণনার ক্রম সম্পর্কে একটি গুরুত্বপূর্ণ পর্যবেক্ষণ	50
8.৭ Deep RNN Backpropagation — দুই দিকে Gradient	50
8.৭.১ মোট Gradient-এর গঠন.....	50
8.৭.২ Pre-activation-এ রূপান্তর.....	50
8.৭.৩ Temporal Backpropagation ($t \rightarrow t - 1$)	51
8.৭.৪ Vertical Backpropagation ($l \rightarrow l - 1$)	51
8.৭.৫ Parameter Gradient	51
[DIAGRAM 10: Forward vs Backward Propagation Concept]	52
8.৮ Deep RNN Parameter Calculation — একটি সম্পূর্ণ উদাহরণ	53
ধাপ ১ — Embedding Layer.....	53
ধাপ ২ — SimpleRNN Layer 1	53
ধাপ ৩ — SimpleRNN Layer 2.....	53
ধাপ ৪ — মোট	53

সাধারণ সূত্র	53
৪.৯ Deep RNN কখন ও কেন ব্যবহার করবেন?.....	54
৪.৯.১ সুবিধা (Advantages)	54
৪.৯.২ সীমাবদ্ধতা (Limitations)	54
৫. PART 5 — Bidirectional RNN (BiRNN)	55
৫.১ কেন Bidirectional RNN প্রয়োজন?	55
৫.১.১ NER উদাহরণ (Named Entity Recognition).....	55
৫.১.২ আরও কিছু উদাহরণ.....	55
৫.২ Architecture.....	56
[DIAGRAM 7: Complete Bidirectional RNN Architecture]	Error! Bookmark not defined.
[DIAGRAM 8: BiRNN Context Flow for NER]	Error! Bookmark not defined.
৫.৩ Forward Pass (বাম → ডান).....	58
৫.৪ Backward Pass (ডান → বাম).....	58
৫.৫ দুই দিক একত্রীকরণ (Combining Both Directions).....	59
৫.৬ সম্পূর্ণ Numerical BiRNN Example.....	59
৫.৬.১ Setup.....	59
৫.৬.২ Forward Direction গণনা ($t = 1 \rightarrow 3$).....	60
৫.৬.৩ Backward Direction গণনা ($t = 3 \rightarrow 1$)	60
৫.৬.৪ সব hidden state একত্রে.....	61
৫.৬.৫ $t = 2$ -এর চূড়ান্ত Output.....	61
৫.৬.৬ Interpretation	61
৫.৬.৭ Parameter Count.....	62
৫.৭ BiRNN Backward Propagation (Gradient গণনা).....	62
৫.৭.১ মূল নীতি.....	62
৫.৭.২ ধাপ ১ — Output থেকে Concatenated State-এ	62
৫.৭.৩ ধাপ ২ — Concatenation-এর gradient ভাগ করা.....	62
৫.৭.৪ ধাপ ৩ — দুই network-এ স্বাধীন BPTT	63
৫.৮ Applications.....	63
৫.৯ Limitations.....	64
৬. PART 6 — সামগ্রিক স্থাপত্যগত অগ্রগতি (Architectural Progression)	65

৬.১ মূল ধারা: Memory নিয়ন্ত্রণের বিবর্তন.....	65
পর্যায় ১ → ২ : Vanilla RNN থেকে LSTM.....	65
পর্যায় ২ → ৩ : LSTM থেকে GRU.....	65
৬.২ সমান্তরাল সম্প্রসারণ: গভীরতা ও দিক.....	65
সম্প্রসারণ ক : RNN → Deep RNN.....	66
সম্প্রসারণ খ : RNN → Bidirectional RNN.....	66
৬.৩ সম্মিলিত রূপ	66
[DIAGRAM 9: Evolution of Recurrent Neural Network Architectures].....	66
৭. PART 7 — সম্পূর্ণ তুলনামূলক সারণি.....	69
৮. PART 8 — সম্পূর্ণ Forward–Backward দৃষ্টিভঙ্গি.....	71
৮.১ সাধারণ কাঠামো.....	71
Forward Pass.....	71
Backward Pass.....	71
তিনটি সার্বজনীন নিয়ম.....	71
৮.২ প্রতিটি architecture-এ এই প্রক্রিয়া কীভাবে পরিবর্তিত হয়.....	71
৮.২.১ Vanilla RNN.....	71
৮.২.২ LSTM	72
৮.২.৩ GRU.....	72
৮.২.৪ Deep RNN.....	72
৮.২.৫ Bidirectional RNN	72
৮.৩ একটি একীভূত পর্যবেক্ষণ.....	73
৯. প্রয়োজনীয় Diagram-সমূহের তালিকা.....	74
১০. Key Takeaways / Final Summary	75

০. Introduction (ভূমিকা)

০.১ Sequence Modeling কী?

Sequence Modeling (অনুক্রম মডেলিং) হলো Machine Learning-এর সেই শাখা যেখানে input বা output অথবা উভয়ই একটি **ক্রমানুসারে সাজানো data** (ordered data) হিসেবে আসে, এবং সেই ক্রম বা order-এর নিজেরই একটি অর্থ থাকে।

একটি sequence-কে আমরা লিখি:

$$X = (x_1, x_2, x_3, \dots, x_T)$$

এখানে:

- x_t = sequence-এর t -তম timestep-এর input vector
- T = sequence-এর মোট length (দৈর্ঘ্য)
- t = time index (সময়ের ক্রম নির্দেশক)

গুরুত্বপূর্ণ বৈশিষ্ট্য: এখানে x_1 এবং x_2 -এর অবস্থান অদলবদল করলে data-র অর্থ বদলে যায়। যেমন “কুকুরটি মানুষকে কামড়াল” এবং “মানুষটি কুকুরকে কামড়াল” — একই শব্দ, কিন্তু ভিন্ন ক্রম, ভিন্ন অর্থ।

Sequence data-র বাস্তব উদাহরণ:

- **Text:** একটি বাক্যের শব্দ বা অক্ষরের ক্রম
- **Speech:** সময় অনুযায়ী audio signal-এর amplitude
- **Time Series:** stock price, তাপমাত্রা, বিদ্যুৎ খরচের দৈনিক রেকর্ড
- **Biomedical Signal:** ECG signal-এর ক্রমিক sample
- **Video:** frame-এর ক্রম
- **DNA Sequence:** nucleotide-এর ক্রম

Sequence modeling-এর মূল লক্ষ্য হলো এমন একটি function f শেখা যা শুধু current input নয়, বরং **অতীতের context** ব্যবহার করে prediction করে:

$$y_t = f(x_1, x_2, \dots, x_t)$$

০.২ কেন ordinary Feedforward Network sequence-এর জন্য যথেষ্ট নয়?

একটি সাধারণ **Feedforward Neural Network** (FFN) বা Multi-Layer Perceptron (MLP)-এর কাজ হলো:

$$y = f(Wx + b)$$

অর্থাৎ একটি fixed-size input vector x নিয়ে একটি fixed-size output তৈরি করা। Sequence data-র ক্ষেত্রে এর চারটি মৌলিক সীমাবদ্ধতা আছে:

(১) Fixed Input Size (নির্দিষ্ট আকারের input):

FFN-এর input layer-এর neuron সংখ্যা স্থির। কিন্তু বাক্যের দৈর্ঘ্য পরিবর্তনশীল — কোনো বাক্যে ৫টি শব্দ, কোনোটিতে ৫০টি। প্রতিটি length-এর জন্য আলাদা network তৈরি করা আবাস্তব।

(২) No Memory (স্মৃতির অনুপস্থিতি):

FFN প্রতিটি input-কে সম্পূর্ণ স্বাধীনভাবে (independently) process করে। x_5 process করার সময় x_1, x_2, x_3, x_4 সম্পর্কে তার কোনো ধারণা থাকে না। কিন্তু ভাষা বোঝার জন্য অতীতের context অপরিহার্য।

(৩) No Parameter Sharing (parameter ভাগাভাগি নেই):

যদি একটি ১০০-শব্দের বাক্যকে flatten করে FFN-এ দেওয়া হয়, তাহলে ১ নম্বর অবস্থানে “ভালো” শব্দটি চিনতে শেখা weight ৫০ নম্বর অবস্থানে কাজে লাগবে না। প্রতিটি position-এর জন্য আলাদা weight শিখতে হবে — যা parameter সংখ্যা বিস্ফোরিত করে এবং generalization নষ্ট করে।

(৪) No Temporal Structure (সময়ের কাঠামো নেই):

Input flatten করলে ক্রমের তথ্য (order information) হারিয়ে যায়।

Important Note: এই চারটি সমস্যার সমাধান হিসেবেই **Recurrent Neural Network (RNN)** ধারণাটি এসেছে — একটি network যা একই weight বারবার ব্যবহার করে, এবং একটি **internal state**-এ অতীতের তথ্য জমা রাখে।

০.৩ Vanilla RNN-এর মূল ধারণা (Foundation Recap)

এই অংশটি কেবল পরবর্তী আলোচনার ভিত্তি তৈরির জন্য। বিস্তারিত Vanilla RNN আলোচনা আলাদা note-এ আছে।

Vanilla RNN-এর কেন্দ্রীয় সমীকরণ:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{hy} h_t + b_y$$

প্রতিটি symbol-এর অর্থ:

Symbol	অর্থ	Dimension
x_t	t সময়ের input vector	$n_x \times 1$
h_{t-1}	পূর্ববর্তী timestep-এর hidden state (অতীতের স্মৃতি)	$n_h \times 1$
h_t	বর্তমান timestep-এর hidden state	$n_h \times 1$

W_{xh}	input থেকে hidden-এ weight matrix	$n_h \times n_x$
W_{hh}	hidden থেকে hidden-এ recurrent weight matrix	$n_h \times n_h$
b_h	bias vector	$n_h \times 1$
tanh	non-linear activation function	element-wise

এই একটিমাত্র সমীকরণ থেকে চারটি ধারণা পরিষ্কার হয়:

(ক) Current Input ($W_{xh}x_t$): বর্তমান মুহূর্তে কী নতুন তথ্য এলো।

(খ) Previous Memory ($W_{hh}h_{t-1}$): অতীত থেকে কী তথ্য বহন করে আনা হলো। এই পদটিই RNN-কে “স্মৃতিসম্পন্ন” করে।

(গ) Recurrent Connection: h_t গণনা করতে h_{t-1} লাগে, আবার h_{t+1} গণনা করতে h_t লাগে। অর্থাৎ একই W_{hh} প্রতিটি timestep-এ পুনর্ব্যবহৃত (reused) হয় — এটিই **parameter sharing**।

(ঘ) Temporal Dependency: h_t পরোক্ষভাবে x_1 থেকে x_t পর্যন্ত সব input-এর উপর নির্ভরশীল:

$$h_t = f(x_t, f(x_{t-1}, f(x_{t-2}, \dots)))$$

০.৪ Vanilla RNN-এর প্রধান Limitation

তত্ত্বগতভাবে RNN যেকোনো দৈর্ঘ্যের অতীত মনে রাখতে পারার কথা। বাস্তবে পারে না। কারণটি গাণিতিক।

০.৪.১ Gradient-এর Chain: সমস্যার উৎস

Backpropagation Through Time (BPTT)-তে timestep t -এর loss থেকে timestep k ($k < t$)-এর দিকে gradient যেতে হলে chain rule অনুযায়ী মাঝের সব ধাপ পেরোতে হয়:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

Vanilla RNN-এ প্রতিটি পদ হলো:

$$\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(1 - h_j^2) \cdot W_{hh}$$

এখানে $\text{diag}(1 - h_j^2)$ হলো tanh-এর derivative (কারণ $\tanh'(a) = 1 - \tanh^2(a)$)।

অর্থাৎ পুরো gradient-টি হলো একই ধরনের matrix-এর **বারবার গুণফল (repeated product)**:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \text{diag}(1 - h_j^2) W_{hh}$$

০.৪.২ Vanishing Gradient (গ্রেডিয়েন্ট বিলুপ্তি)

লক্ষ করুন, $\tanh$ -এর derivative-এর সর্বোচ্চ মান ১ (যখন $h_j = 0$), এবং সাধারণত এটি ১-এর অনেক কম:

$$0 < 1 - h_j^2 \leq 1$$

ধরা যাক প্রতিটি ধাপে gradient-এর magnitude গড়ে 0.5 গুণ করে কমছে। তাহলে ২০ timestep পিছনে যেতে:

$$(0.5)^{20} \approx 9.5 \times 10^{-7}$$

অর্থাৎ gradient কার্যত শূন্য। ফলাফল: **দূরবর্তী অতীতের weight আর update হয় না।** Network দীর্ঘমেয়াদি সম্পর্ক শিখতে ব্যর্থ হয়।

০.৪.৩ Exploding Gradient (গ্রেডিয়েন্ট বিস্ফোরণ)

বিপরীত ক্ষেত্রে, যদি W_{hh} -এর largest eigenvalue ১-এর বেশ বড় হয়, তাহলে প্রতিটি ধাপে gradient বেড়ে যায়:

$$(1.5)^{20} \approx 3325$$

ফলে weight update বিশাল হয়ে যায়, loss NaN হয়ে যায়, training অস্থিতিশীল হয়।

Note: Exploding gradient-এর একটি সহজ প্রকৌশলগত সমাধান আছে — **Gradient Clipping** (gradient-এর norm একটি threshold-এর মধ্যে সীমাবদ্ধ করা)। কিন্তু Vanishing Gradient-এর এমন সহজ সমাধান নেই; এর জন্য **architecture পরিবর্তন** দরকার। এখান থেকেই LSTM-এর জন্ম।

০.৪.৪ Long-Term Dependency Problem

উদাহরণ:

“আমি বাংলাদেশে বড় হয়েছি ... (৪০টি শব্দ) ... তাই আমি সাবলীলভাবে **বাংলা** বলতে পারি।”

শেষ শব্দটি predict করতে হলে ৪০ শব্দ আগের “বাংলাদেশ” শব্দটি মনে রাখতে হবে। Vanilla RNN-এ vanishing gradient-এর কারণে এই সংযোগ শেখা কার্যত অসম্ভব।

এছাড়াও Vanilla RNN-এ প্রতিটি timestep-এ hidden state **সম্পূর্ণ overwrite** হয়ে যায়:

$$h_t = \tanh(\dots)$$

অর্থাৎ পুরনো তথ্য ধরে রাখা বা নির্বাচিতভাবে ভুলে যাওয়ার কোনো **নিয়ন্ত্রণ ব্যবস্থা (control mechanism)** নেই।

০.৫ এই Document-এর কেন্দ্রীয় Progression

পুরো নোটটি নিচের যুক্তিক্রম অনুসরণ করবে। এটিই সব architecture বোঝার মূল সূত্র:

ধাপ	সমস্যা (Problem)	স্থাপত্যগত পরিবর্তন (Architectural Change)	ফলাফল (Consequence)
Vanilla RNN	Long-term dependency, vanishing gradient	—	দীর্ঘ স্মৃতি রাখতে ব্যর্থ
LSTM	উপরের সমস্যা	Cell State + ৩টি Gate (additive update)	নিয়ন্ত্রিত ও দীর্ঘস্থায়ী স্মৃতি
GRU	LSTM-এর জটিলতা ও বেশি parameter	২টি Gate, একটিমাত্র state	সরল, দ্রুত, তুলনীয় কর্মক্ষমতা
Deep RNN	একস্তরের সীমিত representational power	একাধিক recurrent layer উল্লম্বভাবে stack	Hierarchical/abstract feature
BiRNN	ভবিষ্যৎ context দেখা যায় না	দুটি বিপরীতমুখী RNN + concatenation	পূর্ণ (past + future) context

১. PART 1 — Long Short-Term Memory (LSTM)

১.১ LSTM কেন তৈরি করা হলো?

Vanilla RNN-এর মূল ত্রুটি দুটি, এবং দুটিই একই মূল কারণ থেকে আসে:

(ক) Multiplicative State Update: প্রতিটি timestep-এ hidden state পুরোপুরি নতুন করে তৈরি হয় এবং gradient-পথে বারবার W_{hh} ও $\tanh'$ দিয়ে গুণ হতে থাকে। বারবার গুণ মানেই exponential বৃদ্ধি বা ক্ষয়।

(খ) No Selective Control: কোন তথ্য রাখা হবে আর কোনটি ফেলে দেওয়া হবে — তা ঠিক করার কোনো আলাদা প্রক্রিয়া নেই।

১৯৯৭ সালে Hochreiter ও Schmidhuber-এর প্রস্তাবিত LSTM-এর মূল অন্তর্দৃষ্টি (key insight) হলো:

স্মৃতি update করার জন্য গুণ (multiplication)-এর বদলে যোগ (addition) ব্যবহার করো, এবং কী যোগ হবে বা কী মুছে যাবে তা নির্ধারণের জন্য শেখযোগ্য (learnable) gate ব্যবহার করো।

গাণিতিকভাবে এই পার্থক্যটি অত্যন্ত গুরুত্বপূর্ণ:

$$h_t = \tanh(W_{hh}h_{t-1} + \dots) \quad \text{vs} \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Vanilla RNN: multiplicative LSTM: additive

যোগ operation-এর derivative হলো ১ (identity)। তাই gradient backward যাওয়ার সময় বারবার ক্ষয় না হয়ে একটি “মহাসড়ক” (highway) ধরে প্রবাহিত হতে পারে। এই বিষয়টি ধারা ১.৯-এ গাণিতিকভাবে দেখানো হয়েছে।

১.২ LSTM-এর Core Idea

LSTM-এর চারটি মৌলিক উপাদান:

(ক) Cell State (C_t) — দীর্ঘমেয়াদি স্মৃতি

Cell State হলো LSTM-এর প্রধান স্মৃতি-বাহক (memory carrier)। এটি পুরো sequence জুড়ে সামান্য linear পরিবর্তন নিয়ে বয়ে চলে। একে বলা হয় **Information Highway** বা **Conveyor Belt** — যেন একটি চলন্ত বেল্ট যার উপর তথ্য রাখা থাকে, এবং প্রতিটি timestep-এ কিছু জিনিস বেল্ট থেকে সরানো হয় ও কিছু নতুন জিনিস রাখা হয়।

(খ) Hidden State (h_t) — স্বল্পমেয়াদি/প্রকাশযোগ্য স্মৃতি

Hidden State হলো Cell State-এর একটি **filtered**, **ব্যবহারযোগ্য সংস্করণ**। এটিই বাইরে (output layer বা পরবর্তী layer-এ) পাঠানো হয় এবং পরবর্তী timestep-এর gate গণনায় ব্যবহৃত হয়।

মনে রাখার উপমা: C_t হলো আপনার সম্পূর্ণ নোটবুক (সব তথ্য লেখা আছে), আর h_t হলো পরীক্ষার হলে আপনি যে অংশটুকু এই মুহূর্তে মনে করছেন ও ব্যবহার করছেন।

(গ) Gates — নিয়ন্ত্রণ ব্যবস্থা

Gate হলো একটি sigmoid layer যার output প্রতিটি উপাদানের জন্য 0 থেকে 1-এর মধ্যে একটি সংখ্যা দেয়, এবং সেটি element-wise multiplication ($\odot$) দিয়ে তথ্যপ্রবাহ নিয়ন্ত্রণ করে:

- 0 $\rightarrow$ “কিছুই যেতে দিও না” (সম্পূর্ণ বন্ধ)
- 1 $\rightarrow$ “সবকিছু যেতে দাও” (সম্পূর্ণ খোলা)
- 0.5 $\rightarrow$ “অর্ধেক যেতে দাও” (আংশিক)

LSTM-এ তিনটি gate:

Gate	প্রতীক	কাজ
Forget Gate	f_t	পুরনো Cell State-এর কতটুকু ভুলে যাব
Input Gate	i_t	নতুন তথ্যের কতটুকু গ্রহণ করব
Output Gate	o_t	Cell State-এর কতটুকু প্রকাশ করব

(ঘ) Candidate Cell State ($\tilde{C}_t$) — সম্ভাব্য নতুন তথ্য

এটি gate নয়; এটি tanh দিয়ে তৈরি একটি **প্রস্তাবিত নতুন তথ্য** যা $[-1,1]$ পরিসরে থাকে। Input gate ঠিক করে এর কতটুকু আসলে স্মৃতিতে যোগ হবে।

১.৩ LSTM Architecture

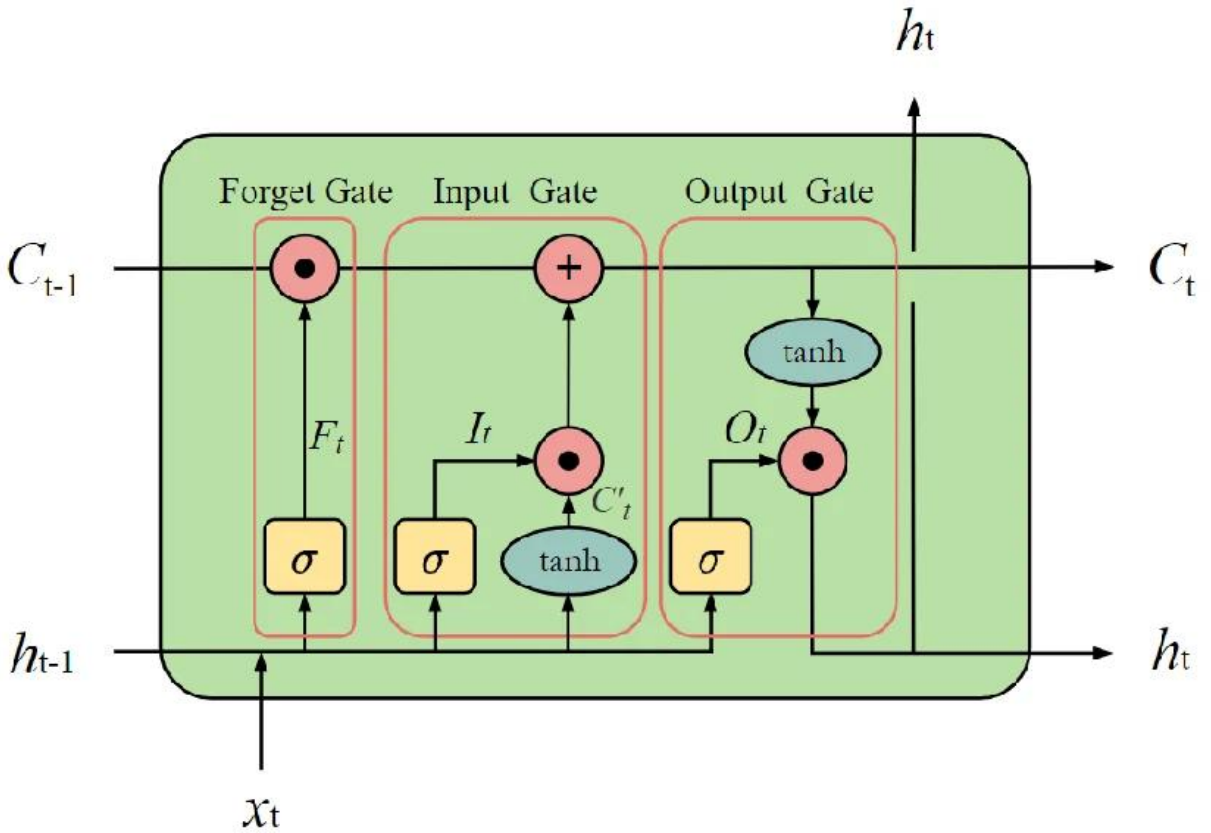
[DIAGRAM 1: Full LSTM Cell Architecture]

Purpose: একটি সম্পূর্ণ LSTM cell-এর ভেতরে x_t, h_{t-1}, C_{t-1} থেকে শুরু করে C_t ও h_t পর্যন্ত সব তথ্যপ্রবাহ একটি ছবিতে দেখানো।

Elements:

- একটি বড় আয়তাকার box (LSTM Cell), যার বাইরে বাম দিক থেকে তিনটি input arrow ঢুকবে: উপরের বাম কোণে C_{t-1} , নিচের বাম কোণে h_{t-1} , এবং একদম নিচ থেকে উল্লম্বভাবে x_t ।
- ডান দিকে দুটি output arrow বেরোবে: উপরের ডান কোণে C_t , নিচের ডান কোণে h_t ; এবং h_t থেকে একটি শাখা উপরের দিকে y_t -এর জন্য।
- h_{t-1} ও x_t একটি concatenation node-এ মিলবে, label: $[h_{t-1}, x_t]$ ।
- Concatenated vector থেকে চারটি সমান্তরাল শাখা যাবে চারটি neural network layer-এ; প্রতিটি হলুদ/কমলা rounded box হিসেবে আঁকা হবে, ভেতরে activation-এর নাম:
 - Box 1: $\sigma \rightarrow$ label **Forget Gate**, equation $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
 - Box 2: $\sigma \rightarrow$ label **Input Gate**, equation $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
 - Box 3: $\tanh \rightarrow$ label **Candidate**, equation $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$
 - Box 4: $\sigma \rightarrow$ label **Output Gate**, equation $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

- Cell-এর একদম উপরে একটি সোজা অনুভূমিক রেখা ($C_{t-1} \rightarrow C_t$), মোটা করে আঁকা, label: **Cell State — Information Highway**।
- এই অনুভূমিক রেখার উপরে ক্রমান্বয়ে দুটি গোলাকার operator node:
 - প্রথমে $\otimes$ (element-wise multiply) — এখানে f_t নিচ থেকে উঠে যুক্ত হবে
 - তারপর $\oplus$ (element-wise add) — এখানে $i_t \odot \tilde{C}_t$ -এর ফল নিচ থেকে উঠে যুক্ত হবে, এবং এই যোগের আগে i_t ও $\tilde{C}_t$ একটি আলাদা $\otimes$ node-এ মিলবে।
- $\oplus$ -এর পরের Cell State থেকে একটি শাখা নিচে নেমে একটি tanh box-এ যাবে, তারপর o_t -এর সঙ্গে $\otimes$ node-এ মিলে h_t তৈরি করবে।
- নিচে একটি ছোট legend box: σ = sigmoid layer, $\tanh$ = tanh layer, $\otimes$ = element-wise multiplication, $\oplus$ = element-wise addition, arrow merge = concatenation, arrow split = copy।

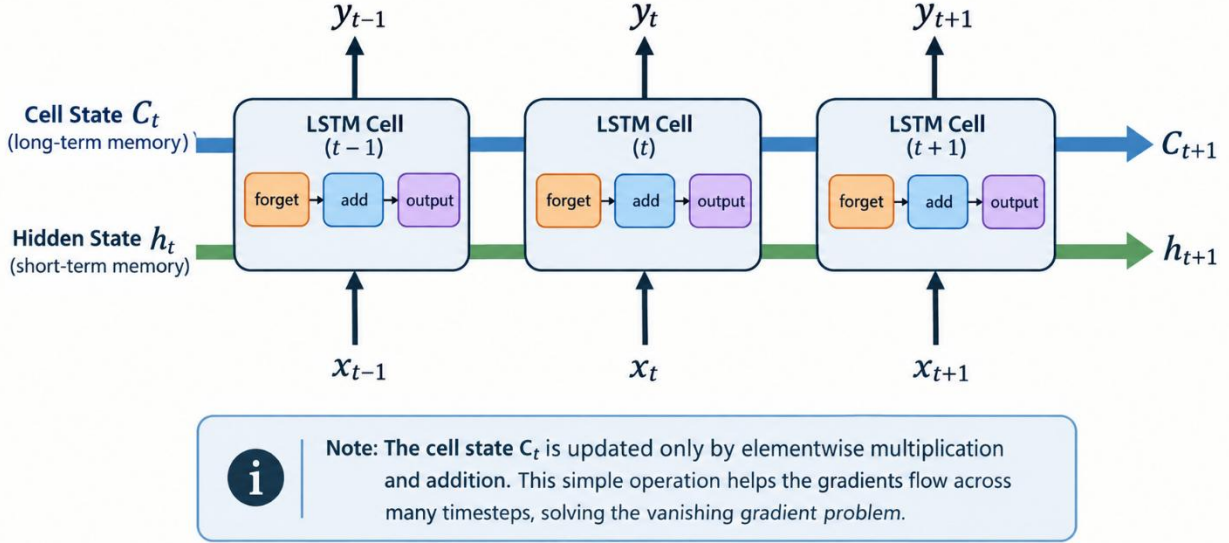


Purpose: তিনটি পরপর timestep জুড়ে C_t ও h_t কীভাবে প্রবাহিত হয়, অর্থাৎ দুটি আলাদা স্মৃতিপথ কীভাবে সময়ের সঙ্গে চলে, তা দেখানো।

Elements:

- পাশাপাশি তিনটি একই রকম LSTM cell box: $t - 1, t, t + 1$ ।
- উপরের স্তরে একটি মোটা অনুভূমিক তীর তিনটি cell-এর ভেতর দিয়ে যাবে — label: **Cell State C (long-term memory)**, রঙ: গাঢ় নীল।

- নিচের স্তরে আরেকটি অনুভূমিক তীর — label: **Hidden State h (short-term / exposed memory)**, রঙ: সবুজ।
- প্রতিটি cell-এর নিচে উল্লম্ব input তীর: x_{t-1}, x_t, x_{t+1} ; উপরে output তীর: y_{t-1}, y_t, y_{t+1} ।
- প্রতিটি cell-এর ভেতরে সংক্ষেপে লেখা থাকবে: forget $\rightarrow$ add $\rightarrow$ output।
- ডান পাশে একটি annotation box: “Cell state changes only through $\odot$ and $+$ — no repeated matrix multiplication, so gradient can flow far back.”



১.৪ LSTM-এর প্রথম Gate — Forget Gate

১.৪.১ What is it? (এটি কী?)

Forget Gate হলো একটি sigmoid layer যা পূর্ববর্তী Cell State C_{t-1} -এর প্রতিটি উপাদানের জন্য নির্ধারণ করে — সেটি কতটুকু ধরে রাখা হবে আর কতটুকু ভুলে যাওয়া হবে।

১.৪.২ Why is it needed? (কেন প্রয়োজন?)

Vanilla RNN-এ পুরনো তথ্য নিজে থেকেই মুছে যেত (uncontrolled overwrite)। কিন্তু কখন ভুলতে হবে তা context-নির্ভর। উদাহরণ: একটি বাক্যে নতুন কর্তা (subject) এলে পুরনো কর্তার লিঙ্গ-সংক্রান্ত তথ্য ভুলে যাওয়া দরকার। Forget gate এই সিদ্ধান্তটি **শিখে নেয়**।

১.৪.৩ Mathematical Equation

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

যেখানে sigmoid:

$$\sigma(a) = \frac{1}{1 + e^{-a}} \in (0,1)$$

Symbol-এর অর্থ:

- $[h_{t-1}, x_t]$ = পূর্ববর্তী hidden state ও বর্তমান input-এর **concatenation** (উল্লম্বভাবে জোড়া লাগানো), dimension $(n_h + n_x) \times 1$
- W_f = forget gate-এর weight matrix, dimension $n_h \times (n_h + n_x)$
- b_f = forget gate-এর bias, dimension $n_h \times 1$
- f_t = forget gate vector, dimension $n_h \times 1$, প্রতিটি উপাদান (0,1)-এ

১.৪.৪ কেন Sigmoid, কেন tanh নয়?

Gate-এর কাজ হলো একটি **ছাঁকনি (filter)** হিসেবে কাজ করা। ছাঁকনির মান ঋণাত্মক হওয়ার কোনো অর্থ নেই – “-0.7 পরিমাণ তথ্য রাখো” অর্থহীন। Sigmoid-এর range ঠিক (0,1), যা “শতকরা কত অংশ রাখব” ধারণাটির সঙ্গে খুব মিলে। অপরদিকে tanh-এর range (-1,1), যা তথ্যের **মান (value)** প্রকাশের জন্য উপযুক্ত, নিয়ন্ত্রণের জন্য নয়।

১.৪.৫ Dimension Tracking

ধরা যাক $n_x = 2$ (input dimension) এবং $n_h = 2$ (hidden dimension)। তাহলে:

$$[h_{t-1}, x_t] \in \mathbb{R}^{4 \times 1}, \quad W_f \in \mathbb{R}^{2 \times 4}, \quad b_f \in \mathbb{R}^{2 \times 1}$$

Matrix multiplication-এর dimension যাচাই:

$$\begin{matrix} (2 \times 4) & \cdot & (4 \times 1) & = & (2 \times 1) & + & (2 \times 1) & \Rightarrow & f_t \in \mathbb{R}^{2 \times 1} \\ \check{W}_f & & [h_{t-1}, x_t] & & \text{result} & & \check{b}_f & & \end{matrix}$$

১.৪.৬ Numerical Example (হাতে-কলমে)

দেওয়া আছে:

$$h_{t-1} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \quad x_t = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}, \quad c_{t-1} = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}$$

ধাপ ১ — Concatenation:

$$z_t = [h_{t-1}, x_t] = \begin{bmatrix} 0.1 \\ -0.2 \\ 1.0 \\ 0.5 \end{bmatrix}$$

ধাপ ২ — Weight ও bias:

$$W_f = \begin{bmatrix} 0.5 & 0.1 & 0.4 & -0.2 \\ -0.3 & 0.2 & 0.6 & 0.1 \end{bmatrix}, \quad b_f = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

ধাপ ৩ — Pre-activation $a_f = W_f z_t + b_f$:

প্রথম row:

$$(0.5)(0.1) + (0.1)(-0.2) + (0.4)(1.0) + (-0.2)(0.5) + 0.1$$

$$= 0.05 - 0.02 + 0.40 - 0.10 + 0.10 = 0.43$$

দ্বিতীয় row:

$$(-0.3)(0.1) + (0.2)(-0.2) + (0.6)(1.0) + (0.1)(0.5) - 0.1$$

$$= -0.03 - 0.04 + 0.60 + 0.05 - 0.10 = 0.48$$

$$a_f = \begin{bmatrix} 0.43 \\ 0.48 \end{bmatrix}$$

ধাপ ৪ – Sigmoid প্রয়োগ:

$$\sigma(0.43) = \frac{1}{1 + e^{-0.43}} = \frac{1}{1 + 0.6505} = 0.6059$$

$$\sigma(0.48) = \frac{1}{1 + e^{-0.48}} = \frac{1}{1 + 0.6188} = 0.6177$$

$$f_t = \begin{bmatrix} 0.6059 \\ 0.6177 \end{bmatrix}$$

ধাপ ৫ – C_{t-1} filter করা:

$$f_t \odot C_{t-1} = \begin{bmatrix} 0.6059 \\ 0.6177 \end{bmatrix} \odot \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix} = \begin{bmatrix} 0.6059 \times 0.3 \\ 0.6177 \times 0.6 \end{bmatrix} = \begin{bmatrix} 0.1818 \\ 0.3706 \end{bmatrix}$$

১.৪.৭ Interpretation (ফলাফলের ব্যাখ্যা)

$f_t \approx [0.61, 0.62]$ মানে হলো network সিদ্ধান্ত নিয়েছে পুরনো স্মৃতির **প্রায় ৬১% ও ৬২% অংশ রেখে দেবে**, বাকি ৩৯% ও ৩৮% ভুলে যাবে। প্রথম উপাদানে ০.৩ কমে ০.১৮১৮ হয়েছে – অর্থাৎ তথ্যটি দুর্বল হয়েছে কিন্তু মুছে যায়নি।

Important Note: f_t -এর মান ১-এর কাছাকাছি হলে Cell State প্রায় অপরিবর্তিত থাকে – এটাই দীর্ঘমেয়াদি স্মৃতি সংরক্ষণের মূল কৌশল। বাস্তবে b_f -কে ধনাত্মক মান (যেমন ১.০) দিয়ে initialize করা হয়, যাতে training-এর শুরুতে gate খোলা থাকে এবং network প্রথমেই সবকিছু ভুলে না যায়।

১.৫ Input Gate এবং Candidate Cell State

১.৫.১ দুটি আলাদা অংশ কেন?

নতুন তথ্য যোগ করার সিদ্ধান্তটি আসলে দুটি ভিন্ন প্রশ্ন:

১. “কী নতুন তথ্য যোগ করা যেতে পারে?” → উত্তর দেয় **Candidate Cell State $\tilde{C}_t$** (মান নির্ধারণ, $\tanh$)
২. “তার কতটুকু আসলে যোগ করব?” → উত্তর দেয় **Input Gate i_t** (পরিমাণ নির্ধারণ, σ)

এই দায়িত্ব বিভাজনই LSTM-কে নমনীয় করে: content ও amount আলাদাভাবে শেখা যায়।

১.৫.২ Equations

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \left(\text{কতটুকু} - \text{range}(0,1) \right)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad \left(\text{কী} - \text{range}(-1,1) \right)$$

$\tilde{C}_t$ -তে $\tanh$ ব্যবহারের কারণ: নতুন তথ্য স্মৃতিকে **বাড়াতেও পারে, কমাতেও পারে**। ঋণাত্মক মান দরকার, তাই zero-centered $\tanh$ ।

১.৫.৩ Numerical Example

একই $z_t = [0.1, -0.2, 1.0, 0.5]^T$ ব্যবহার করে:

$$W_i = \begin{bmatrix} 0.2 & -0.4 & 0.3 & 0.5 \\ 0.1 & 0.3 & -0.2 & 0.4 \end{bmatrix}, \quad b_i = \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

Input Gate গণনা:

Row 1: $(0.2)(0.1) + (-0.4)(-0.2) + (0.3)(1.0) + (0.5)(0.5) + 0.0 = 0.02 + 0.08 + 0.30 + 0.25 = 0.65$

Row 2: $(0.1)(0.1) + (0.3)(-0.2) + (-0.2)(1.0) + (0.4)(0.5) + 0.1 = 0.01 - 0.06 - 0.20 + 0.20 + 0.10 = 0.05$

$$a_i = \begin{bmatrix} 0.65 \\ 0.05 \end{bmatrix} \Rightarrow i_t = \begin{bmatrix} \sigma(0.65) \\ \sigma(0.05) \end{bmatrix} = \begin{bmatrix} 0.6570 \\ 0.5125 \end{bmatrix}$$

Candidate Cell State গণনা:

$$W_c = \begin{bmatrix} 0.4 & 0.2 & -0.3 & 0.1 \\ 0.3 & -0.1 & 0.5 & 0.2 \end{bmatrix}, \quad b_c = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

Row 1: $(0.4)(0.1) + (0.2)(-0.2) + (-0.3)(1.0) + (0.1)(0.5) + 0.05 = 0.04 - 0.04 - 0.30 + 0.05 + 0.05 = -0.20$

Row 2: $(0.3)(0.1) + (-0.1)(-0.2) + (0.5)(1.0) + (0.2)(0.5) - 0.05 = 0.03 + 0.02 + 0.50 + 0.10 - 0.05 = 0.60$

$$a_c = \begin{bmatrix} -0.20 \\ 0.60 \end{bmatrix} \Rightarrow \tilde{C}_t = \begin{bmatrix} \tanh(-0.20) \\ \tanh(0.60) \end{bmatrix} = \begin{bmatrix} -0.1974 \\ 0.5371 \end{bmatrix}$$

যোগ হতে যাওয়া প্রকৃত তথ্য:

$$i_t \odot \tilde{C}_t = \begin{bmatrix} 0.6570 \times (-0.1974) \\ 0.5125 \times 0.5371 \end{bmatrix} = \begin{bmatrix} -0.1297 \\ 0.2752 \end{bmatrix}$$

১.৫.৪ Interpretation

$\tilde{C}_t$ -এর প্রথম উপাদান ঋণাত্মক (-0.1974) – অর্থাৎ network এই dimension-এ স্মৃতির মান কমাতে চাইছে। Input gate 0.6570 হওয়ায় সেই প্রস্তাবের প্রায় ৬৬% কার্যকর হয়েছে (-0.1297)। দ্বিতীয় উপাদানে প্রস্তাব ছিল $+0.5371$, gate ৫১% খোলা, তাই যোগ হবে $+0.2752$ । ## ১.৬ Cell State Update – LSTM-এর হৃদয়

১.৬.১ Equation

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

এই একটি সমীকরণই LSTM-এর সবচেয়ে গুরুত্বপূর্ণ অংশ। এখানে দুটি ভিন্ন কাজ একসঙ্গে ঘটছে:

$$\begin{array}{ccc} f_t \odot C_{t-1} & + & i_t \odot \tilde{C}_t \\ \text{(ক) পুরনো স্মৃতির নিয়ন্ত্রিত সংরক্ষণ} & & \text{(খ) নতুন তথ্যের নিয়ন্ত্রিত সংযোজন} \end{array}$$

১.৬.২ কেন এই গঠনটি এত গুরুত্বপূর্ণ?

(১) **Additive Update:** নতুন Cell State পুরনোটিকে প্রতিস্থাপন করে না, বরং তার সঙ্গে যোগ হয়। Vanilla RNN-এ ছিল $h_t = \tanh(W h_{t-1} + \dots)$ – সম্পূর্ণ overwrite। LSTM-এ হলো $C_t = (\text{স্কেল করা পুরনো}) + (\text{নতুন})$ ।

(২) **Gradient Highway:** পরে দেখাব, $\frac{\partial C_t}{\partial C_{t-1}} = f_t$ – এখানে কোনো weight matrix বা activation derivative নেই। তাই $f_t \approx 1$ হলে gradient প্রায় অক্ষত অবস্থায় বহু timestep পিছিয়ে যেতে পারে।

(৩) **Four Behaviours:** f_t ও i_t -এর মানভেদে cell চারটি আচরণ শিখতে পারে:

f_t	i_t	আচরণ
≈ 1	≈ 0	পুরনো স্মৃতি অপরিবর্তিত রাখো (long-term storage)
≈ 1	≈ 1	পুরনো রাখো এবং নতুন যোগ করো (accumulate)
≈ 0	≈ 1	পুরনো মুছে নতুন লেখো (reset & write)
≈ 0	≈ 0	সব মুছে দাও (clear)

১.৬.৩ Complete Numerical Example

আগের হিসাব থেকে আমাদের কাছে আছে:

$$f_t = \begin{bmatrix} 0.6059 \\ 0.6177 \end{bmatrix}, C_{t-1} = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}, i_t = \begin{bmatrix} 0.6570 \\ 0.5125 \end{bmatrix}, \tilde{C}_t = \begin{bmatrix} -0.1974 \\ 0.5371 \end{bmatrix}$$

ধাপ ১ – পুরনো স্মৃতির অবশিষ্টাংশ:

$$f_t \odot C_{t-1} = \begin{bmatrix} 0.6059 \times 0.3 \\ 0.6177 \times 0.6 \end{bmatrix} = \begin{bmatrix} 0.1818 \\ 0.3706 \end{bmatrix}$$

ধাপ ২ – নতুন তথ্যের অবদান:

$$i_t \odot \tilde{C}_t = \begin{bmatrix} 0.6570 \times (-0.1974) \\ 0.5125 \times 0.5371 \end{bmatrix} = \begin{bmatrix} -0.1297 \\ 0.2752 \end{bmatrix}$$

ধাপ ৩ – যোগ:

$$C_t = \begin{bmatrix} 0.1818 \\ 0.3706 \end{bmatrix} + \begin{bmatrix} -0.1297 \\ 0.2752 \end{bmatrix} = \boxed{\begin{bmatrix} 0.0521 \\ 0.6459 \end{bmatrix}}$$

১.৬.৪ Interpretation

- **Dimension 1:** $C_{t-1} = 0.3$ থেকে $C_t = 0.0521$ – অর্থাৎ এই dimension-এ সঞ্চিত তথ্য প্রায় মুছে গেছে। কারণ forget gate কিছু কেটেছে এবং নতুন candidate খণাত্মক হয়ে আরও কমিয়েছে। এটি “পুরনো প্রসঙ্গ শেষ, নতুন প্রসঙ্গ শুরু” ধরনের ঘটনার প্রতিনিধিত্ব করতে পারে।
- **Dimension 2:** $C_{t-1} = 0.6$ থেকে $C_t = 0.6459$ – তথ্য শুধুটিকে থাকেনি, বরং শক্তিশালী হয়েছে। এটি এমন একটি feature যা network ধারাবাহিকভাবে সমর্থন পাচ্ছে (যেমন বাক্যের মূল কর্তা অপরিবর্তিত)।

Important Note: Cell State-এর মান $[-1,1]$ -এ সীমাবদ্ধ নয়। যেহেতু প্রতিটি timestep-এ যোগ হচ্ছে, তাই দীর্ঘ sequence-এ C_t বড় হতে পারে – এটি ইচ্ছাকৃত, কারণ এতে তথ্য সঞ্চয়ের ক্ষমতা বাড়ে। এজন্যই h_t তৈরির সময় $\tanh(C_t)$ দিয়ে তাকে আবার $[-1,1]$ -এ আনা হয়।

১.৭ Output Gate এবং Hidden State

১.৭.১ Why is it needed?

Cell State-এ অনেক তথ্য জমা থাকে, কিন্তু প্রতিটি মুহূর্তে সব তথ্য প্রাসঙ্গিক নয়। Output gate ঠিক করে – এই মুহূর্তে Cell State-এর কোন অংশটুকু প্রকাশ করা দরকার।

উদাহরণ: Cell State-এ কর্তার বচন (একবচন/বহুবচন) জমা আছে। কিন্তু সেই তথ্যটি কেবল ক্রিয়াপদ predict করার মুহূর্তেই দরকার, বিশেষণ predict করার সময় নয়।

১.৭.২ Equations

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

এখানে দুটি ধাপ:

1. $\tanh(C_t) \rightarrow$ Cell State-কে $[-1,1]$ -এ squash করা (normalization-এর ভূমিকা)
2. $o_t \odot (\cdot) \rightarrow$ কোন অংশ প্রকাশ পাবে তা নির্বাচন (filtering-এর ভূমিকা)

১.৭.৩ Dimension Tracking

$$W_o \in \mathbb{R}^{n_h \times (n_h + n_x)} = \mathbb{R}^{2 \times 4}, \quad b_o \in \mathbb{R}^{2 \times 1}, \quad o_t \in \mathbb{R}^{2 \times 1}, \quad h_t \in \mathbb{R}^{2 \times 1}$$

লক্ষণীয়: h_t -এর dimension সর্বদা C_t -এর সমান, কারণ element-wise multiplication-এ dimension অপরিবর্তিত থাকে।

১.৭.৪ Numerical Example

$$W_o = \begin{bmatrix} 0.1 & 0.5 & 0.2 & -0.3 \\ 0.4 & -0.2 & 0.1 & 0.3 \end{bmatrix}, \quad b_o = \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}$$

Row 1: $(0.1)(0.1) + (0.5)(-0.2) + (0.2)(1.0) + (-0.3)(0.5) + 0.1 = 0.01 - 0.10 + 0.20 - 0.15 + 0.10 = 0.06$

Row 2: $(0.4)(0.1) + (-0.2)(-0.2) + (0.1)(1.0) + (0.3)(0.5) + 0.0 = 0.04 + 0.04 + 0.10 + 0.15 = 0.33$

$$o_t = \begin{bmatrix} \sigma(0.06) \\ \sigma(0.33) \end{bmatrix} = \begin{bmatrix} 0.5150 \\ 0.5818 \end{bmatrix}$$

$\tanh(C_t)$ গণনা:

$$\tanh(0.0521) = 0.0520, \quad \tanh(0.6459) = 0.5689$$

$$\tanh(C_t) = \begin{bmatrix} 0.0520 \\ 0.5689 \end{bmatrix}$$

চূড়ান্ত Hidden State:

$$h_t = o_t \odot \tanh(C_t) = \begin{bmatrix} 0.5150 \times 0.0520 \\ 0.5818 \times 0.5689 \end{bmatrix} = \boxed{\begin{bmatrix} 0.0268 \\ 0.3310 \end{bmatrix}}$$

১.৭.৫ Interpretation

h_t -এর প্রথম উপাদান প্রায় শূন্য — কারণ Cell State-এই ওই dimension-এ তথ্য কম ছিল। দ্বিতীয় উপাদান 0.3310 — Cell State-এ শক্তিশালী তথ্য ছিল (0.6459), কিন্তু output gate অর্ধেকের কিছু বেশি খোলা থাকায় (0.5818) তার একাংশ প্রকাশ পেয়েছে। অর্থাৎ **তথ্যটি স্মৃতিতে রয়ে গেল, কিন্তু সম্পূর্ণ প্রকাশ পেল না** — পরবর্তী timestep-এ প্রয়োজনে আবার প্রকাশ করা যাবে। এটিই দুটি আলাদা state রাখার প্রধান সুবিধা।

১.৮ Complete LSTM Forward Pass — একত্রে

নিচে একই উদাহরণের পুরো গণনা এক জায়গায় সাজানো হলো, যাতে তথ্যপ্রবাহের ক্রম পরিষ্কার হয়।

Input:

$$x_t = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}, \quad h_{t-1} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \quad c_{t-1} = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}, \quad z_t = \begin{bmatrix} 0.1 \\ -0.2 \\ 1.0 \\ 0.5 \end{bmatrix}$$

ধাপে ধাপে ফলাফল:

ধাপ	Pre-activation	Activation	ফলাফল
1. Forget Gate	$a_f = [0.43, 0.48]$	σ	$f_t = [0.6059, 0.6177]$
2. Input Gate	$a_i = [0.65, 0.05]$	σ	$i_t = [0.6570, 0.5125]$
3. Candidate	$a_c = [-0.20, 0.60]$	$\tanh$	$\tilde{c}_t = [-0.1974, 0.5371]$
4. Old memory kept	—	$f_t \odot c_{t-1}$	$[0.1818, 0.3706]$
5. New info added	—	$i_t \odot \tilde{c}_t$	$[-0.1297, 0.2752]$
6. Cell State	—	যোগ	$C_t = [0.0521, 0.6459]$
7. Output Gate	$a_o = [0.06, 0.33]$	σ	$o_t = [0.5150, 0.5818]$
8. Squashed cell	—	$\tanh(C_t)$	$[0.0520, 0.5689]$
9. Hidden State	—	$o_t \odot \tanh(C_t)$	$h_t = [0.0268, 0.3310]$

প্রবাহচিত্র:

$$(x_t, h_{t-1}) \rightarrow f_t, i_t, \tilde{c}_t, o_t \rightarrow C_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \rightarrow h_t = o_t \odot \tanh(C_t)$$

এবং এই C_t ও h_t পরবর্তী timestep-এ input হিসেবে যাবে:

$$(x_{t+1}, h_t, C_t) \rightarrow (h_{t+1}, C_{t+1}) \rightarrow \dots$$

Important Note: চারটি gate/candidate গণনা সমান্তরালভাবে (in parallel) করা যায়, কারণ চারটিই একই input $[h_{t-1}, x_t]$ ব্যবহার করে। বাস্তব implementation-এ চারটি weight matrix একত্রে একটি বড় $4n_h \times (n_h + n_x)$ matrix হিসেবে রাখা হয় এবং একবারেই গুণ করা হয় — এতে GPU-তে গতি বাড়ে।

১.৯ LSTM Backward Propagation

এটি সবচেয়ে গুরুত্বপূর্ণ ও সূক্ষ্ম অংশ। আমরা computational graph ধরে ধাপে ধাপে এগোব।

১.৯.১ প্রয়োজনীয় Derivative-সমূহ

Function	Derivative	বৈশিষ্ট্য
$\sigma(a)$	$\sigma(a) (1 - \sigma(a))$	সর্বোচ্চ মান 0.25 (at $a = 0$)
$\tanh(a)$	$1 - \tanh^2(a)$	সর্বোচ্চ মান 1 (at $a = 0$)
$u \odot v$ (w.r.t. u)	v	element-wise, dimension অপরিবর্তিত
$u + v$	1 (উভয়ের জন্য)	gradient ছবছ কপি হয়ে যায়

Wz (w.r.t. W)	z^T	outer product তৈরি হয়
--------------------	-------	------------------------

১.৯.২ Computational Graph

Forward-এ নির্ভরতার ক্রম:

$$z_t \rightarrow \{a_f, a_i, a_c, a_o\} \rightarrow \{f_t, i_t, \tilde{C}_t, o_t\} \rightarrow C_t \rightarrow h_t \rightarrow L$$

Backward-এ ঠিক উল্টো দিকে যেতে হবে। ধরা যাক আমাদের কাছে উপরের layer থেকে এসেছে:

$$\delta h_t \equiv \frac{\partial L}{\partial h_t}$$

সতর্কতা: h_t দুই জায়গায় ব্যবহৃত হয় — (ক) এই timestep-এর output-এ, (খ) পরবর্তী timestep-এর gate গণনায়। তাই প্রকৃত gradient হলো দুটির যোগফল:

$$\delta h_t = \frac{\partial L_t}{\partial h_t} + \frac{\partial L_{>t}}{\partial h_t}$$

একইভাবে δC_t -তেও পরবর্তী timestep থেকে আসা অংশ যোগ হয়।

১.৯.৩ ধাপ ১ — Output Gate-এর দিকে

$h_t = o_t \odot \tanh(C_t)$ থেকে product rule প্রয়োগ করে:

$$\frac{\partial L}{\partial o_t} = \delta h_t \odot \tanh(C_t)$$

$$\frac{\partial L}{\partial a_o} = \frac{\partial L}{\partial o_t} \odot o_t \odot (1 - o_t) \quad (\text{sigmoid derivative})$$

১.৯.৪ ধাপ ২ — Cell State-এর দিকে (সবচেয়ে গুরুত্বপূর্ণ)

$$\delta C_t = \underbrace{\delta h_t \odot o_t \odot (1 - \tanh^2(C_t))}_{h_t\text{-এর পথ দিয়ে আসা}} + \underbrace{\delta C_{t+1} \odot f_{t+1}}_{\text{পরবর্তী timestep থেকে আসা}}$$

১.৯.৫ ধাপ ৩ — তিন Gate ও Candidate-এর দিকে

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ থেকে:

$$\frac{\partial L}{\partial f_t} = \delta C_t \odot C_{t-1}, \quad \frac{\partial L}{\partial a_f} = \frac{\partial L}{\partial f_t} \odot f_t \odot (1 - f_t)$$

$$\frac{\partial L}{\partial i_t} = \delta C_t \odot \tilde{C}_t, \quad \frac{\partial L}{\partial a_i} = \frac{\partial L}{\partial i_t} \odot i_t \odot (1 - i_t)$$

$$\frac{\partial L}{\partial \tilde{C}_t} = \delta C_t \odot i_t, \quad \frac{\partial L}{\partial a_c} = \frac{\partial L}{\partial \tilde{C}_t} \odot (1 - \tilde{C}_t^2)$$

১.৯.৬ ধাপ ৪ — Parameter Gradient

প্রতিটি gate $* \in \{f, i, C, o\}$ -এর জন্য:

$$\frac{\partial L}{\partial W_*} = \frac{\partial L}{\partial a_*} z_t^\top \quad (n_h \times 1)(1 \times (n_h + n_x)) = n_h \times (n_h + n_x) \checkmark$$

$$\frac{\partial L}{\partial b_*} = \frac{\partial L}{\partial a_*}$$

Note: যেহেতু একই W_* সব timestep-এ ব্যবহৃত হয়, প্রকৃত gradient হলো সব timestep-এর যোগফল:

$$\frac{\partial L}{\partial W_*} = \sum_{t=1}^T \frac{\partial L}{\partial a_*^{(t)}} z_t^\top$$

১.৯.৭ ধাপ ৫ — পূর্ববর্তী state-এর দিকে gradient

$$\frac{\partial L}{\partial C_{t-1}} = \delta C_t \odot f_t$$

$$\frac{\partial L}{\partial z_t} = \sum_{* \in \{f, i, C, o\}} W_*^\top \frac{\partial L}{\partial a_*}$$

এবং $z_t = [h_{t-1}, x_t]$ হওয়ায় এই vector-কে ভাগ করে নিতে হয়:

$$\frac{\partial L}{\partial h_{t-1}} = \left(\frac{\partial L}{\partial z_t} \right)_{1:n_h}, \quad \frac{\partial L}{\partial x_t} = \left(\frac{\partial L}{\partial z_t} \right)_{n_h+1:n_h+n_x}$$

১.৯.৮ Numerical Backward Example

ধরা যাক উপরের forward pass-এর পর loss থেকে এসেছে:

$$\delta h_t = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

এবং সরলতার জন্য ধরি t শেষ timestep, তাই $\delta C_{t+1} = 0$

ধাপ ১ — Output gate gradient:

$$\frac{\partial L}{\partial o_t} = \delta h_t \odot \tanh(C_t) = \begin{bmatrix} 0.1 \times 0.0520 \\ -0.2 \times 0.5689 \end{bmatrix} = \begin{bmatrix} 0.0052 \\ -0.1138 \end{bmatrix}$$

$$\frac{\partial L}{\partial a_o} = \begin{bmatrix} 0.0052 \times 0.5150 \times (1 - 0.5150) \\ -0.1138 \times 0.5818 \times (1 - 0.5818) \end{bmatrix} = \begin{bmatrix} 0.0013 \\ -0.0277 \end{bmatrix}$$

ধাপ ২ — Cell state gradient:

$1 - \tanh^2(C_t)$ গণনা: $1 - (0.0520)^2 = 0.9973$ এবং $1 - (0.5689)^2 = 0.6764$

$$\delta C_t = \begin{bmatrix} 0.1 \times 0.5150 \times 0.9973 \\ -0.2 \times 0.5818 \times 0.6764 \end{bmatrix} = \begin{bmatrix} 0.0514 \\ -0.0787 \end{bmatrix}$$

ধাপ ৩ — Gate gradient:

$$\frac{\partial L}{\partial f_t} = \delta C_t \odot C_{t-1} = \begin{bmatrix} 0.0514 \times 0.3 \\ -0.0787 \times 0.6 \end{bmatrix} = \begin{bmatrix} 0.0154 \\ -0.0472 \end{bmatrix}$$

$$\frac{\partial L}{\partial a_f} = \begin{bmatrix} 0.0154 \times 0.6059 \times 0.3941 \\ -0.0472 \times 0.6177 \times 0.3823 \end{bmatrix} = \begin{bmatrix} 0.0037 \\ -0.0112 \end{bmatrix}$$

$$\frac{\partial L}{\partial i_t} = \delta C_t \odot \tilde{C}_t = \begin{bmatrix} 0.0514 \times (-0.1974) \\ -0.0787 \times 0.5371 \end{bmatrix} = \begin{bmatrix} -0.0101 \\ -0.0423 \end{bmatrix}$$

$$\frac{\partial L}{\partial \tilde{C}_t} = \delta C_t \odot i_t = \begin{bmatrix} 0.0514 \times 0.6570 \\ -0.0787 \times 0.5125 \end{bmatrix} = \begin{bmatrix} 0.0337 \\ -0.0403 \end{bmatrix}$$

$$\frac{\partial L}{\partial a_c} = \frac{\partial L}{\partial \tilde{C}_t} \odot (1 - \tilde{C}_t^2) = \begin{bmatrix} 0.0337 \times 0.9610 \\ -0.0403 \times 0.7115 \end{bmatrix} = \begin{bmatrix} 0.0324 \\ -0.0287 \end{bmatrix}$$

ধাপ ৪ — Weight gradient (উদাহরণস্বরূপ W_f):

$$\begin{aligned} \frac{\partial L}{\partial W_f} &= \frac{\partial L}{\partial a_f} z_t^T = \begin{bmatrix} 0.0037 \\ -0.0112 \end{bmatrix} \begin{bmatrix} 0.1 & -0.2 & 1.0 & 0.5 \end{bmatrix} \\ &= \begin{bmatrix} 0.00037 & -0.00074 & 0.0037 & 0.0018 \\ -0.00112 & 0.00223 & -0.0112 & -0.0056 \end{bmatrix} \end{aligned}$$

Dimension যাচাই: $(2 \times 1)(1 \times 4) = (2 \times 4) \checkmark$ — W_f -এর dimension-এর সঙ্গে মিলে গেছে।

ধাপ ৫ — পূর্ববর্তী Cell State-এর দিকে:

$$\frac{\partial L}{\partial C_{t-1}} = \delta C_t \odot f_t = \begin{bmatrix} 0.0514 \times 0.6059 \\ -0.0787 \times 0.6177 \end{bmatrix} = \begin{bmatrix} 0.0311 \\ -0.0486 \end{bmatrix}$$

১.৯.৯ কেন LSTM Vanishing Gradient কিছুটা মোকাবিলা করতে পারে?

এটিই পুরো architecture-এর যৌক্তিক ভিত্তি। তুলনাটি লক্ষ করুন:

Vanilla RNN-এ:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2) \cdot W_{hh}$$

T ধাপ পিছিয়ে যেতে এই matrix-এর T গুণফল লাগে। এতে দুটি ক্ষয়কারী উপাদান বারবার আসে: $\tanh' < 1$ এবং W_{hh} -এর eigenvalue।

LSTM-এ (Cell State পথে):

$$\frac{\partial C_t}{\partial C_{t-1}} = \text{diag}(f_t)$$

$$\Rightarrow \frac{\partial C_t}{\partial C_k} \approx \prod_{j=k+1}^t \text{diag}(f_j)$$

এখানে কোনো weight matrix নেই, কোনো activation derivative নেই — শুধু forget gate-এর মান।

পরিণতি: যদি network শেখে যে একটি নির্দিষ্ট তথ্য দীর্ঘ সময় ধরে রাখা দরকার, তাহলে সে সেই dimension-এ $f_j \approx 1$ রাখবে। তখন:

$$\prod_{j=k+1}^t f_j \approx 1^{(t-k)} = 1$$

অর্থাৎ gradient প্রায় অবিকৃত অবস্থায় বহু timestep পিছিয়ে যেতে পারে। **সংখ্যিক তুলনা:** 20 timestep পিছিয়ে যেতে —

- Vanilla RNN (0.5 হারে ক্ষয়): $(0.5)^{20} \approx 9.5 \times 10^{-7}$ (কার্যত শূন্য)
- LSTM ($f_j = 0.99$): $(0.99)^{20} \approx 0.818$ (৮২% অক্ষত)

Important Note (একটি সং সীমাবদ্ধতা): LSTM vanishing gradient সম্পূর্ণভাবে দূর করে না, শুধু mitigate বা লাঘব করে। যদি f_j -এর মান 1-এর চেয়ে লক্ষণীয়ভাবে ছোট হয় (যেমন 0.5), তাহলে LSTM-এও gradient exponentially ক্ষয় পাবে। LSTM-এর অবদান হলো: এই ক্ষয়ের হার এখন **শেখযোগ্য (learnable)** — network নিজেই ঠিক করতে পারে কোন তথ্য কতদিন রাখবে। এছাড়াও h_t -এর পথে এখনও weight multiplication আছে, সেখানে exploding gradient ঘটতে পারে, তাই gradient clipping এখনও প্রয়োজনীয়।

১.১০ LSTM Parameter Calculation

১.১০.১ Derivation (উৎপত্তি)

LSTM-এ চারটি আলাদা neural network layer আছে: forget gate, input gate, candidate, output gate। প্রতিটির গঠন একই:

$$\text{layer} = \sigma \text{ বা } \tanh(W_*[h_{t-1}, x_t] + b_*)$$

একটি layer-এর parameter:

- Input vector-এর দৈর্ঘ্য: $n_h + n_x$ (concatenation-এর কারণে)
- Output vector-এর দৈর্ঘ্য: n_h
- Weight matrix: $W_* \in \mathbb{R}^{n_h \times (n_h + n_x)} \rightarrow$ parameter সংখ্যা $n_h(n_h + n_x) = n_h n_x + n_h^2$
- Bias: $b_* \in \mathbb{R}^{n_h} \rightarrow$ parameter সংখ্যা n_h

$$\text{এক layer} = n_x n_h + n_h^2 + n_h$$

চারটি layer থাকায়:

$$\text{Total}_{\text{LSTM}} = 4(n_x n_h + n_h^2 + n_h)$$

সমতুল্য রূপ (Keras-এ এভাবেই হিসাব হয়):

$$\text{Total} = 4 n_h (n_x + n_h + 1)$$

১.১০.২ Numerical Example ১ (আমাদের ছোট উদাহরণ)

$$n_x = 2, n_h = 2:$$

$$4(2 \times 2 + 2^2 + 2) = 4(4 + 4 + 2) = 4 \times 10 = \boxed{40}$$

যাচাই: ৪টি weight matrix, প্রতিটি $2 \times 4 = 8 \rightarrow 32$; ৪টি bias, প্রতিটি $2 \rightarrow 8$; মোট ৪০ ✓

১.১০.৩ Numerical Example ২ (বাস্তবসম্মত আকার)

একটি text classification model: Embedding dimension $n_x = 100$, LSTM units $n_h = 128$ ।

$$\begin{aligned} 4(100 \times 128 + 128^2 + 128) &= 4(12,800 + 16,384 + 128) \\ &= 4 \times 29,312 = \boxed{117,248} \end{aligned}$$

১.১০.৪ Vanilla RNN-এর সঙ্গে তুলনা

একই n_x, n_h -এর জন্য Vanilla RNN-এর parameter:

$$n_x n_h + n_h^2 + n_h = 29,312$$

অর্থাৎ LSTM-এর parameter সংখ্যা ঠিক **৪ গুণ**। এটিই LSTM-এর প্রধান মূল্য (cost): বেশি memory, বেশি computation, বেশি training data-র প্রয়োজন। এই খরচ কমানোর প্রচেষ্টা থেকেই পরবর্তী architecture — **GRU** — এসেছে।

২. PART 2 — Gated Recurrent Unit (GRU)

২.১ কেন GRU?

LSTM long-term dependency সমস্যার একটি কার্যকর সমাধান দিয়েছিল, কিন্তু তার সঙ্গে এসেছিল কিছু খরচ:

LSTM-এর খরচ	ব্যাখ্যা
৪ গুণ parameter	চারটি আলাদা weight matrix ও bias
দুটি state	C_t ও h_t উভয়ই বহন ও সংরক্ষণ করতে হয়
বেশি computation	প্রতি timestep-এ ৪টি matrix multiplication
Overfitting-এর ঝুঁকি	ছোট dataset-এ বেশি parameter মানে বেশি overfitting
ধীর training	বেশি gradient গণনা, বেশি memory transfer

২০১৪ সালে Cho ও সহকর্মীরা প্রশ্ন তুললেন: LSTM-এর সব উপাদান কি আসলেই অপরিহার্য? তাঁদের পর্যবেক্ষণ দুটি:

(১) Forget ও Input gate পরস্পরের পরিপূরক। বাস্তব প্রয়োগে দেখা যায়, যখন network পুরনো তথ্য ভুলতে চায় (f_t ছোট), তখনই সে নতুন তথ্য নিতে চায় (i_t বড়)। তাহলে দুটি আলাদা gate না রেখে একটি gate দিয়ে দুটি কাজই করানো যায়:

$$f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \rightarrow (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

LSTM: দুটি স্বাধীন gate GRU: একটি gate, পরিপূরক ভাগ

(২) দুটি আলাদা state কি প্রয়োজন? h_t যদি C_t -এরই একটি filtered রূপ হয়, তবে দুটি আলাদা vector না রেখে একটিমাত্র hidden state দিয়ে কাজ চালানো যায় কি না — GRU-এর উত্তর: হ্যাঁ, যায়।

Logical Transition:

LSTM (৪ layer, ২ state) → gate একীভূতকরণ + state একীভূতকরণ → GRU (৩ layer, ১ state)

২.২ GRU Core Idea

GRU-তে তিনটি মূল উপাদান:

(ক) Single Hidden State (h_t)

কোনো আলাদা Cell State নেই। h_t -ই একই সঙ্গে দীর্ঘমেয়াদি স্মৃতি এবং প্রকাশযোগ্য output — দুটি ভূমিকা পালন করে। ফলে state বহন করার খরচ অর্ধেক হয়ে যায়।

(খ) Reset Gate (r_t)

নির্ধারণ করে — **নতুন candidate তৈরির সময়** পুরনো স্মৃতির কতটুকু ব্যবহার করা হবে। $r_t \approx 0$ মানে “পুরনো প্রসঙ্গ ভুলে গিয়ে কেবল বর্তমান input থেকে নতুন করে শুরু করো”। এটি স্বল্পমেয়াদি সম্পর্ক ধরার জন্য উপযোগী।

লক্ষণীয়: reset gate পুরনো স্মৃতি **মুছে ফেলে না**; এটি কেবল candidate গণনার সময় পুরনো স্মৃতির প্রভাব কমায়। h_{t-1} চূড়ান্ত সমীকরণে আলাদাভাবেও থাকে।

(গ) Update Gate (z_t)

LSTM-এর forget ও input gate-এর সম্মিলিত ভূমিকা পালন করে। এটি একটি **interpolation coefficient**: পুরনো স্মৃতি ও নতুন candidate-এর মধ্যে ভারসাম্য নির্ধারণ করে। $z_t = 0$ মানে “পুরোপুরি পুরনো রাখো”, $z_t = 1$ মানে “পুরোপুরি নতুন নাও”।

(ঘ) Candidate Hidden State ($\tilde{h}_t$)

প্রস্তাবিত নতুন স্মৃতি, tanh দিয়ে তৈরি, range $(-1,1)$ ।

সম্পূর্ণ GRU সমীকরণসমূহ:

$$\begin{aligned} r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) \\ z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \\ \tilde{h}_t &= \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

২.৩ GRU Architecture

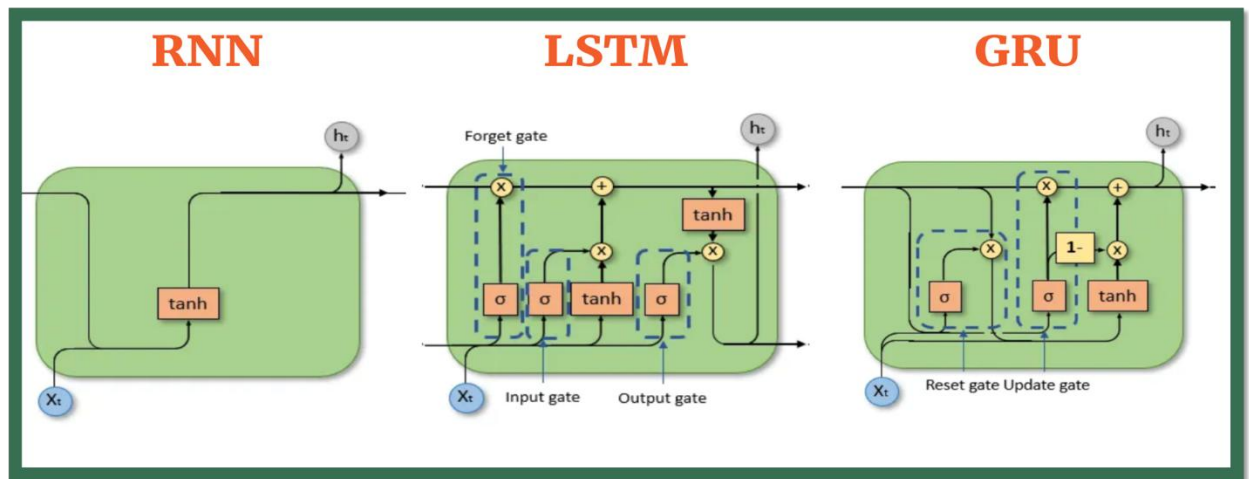
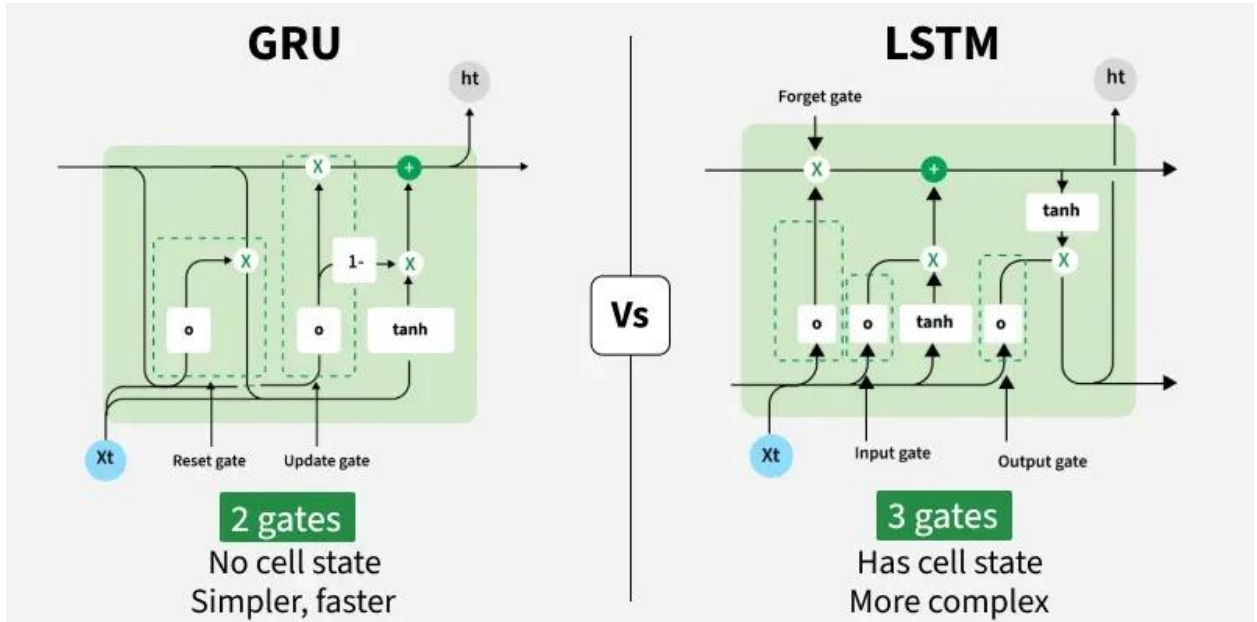
[DIAGRAM 3: Complete GRU Cell Architecture]

Purpose: একটি GRU cell-এর ভেতরে h_{t-1} ও x_t থেকে $r_t, z_t, \tilde{h}_t$ হয়ে h_t পর্যন্ত সম্পূর্ণ তথ্যপ্রবাহ দেখানো, এবং LSTM-এর তুলনায় সরলতা বোঝানো।

Elements:

- একটি বড় rounded rectangle (GRU Cell)। বাম দিক থেকে একটিমাত্র অনুভূমিক input তীর: h_{t-1} (কোনো C_{t-1} নেই — এটিই LSTM-এর সঙ্গে প্রধান দৃশ্যমান পার্থক্য)। নিচ থেকে উল্লম্ব তীর: x_t ।
- ডান দিক থেকে একটিমাত্র output তীর: h_t ; এবং তার একটি শাখা উপরের দিকে y_t ।
- h_{t-1} ও x_t concatenate হয়ে দুটি sigmoid box-এ যাবে:
 - Box $\sigma \rightarrow$ **Reset Gate**, caption: $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$
 - Box $\sigma \rightarrow$ **Update Gate**, caption: $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$

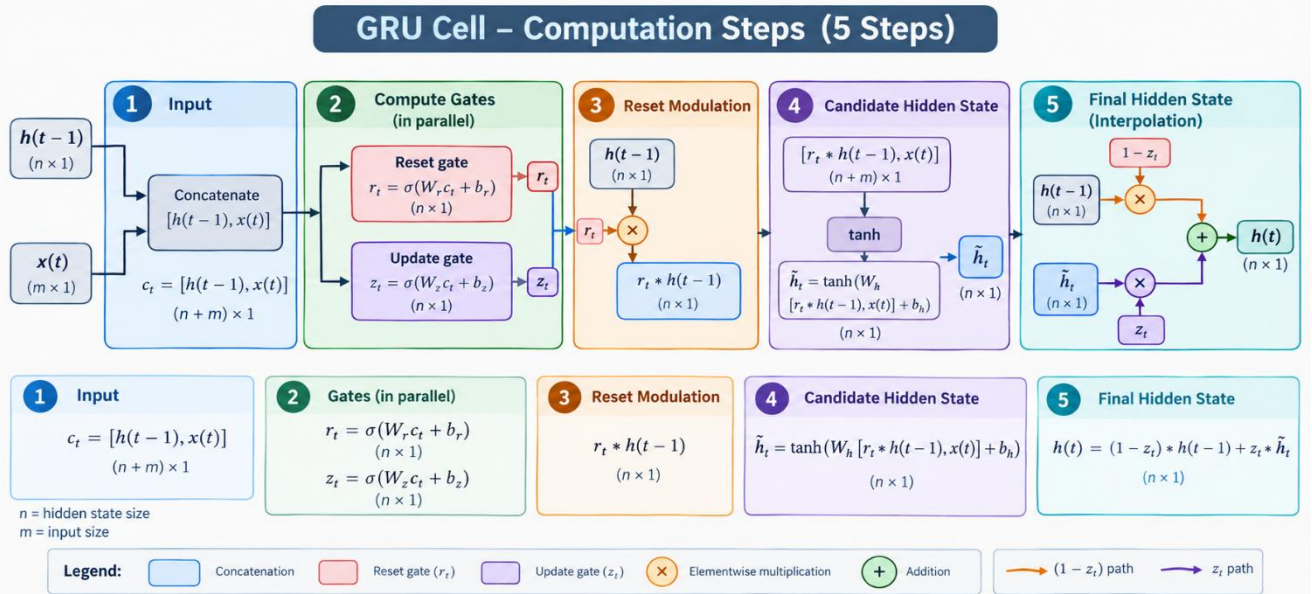
- r_t একটি $\otimes$ node-এ h_{t-1} -এর সঙ্গে গুণ হবে; ফলাফল $r_t \odot h_{t-1}$ -কে x_t -এর সঙ্গে concatenate করে একটি tanh box-এ পাঠানো হবে $\rightarrow$ **Candidate**, caption: $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$ ।
- ডান পাশে একটি interpolation অংশ: z_t থেকে দুটি শাখা — একটি সরাসরি $\otimes$ node-এ $\tilde{h}_t$ -এর সঙ্গে; আরেকটি একটি $(1 - \cdot)$ box পেরিয়ে $\otimes$ node-এ h_{t-1} -এর সঙ্গে। দুটি ফলাফল একটি $\oplus$ node-এ যোগ হয়ে h_t তৈরি করবে।
- $(1 - z_t)$ ও z_t শাখা দুটি ভিন্ন রঙে (যেমন কমলা ও নীল) আঁকতে হবে, পাশে label: “old memory share” ও “new memory share”।
- নিচে legend: σ = sigmoid, $\tanh$ = tanh, $\otimes$ = elementwise multiply, $\oplus$ = add, $1 -$ = complement.



Purpose: GRU-এর গণনা কোন ক্রমে ঘটে (dependency order) তা ধাপে ধাপে দেখানো — বিশেষ করে r_t কেন $\tilde{h}_t$ -এর আগে গণনা করতে হয়।

Elements:

- বাম থেকে ডানে পাঁচটি ধাপের flowchart, প্রতিটি একটি বাক্সে:
 - Step 1: Input** — h_{t-1}, x_t (ধূসর বাক্স)
 - Step 2: Gates** — r_t ও z_t সমান্তরালভাবে গণনা (দুটি পাশাপাশি বাক্স, একই স্তরে, তীর দিয়ে দেখানো যে দুটি স্বাধীন)
 - Step 3: Filtering** — $r_t \odot h_{t-1}$ (সবুজ বাক্স)
 - Step 4: Candidate** — $\tilde{h}_t$ (হলুদ বাক্স), Step 3 ও x_t থেকে তীর আসবে
 - Step 5: Interpolation** — $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ (নীল বাক্স), Step 2-এর z_t , Step 4 এবং মূল h_{t-1} — তিনটি থেকে তীর আসবে
- প্রতিটি বাক্সের নিচে সংশ্লিষ্ট সমীকরণ এবং dimension (যেমন $r_t \in \mathbb{R}^{n_h}$)।
- একটি লাল ডটেড তীর Step 2 থেকে Step 5-এ সরাসরি যাবে, label: “ z_t is needed only at the end”।



২.৪ Reset Gate

২.৪.১ What & Why

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Reset gate-এর কাজ: **নতুন candidate তৈরির সময় অতীতের কতটুকু প্রাসঙ্গিক তা নির্ধারণ করা।**

ধরুন একটি দীর্ঘ অনুচ্ছেদে নতুন বাক্য বা নতুন প্রসঙ্গ শুরু হলো। তখন পুরনো প্রসঙ্গ নতুন candidate তৈরিতে বাধা হয়ে দাঁড়াতে পারে। $r_t \approx 0$ করে network কার্যত বলে: “এই মুহূর্তের নতুন তথ্যটি অতীত থেকে স্বাধীনভাবে তৈরি করো।” তখন:

$$\tilde{h}_t \approx \tanh(W_h[0, x_t] + b_h)$$

অর্থাৎ cell একটি সাধারণ feedforward unit-এর মতো আচরণ করে – অতীত-নিরপেক্ষ।

২.৪.২ Dimension Tracking (বিস্তারিত)

ধরা যাক একটি বাস্তবসম্মত configuration:

$$x_t \in \mathbb{R}^{3 \times 1} \ (n_x = 3), \quad h_{t-1} \in \mathbb{R}^{4 \times 1} \ (n_h = 4)$$

Concatenation:

$$[h_{t-1}, x_t] \in \mathbb{R}^{(4+3) \times 1} = \mathbb{R}^{7 \times 1}$$

(সমতুল্যভাবে row-vector রূপে: $h_{t-1} \in \mathbb{R}^{1 \times 4}$, $x_t \in \mathbb{R}^{1 \times 3}$ হলে $[h_{t-1}, x_t] \in \mathbb{R}^{1 \times 7}$)

Weight ও bias:

$$W_r \in \mathbb{R}^{4 \times 7}, \quad b_r \in \mathbb{R}^{4 \times 1}$$

Multiplication যাচাই:

$$(4 \times 7) \cdot (7 \times 1) = (4 \times 1) + (4 \times 1) = r_t \in \mathbb{R}^{4 \times 1} \checkmark$$

$$\check{W}_r \quad [h_{t-1}, x_t]$$

একটি gate-এর parameter count:

$$4 \times 7 + 4 = 28 + 4 = 32$$

$$\check{W}_r \quad \check{b}_r$$

তিনটি layer (r, z, h) থাকায় এই configuration-এ মোট: $3 \times 32 = 96$ parameter।

২.৫ Reset Filtering — $r_t \odot h_{t-1}$

এই ধাপে reset gate আসলে কাজ করে। একটি ছোট numerical উদাহরণ ($n_h = 2$):

$$h_{t-1} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad r_t = \begin{bmatrix} 0.6964 \\ 0.4182 \end{bmatrix}$$

(নিচের ২.৬ অনুচ্ছেদে r_t -এর পূর্ণ গণনা দেখানো হয়েছে)

$$r_t \odot h_{t-1} = \begin{bmatrix} 0.6964 \times 0.5 \\ 0.4182 \times (-0.3) \end{bmatrix} = \begin{bmatrix} 0.3482 \\ -0.1255 \end{bmatrix}$$

Interpretation: প্রথম উপাদানে পুরনো স্মৃতির প্রায় ৭০% রাখা হয়েছে ($0.5 \rightarrow 0.3482$) – অর্থাৎ candidate তৈরিতে এই তথ্যটি প্রাসঙ্গিক। দ্বিতীয় উপাদানে মাত্র ৪২% ($-0.3 \rightarrow -0.1255$) – এই তথ্যটি কম প্রাসঙ্গিক বলে বিবেচিত হয়েছে।

২.৬ Candidate Hidden State এবং সম্পূর্ণ Numerical Example

আমরা এখন একটি পূর্ণ GRU forward pass হাতে-কলমে করব।

২.৬.১ প্রাথমিক মান

$$h_{t-1} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad x_t = \begin{bmatrix} 1.0 \\ 0.2 \end{bmatrix}, \quad z_t^{\text{concat}} = [h_{t-1}, x_t] = \begin{bmatrix} 0.5 \\ -0.3 \\ 1.0 \\ 0.2 \end{bmatrix}$$

(এখানে $n_x = 2$, $n_h = 2$, তাই concatenated vector $\in \mathbb{R}^{4 \times 1}$)

২.৬.২ Reset Gate গণনা

$$W_r = \begin{bmatrix} 0.3 & -0.2 & 0.5 & 0.1 \\ 0.1 & 0.4 & -0.3 & 0.2 \end{bmatrix}, \quad b_r = \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}$$

Row 1: $(0.3)(0.5) + (-0.2)(-0.3) + (0.5)(1.0) + (0.1)(0.2) + 0.1 = 0.15 + 0.06 + 0.50 + 0.02 + 0.10 = 0.83$

Row 2: $(0.1)(0.5) + (0.4)(-0.3) + (-0.3)(1.0) + (0.2)(0.2) + 0.0 = 0.05 - 0.12 - 0.30 + 0.04 = -0.33$

$$r_t = \begin{bmatrix} \sigma(0.83) \\ \sigma(-0.33) \end{bmatrix} = \begin{bmatrix} 0.6964 \\ 0.4182 \end{bmatrix}$$

২.৬.৩ Update Gate গণনা

$$W_z = \begin{bmatrix} 0.2 & 0.3 & -0.1 & 0.4 \\ -0.2 & 0.1 & 0.3 & 0.2 \end{bmatrix}, \quad b_z = \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

Row 1: $(0.2)(0.5) + (0.3)(-0.3) + (-0.1)(1.0) + (0.4)(0.2) + 0.0 = 0.10 - 0.09 - 0.10 + 0.08 = -0.01$

Row 2: $(-0.2)(0.5) + (0.1)(-0.3) + (0.3)(1.0) + (0.2)(0.2) + 0.1 = -0.10 - 0.03 + 0.30 + 0.04 + 0.10 = 0.31$

$$z_t = \begin{bmatrix} \sigma(-0.01) \\ \sigma(0.31) \end{bmatrix} = \begin{bmatrix} 0.4975 \\ 0.5769 \end{bmatrix}$$

২.৬.৪ Candidate Hidden State গণনা

ধাপ ক — Reset filtering (২.৫ থেকে):

$$r_t \odot h_{t-1} = \begin{bmatrix} 0.3482 \\ -0.1255 \end{bmatrix}$$

ধাপ খ — নতুন concatenation:

$$[r_t \odot h_{t-1}, x_t] = \begin{bmatrix} 0.3482 \\ -0.1255 \\ 1.0 \\ 0.2 \end{bmatrix}$$

Important Note: লক্ষ করুন, candidate গণনায় ব্যবহৃত concatenated vector-টি gate গণনায় ব্যবহৃতটির থেকে **ভিন্ন** — এখানে h_{t-1} -এর জায়গায় $r_t \odot h_{t-1}$ বসেছে। এটি GRU-এর একটি সূক্ষ্ম কিন্তু গুরুত্বপূর্ণ বিশদ, যা backward pass-এ r_t -এর দিকে gradient পাঠাতে ভূমিকা রাখে।

ধাপ গ — Weight প্রয়োগ:

$$W_h = \begin{bmatrix} 0.4 & 0.1 & 0.2 & -0.3 \\ 0.2 & -0.4 & 0.3 & 0.1 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0.05 \\ 0.05 \end{bmatrix}$$

$$\begin{aligned} \text{Row 1: } & (0.4)(0.3482) + (0.1)(-0.1255) + (0.2)(1.0) + (-0.3)(0.2) + 0.05 \\ & = 0.1393 - 0.0125 + 0.2000 - 0.0600 + 0.0500 = 0.3167 \end{aligned}$$

$$\begin{aligned} \text{Row 2: } & (0.2)(0.3482) + (-0.4)(-0.1255) + (0.3)(1.0) + (0.1)(0.2) + 0.05 \\ & = 0.0696 + 0.0502 + 0.3000 + 0.0200 + 0.0500 = 0.4898 \end{aligned}$$

ধাপ ঘ — tanh প্রয়োগ:

$$\tilde{h}_t = \begin{bmatrix} \tanh(0.3167) \\ \tanh(0.4898) \end{bmatrix} = \begin{bmatrix} 0.3065 \\ 0.4541 \end{bmatrix}$$

২.৬.৫ Dimension যাচাই

$$(2 \times 4) \cdot (4 \times 1) = (2 \times 1) + (2 \times 1) = \tilde{h}_t \in \mathbb{R}^{2 \times 1} \checkmark$$

$$\tilde{W}_h \quad [r \odot h, x] \quad \tilde{b}_h$$

২.৭ Update Gate-এর ভূমিকা

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

z_t কী নিয়ন্ত্রণ করে — তা চূড়ান্ত সমীকরণ থেকেই স্পষ্ট:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

এটি একটি **convex combination** বা weighted average, যেখানে দুটি ওজনের যোগফল সর্বদা ১:

$$(1 - z_t) + z_t = 1$$

z_t -এর মান	$(1 - z_t)$	আচরণ	তুলনীয় LSTM অবস্থা
≈ 0	≈ 1	পুরনো স্মৃতি হুবহু বহন করে নিয়ে যাও ($h_t \approx h_{t-1}$)	$f_t \approx 1, i_t \approx 0$

≈ 1	≈ 0	পুরনো ফেলে দিয়ে নতুন candidate গ্রহণ করো	$f_t \approx 0, i_t \approx 1$
≈ 0.5	≈ 0.5	সমান ভাৱে মিশ্ৰণ	মধ্যবৰ্তী অবস্থা

Important Note: $z_t \approx 0$ অবস্থাটিই GRU-এৰ long-term memory-ৰ চাৰিকাঠি। তখন $h_t \approx h_{t-1}$ — অৰ্থাৎ তথ্য কোনো matrix multiplication বা activation ছাড়াই সরাসৰি পৰবৰ্তী timestep-এ চলে যায়। এটি LSTM-এৰ $f_t \approx 1$ অবস্থাৰ সৰাসৰি সমতুল্য, এবং একইভাবে gradient highway তৈৰি কৰে।

লক্ষণীয় একটি পাৰ্থক্য: LSTM-এ f_t ও i_t স্বাধীন, তাই “পুরনোও ৰাখো, নতুনও পূৰোপূৰি যোগ কৰো” ($f = 1, i = 1$) সম্ভব — স্মৃতি সঞ্চিত (accumulate) হতে পাৰে। GRU-তে দুটি ওজন বাধ্যতামূলকভাবে পৰিপূৰক, তাই এই accumulation সম্ভব নয়। এটি GRU-কে সৰল কৰেছে, কিন্তু কিছুটা কম নমনীয়ও কৰেছে।

২.৮ Final Hidden State — সম্পূৰ্ণ Numerical Example

আমাদেৰ কাছে আছে:

$$h_{t-1} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad z_t = \begin{bmatrix} 0.4975 \\ 0.5769 \end{bmatrix}, \quad \tilde{h}_t = \begin{bmatrix} 0.3065 \\ 0.4541 \end{bmatrix}$$

ধাপ ১ — $(1 - z_t)$ গণনা:

$$1 - z_t = \begin{bmatrix} 1 - 0.4975 \\ 1 - 0.5769 \end{bmatrix} = \begin{bmatrix} 0.5025 \\ 0.4231 \end{bmatrix}$$

ধাপ ২ — পুরনো স্মৃতিৰ অংশ:

$$(1 - z_t) \odot h_{t-1} = \begin{bmatrix} 0.5025 \times 0.5 \\ 0.4231 \times (-0.3) \end{bmatrix} = \begin{bmatrix} 0.2513 \\ -0.1269 \end{bmatrix}$$

ধাপ ৩ — নতুন candidate-এৰ অংশ:

$$z_t \odot \tilde{h}_t = \begin{bmatrix} 0.4975 \times 0.3065 \\ 0.5769 \times 0.4541 \end{bmatrix} = \begin{bmatrix} 0.1525 \\ 0.2620 \end{bmatrix}$$

ধাপ ৪ — যোগ:

$$h_t = \begin{bmatrix} 0.2513 \\ -0.1269 \end{bmatrix} + \begin{bmatrix} 0.1525 \\ 0.2620 \end{bmatrix} = \boxed{\begin{bmatrix} 0.4038 \\ 0.1350 \end{bmatrix}}$$

Interpretation

- **Dimension 1:** $h_{t-1} = 0.5 \rightarrow h_t = 0.4038$ । $z_t \approx 0.5$ হওয়ায় পুরনো ও নতুন প্ৰায় সমান ভাৱে মিশেছে; candidate (0.3065) পুরনোৰ চেয়ে ছোট হওয়ায় মান কিছুটা কমেছে।
- **Dimension 2:** $h_{t-1} = -0.3 \rightarrow h_t = +0.1350$ — চিহ্ন বদলে গেছে! কাৰণ candidate দৃঢ়ভাবে ধনাত্মক (0.4541) এবং update gate ৫৮% খোলা। অৰ্থাৎ এই dimension-এ network একটি স্পষ্ট মত-পৰিবৰ্তন কৰেছে, যা নতুন input-এৰ প্ৰবল প্ৰভাব নিৰ্দেশ কৰে।

সম্পূৰ্ণ GRU forward pass সাৰণি:

ধাপ	Pre-activation	ফলাফল
Reset Gate	$[0.83, -0.33]$	$r_t = [0.6964, 0.4182]$
Update Gate	$[-0.01, 0.31]$	$z_t = [0.4975, 0.5769]$
Reset filtering	—	$r_t \odot h_{t-1} = [0.3482, -0.1255]$
Candidate	$[0.3167, 0.4898]$	$\tilde{h}_t = [0.3065, 0.4541]$
Old share	—	$[0.2513, -0.1269]$
New share	—	$[0.1525, 0.2620]$
Final	—	$h_t = [0.4038, 0.1350]$

২.৯ GRU Backward Propagation

২.৯.১ Computational Graph

$$\{h_{t-1}, x_t\} \rightarrow \{r_t, z_t\} \rightarrow \tilde{h}_t \rightarrow h_t \rightarrow L$$

লক্ষণীয়, h_{t-1} তিনটি পথে h_t -কে প্রভাবিত করে:

1. সরাসরি: $(1 - z_t) \odot h_{t-1}$
2. Candidate-এর মাধ্যমে: $r_t \odot h_{t-1} \rightarrow \tilde{h}_t$
3. Gate-দুটির মাধ্যমে: $h_{t-1} \rightarrow r_t, z_t$

Multivariable chain rule অনুযায়ী তিনটি পথের অবদান যোগ করতে হবে।

২.৯.২ ধাপে ধাপে Derivation

ধরা যাক $\delta h_t = \frac{\partial L}{\partial h_t}$ (উপর থেকে ও পরবর্তী timestep থেকে আসা মোট gradient)।

(ক) Update gate-এর দিকে:

$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ -কে z_t দিয়ে differentiate করলে:

$$\frac{\partial h_t}{\partial z_t} = -h_{t-1} + \tilde{h}_t = \tilde{h}_t - h_{t-1}$$

$$\Rightarrow \frac{\partial L}{\partial z_t} = \delta h_t \odot (\tilde{h}_t - h_{t-1})$$

$$\frac{\partial L}{\partial a_z} = \frac{\partial L}{\partial z_t} \odot z_t \odot (1 - z_t)$$

(খ) Candidate-এর দিকে:

$$\frac{\partial L}{\partial \tilde{h}_t} = \delta h_t \odot z_t$$

$$\frac{\partial L}{\partial a_h} = \frac{\partial L}{\partial \tilde{h}_t} \odot (1 - \tilde{h}_t^2)$$

(গ) Reset gate-এর দিকে:

$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$ -এ W_h -কে দুই অংশে ভাগ করি: $W_h = [W_{h \cdot h} \ W_{h \cdot x}]$, যেখানে $W_{h \cdot h} \in \mathbb{R}^{n_h \times n_h}$ অংশটি $r_t \odot h_{t-1}$ -এর সঙ্গে গুণ হয়। তাহলে:

$$\frac{\partial L}{\partial (r_t \odot h_{t-1})} = W_{h \cdot h}^\top \frac{\partial L}{\partial a_h}$$

$$\frac{\partial L}{\partial r_t} = \frac{\partial L}{\partial (r_t \odot h_{t-1})} \odot h_{t-1}$$

$$\frac{\partial L}{\partial a_r} = \frac{\partial L}{\partial r_t} \odot r_t \odot (1 - r_t)$$

(ঘ) Parameter gradient:

$$\frac{\partial L}{\partial W_z} = \frac{\partial L}{\partial a_z} [h_{t-1}, x_t]^\top, \quad \frac{\partial L}{\partial b_z} = \frac{\partial L}{\partial a_z}$$

$$\frac{\partial L}{\partial W_r} = \frac{\partial L}{\partial a_r} [h_{t-1}, x_t]^\top, \quad \frac{\partial L}{\partial W_h} = \frac{\partial L}{\partial a_h} [r_t \odot h_{t-1}, x_t]^\top$$

(ঙ) পূর্ববর্তী hidden state-এর দিকে (তিন পথের যোগফল):

$$\frac{\partial L}{\partial h_{t-1}} = \underbrace{\delta h_t \odot (1 - z_t)}_{\text{পথ ১: সরাসরি}} + \underbrace{\frac{\partial L}{\partial (r_t \odot h_{t-1})} \odot r_t}_{\text{পথ ২: candidate-এর মাধ্যমে}} + \underbrace{W_{r \cdot h}^\top \frac{\partial L}{\partial a_r} + W_{z \cdot h}^\top \frac{\partial L}{\partial a_z}}_{\text{পথ ৩: gate-দুটির মাধ্যমে}}$$

২.৯.৩ Numerical Backward Example

ধরা যাক $\delta h_t = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$ ।

Update gate gradient:

$$\tilde{h}_t - h_{t-1} = \begin{bmatrix} 0.3065 - 0.5 \\ 0.4541 - (-0.3) \end{bmatrix} = \begin{bmatrix} -0.1935 \\ 0.7541 \end{bmatrix}$$

$$\frac{\partial L}{\partial z_t} = \begin{bmatrix} 0.1 \times (-0.1935) \\ -0.2 \times 0.7541 \end{bmatrix} = \begin{bmatrix} -0.0193 \\ -0.1508 \end{bmatrix}$$

$$\frac{\partial L}{\partial a_z} = \begin{bmatrix} -0.0193 \times 0.4975 \times 0.5025 \\ -0.1508 \times 0.5769 \times 0.4231 \end{bmatrix} = \begin{bmatrix} -0.0048 \\ -0.0368 \end{bmatrix}$$

Candidate gradient:

$$\frac{\partial L}{\partial \tilde{h}_t} = \begin{bmatrix} 0.1 \times 0.4975 \\ -0.2 \times 0.5769 \end{bmatrix} = \begin{bmatrix} 0.0498 \\ -0.1154 \end{bmatrix}$$

$$1 - \tilde{h}_t^2 = \begin{bmatrix} 1 - 0.0940 \\ 1 - 0.2062 \end{bmatrix} = \begin{bmatrix} 0.9060 \\ 0.7938 \end{bmatrix}$$

$$\frac{\partial L}{\partial a_h} = \begin{bmatrix} 0.0498 \times 0.9060 \\ -0.1154 \times 0.7938 \end{bmatrix} = \begin{bmatrix} 0.0451 \\ -0.0916 \end{bmatrix}$$

সরাসরি পথে h_{t-1} -এর gradient (পথ ১):

$$\delta h_t \odot (1 - z_t) = \begin{bmatrix} 0.1 \times 0.5025 \\ -0.2 \times 0.4231 \end{bmatrix} = \begin{bmatrix} 0.0503 \\ -0.0846 \end{bmatrix}$$

২.৯.৪ GRU-তে Gradient Flow-এর বিশ্লেষণ

সবচেয়ে গুরুত্বপূর্ণ পদটি হলো পথ ১:

$$\frac{\partial h_t}{\partial h_{t-1}}|_{\text{direct}} = \text{diag}(1 - z_t)$$

LSTM-এর $\text{diag}(f_t)$ -এর সঙ্গে এর গঠনগত মিল সম্পূর্ণ: কোনো weight matrix নেই, কোনো activation derivative নেই। তাই $z_t \approx 0$ হলে:

$$\prod_{j=k+1}^t (1 - z_j) \approx 1$$

অর্থাৎ GRU-ও একই কৌশলে vanishing gradient লাঘব করে। পার্থক্য কেবল এই যে LSTM-এ এই পথটি আলাদা Cell State-এ, আর GRU-তে তা সরাসরি hidden state-এই।

২.১০ GRU Parameter Calculation

২.১০.১ Derivation

GRU-তে তিনটি layer: reset gate, update gate, candidate। প্রতিটির input দৈর্ঘ্য $n_h + n_x$ এবং output দৈর্ঘ্য n_h :

$$\text{এক layer} = \underbrace{n_h(n_h + n_x)}_{\text{weight}} + \underbrace{n_h}_{\text{bias}} = n_x n_h + n_h^2 + n_h$$

$$\text{Total}_{\text{GRU}} = 3(n_x n_h + n_h^2 + n_h) = 3 n_h (n_x + n_h + 1)$$

২.১০.২ Numerical Examples

উদাহরণ ১ – আমাদের ছোট cell ($n_x = 2, n_h = 2$):

$$3(2 \times 2 + 2^2 + 2) = 3(4 + 4 + 2) = 3 \times 10 = \boxed{30}$$

উদাহরণ ২ – ২.৪ অনুচ্ছেদের configuration ($n_x = 3, n_h = 4$):

$$3(3 \times 4 + 4^2 + 4) = 3(12 + 16 + 4) = 3 \times 32 = \boxed{96}$$

উদাহরণ ৩ – বাস্তবসম্মত আকার ($n_x = 100, n_h = 128$):

$$3(100 \times 128 + 128^2 + 128) = 3(12,800 + 16,384 + 128) = 3 \times 29,312 = \boxed{87,936}$$

২.১০.৩ LSTM-এর সঙ্গে গাণিতিক তুলনা

$$\frac{\text{Params}_{\text{GRU}}}{\text{Params}_{\text{LSTM}}} = \frac{3(n_x n_h + n_h^2 + n_h)}{4(n_x n_h + n_h^2 + n_h)} = \frac{3}{4} = 0.75$$

অর্থাৎ n_x ও n_h -এর মান যাই হোক, GRU-তে সর্বদা LSTM-এর ঠিক ৭৫% parameter থাকে — ২৫% সাশ্রয়। উপরের উদাহরণে: 117,248 বনাম 87,936, পার্থক্য 29,312।

Important Note: parameter কম মানেই মডেল “ভালো” নয়। কম parameter → কম memory, দ্রুত training, ছোট dataset-এ কম overfitting। কিন্তু একই সঙ্গে → কম representational capacity। কোনটি ভালো হবে তা dataset, sequence-এর দৈর্ঘ্য এবং task-এর জটিলতার উপর নির্ভরশীল; empirically যাচাই করাই একমাত্র নির্ভরযোগ্য পথ।

৩. PART 3 — LSTM বনাম GRU: বিস্তারিত তুলনা

নিচের তুলনাটি কেবল বৈশিষ্ট্যের তালিকা নয়; প্রতিটি সারি একটি গঠনগত পার্থক্য এবং তার ফলাফল দেখায়।

বিষয়	LSTM	GRU
Architecture	৪টি neural layer (৩ gate + ১ candidate)	৩টি neural layer (২ gate + ১ candidate)
Gates	Forget f_t , Input i_t , Output o_t	Reset r_t , Update z_t
Memory States	দুটি: Cell State C_t ও Hidden State h_t	একটি: Hidden State h_t
Memory update	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ (স্বাধীন ওজন, যোগফল ১-এ সীমিত নয়)	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ (পরিপূরক ওজন, যোগফল সর্বদা ১)
Output নিয়ন্ত্রণ	আছে — output gate ঠিক করে স্মৃতির কতটুকু প্রকাশ পাবে	নেই — সম্পূর্ণ h_t সরাসরি প্রকাশিত হয়
Parameters	$4(n_x n_h + n_h^2 + n_h)$	$3(n_x n_h + n_h^2 + n_h)$, অর্থাৎ ৭৫%
Computation/timestep	৪টি matrix multiplication	৩টি matrix multiplication
Memory usage (runtime)	দুটি state vector সংরক্ষণ	একটি state vector সংরক্ষণ
Gradient path	$\partial C_t / \partial C_{t-1} = \text{diag}(f_t)$ — আলাদা, বিশুদ্ধ highway	$\partial h_t / \partial h_{t-1} _{\text{direct}} = \text{diag}(1 - z_t)$, তবে আরও দুটি পথের সঙ্গে মিশ্রিত
Memory accumulation	সম্ভব ($f_t \approx 1$ ও $i_t \approx 1$ একসঙ্গে)	সম্ভব নয় (ওজন পরিপূরক)
Long-term dependency	খুব দীর্ঘ sequence-এ প্রায়ই কিছুটা সুবিধাজনক (আলাদা cell state ও বেশি ধারণক্ষমতার কারণে)	মঝঝারি দৈর্ঘ্যে সাধারণত তুলনীয় ফল দেয়
Training সময়	তুলনামূলক বেশি	তুলনামূলক কম (প্রায় ২৫–৩০% দ্রুত হতে পারে)
ছোট dataset	বেশি parameter $\rightarrow$ overfitting-এর ঝুঁকি বেশি	কম parameter $\rightarrow$ সাধারণত বেশি robust
বড় dataset	বাড়তি ধারণক্ষমতা কাজে লাগাতে পারে	ধারণক্ষমতা সীমিত হতে পারে
প্রচলিত ব্যবহার	Machine translation, দীর্ঘ document modeling, speech	Real-time/embedded system, ছোট ও মঝঝারি dataset, দ্রুত prototyping

Important Note — সতর্কতা: “GRU সবসময় দ্রুত” বা “LSTM সবসময় ভালো” — দুটিই অতি-সরলীকরণ। Parameter সংখ্যা ২৫% কম হলেও প্রকৃত গতি hardware, batch size ও library implementation (যেমন cuDNN-optimized kernel)-এর উপর নির্ভর করে; কোনো কোনো ক্ষেত্রে পার্থক্য প্রায় অদৃশ্য হয়ে যায়। গবেষণায় (যেমন Chung et al., 2014; Jozefowicz et al., 2015) দেখা গেছে দুটির কর্মক্ষমতা task-ভেদে ভিন্ন হয় এবং কোনো একটি সার্বজনীনভাবে শ্রেষ্ঠ নয়। সিদ্ধান্ত নেওয়ার সঠিক পদ্ধতি হলো নিজের dataset-এ উভয়টি পরীক্ষা করে validation performance তুলনা করা।

8. PART 4 — Deep RNN / Stacked RNN

8.১ Deep RNN কী?

এতক্ষণ আমরা যে সব architecture দেখেছি (RNN, LSTM, GRU) — সেগুলোর প্রতিটিতে **একটিমাত্র recurrent layer** ছিল। **Deep RNN** বা **Stacked RNN** হলো সেই কাঠামো যেখানে একাধিক recurrent layer একটির উপর আরেকটি — অর্থাৎ **উল্লম্বভাবে (vertically)** — সাজানো থাকে।

মূল ধারণাটি সরল:

প্রথম layer-এর প্রতিটি timestep-এর hidden state $h_t^{(1)}$ -ই দ্বিতীয় layer-এর সেই timestep-এর **input** হিসেবে কাজ করে।

$$x_t \rightarrow h_t^{(1)} \rightarrow h_t^{(2)} \rightarrow \dots \rightarrow h_t^{(L)} \rightarrow y_t$$

এখানে প্রতিটি layer **নিজস্ব স্বাধীন parameter** ($W^{(l)}, U^{(l)}, b^{(l)}$) রাখে এবং **নিজস্ব hidden state** বহন করে।

কেন এটি প্রয়োজন? Feedforward network-এ আমরা জানি — একাধিক layer থাকলে network ক্রমান্বয়ে বিমূর্ত (abstract) feature শেখে। যেমন CNN-এ প্রথম layer প্রান্তরেখা (edge), পরের layer আকৃতি, তার পরের layer বস্তু চেনে। Deep RNN-এ একই নীতি সময়-নির্ভর data-র ক্ষেত্রে প্রয়োগ হয়:

- **Layer 1:** নিম্নস্তরের, স্বল্পমেয়াদি pattern (যেমন অক্ষর থেকে শব্দাংশ, বা ECG signal-এ একটি peak)
- **Layer 2:** মধ্যস্তরের pattern (যেমন শব্দগুচ্ছ, বা একটি সম্পূর্ণ QRS complex)
- **Layer 3:** উচ্চস্তরের, দীর্ঘমেয়াদি অর্থ (যেমন বাক্যের ভাব, বা হৃদস্পন্দনের ছন্দগত অস্বাভাবিকতা)

8.২ Temporal Depth বনাম Spatial Depth

এটি Deep RNN বোঝার সবচেয়ে গুরুত্বপূর্ণ ধারণা। একটি RNN আসলে **দুই অর্থে গভীর** হতে পারে:

ধরন	দিক	যা বাড়ে	নিয়ন্ত্রক parameter
Temporal Depth (সময়গত গভীরতা)	অনুভূমিক ($t \rightarrow t + 1$)	কত দূর অতীত পর্যন্ত তথ্য পৌঁছায়	Sequence length T
Spatial Depth (স্থানিক/স্তরগত গভীরতা)	উল্লম্ব ($l \rightarrow l + 1$)	প্রতিটি timestep-এ representation কত বিমূর্ত	Layer সংখ্যা L

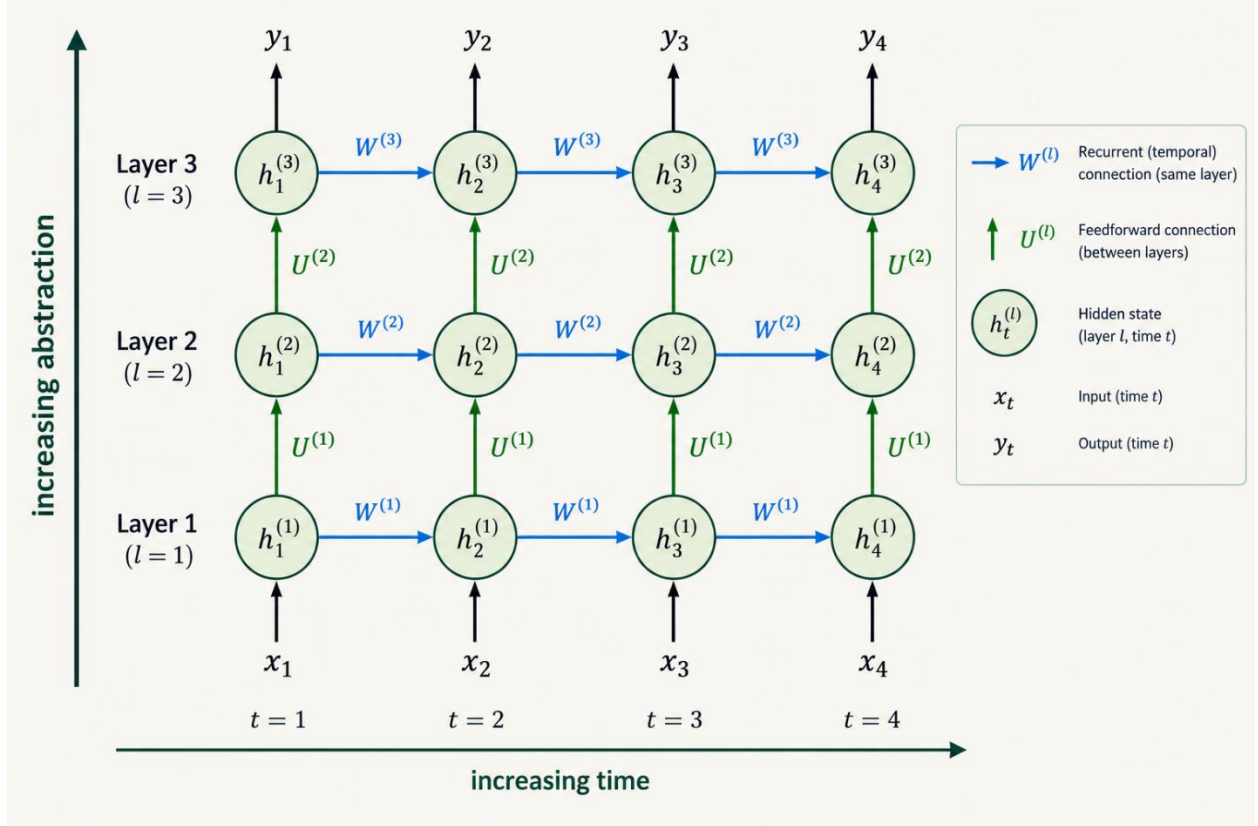
একটি সাধারণ RNN ইতিমধ্যেই temporal অর্থে “গভীর” — কারণ unfold করলে T টি ধাপ পাওয়া যায়। কিন্তু সেই গভীরতা **একই weight পুনরাবৃত্তি** করে তৈরি, তাই তা নতুন স্তরের বিমূর্ততা দেয় না। Spatial depth যোগ করলেই প্রকৃত hierarchical representation পাওয়া যায়।

[DIAGRAM 5: Deep RNN — Stacked and Unfolded in Time]

Purpose: একই ছবিতে temporal (অনুভূমিক) ও spatial (উল্লম্ব) — দুই ধরনের গভীরতা একসঙ্গে দেখানো।

Elements:

- একটি গ্রিড (grid) আঁকতে হবে: অনুভূমিক অক্ষে timestep $t = 1, 2, 3, \dots, T$; উল্লম্ব অক্ষে layer $l = 1, 2, \dots, L$ ।
- প্রতিটি গ্রিড-বিন্দুতে একটি বৃত্ত বা বাক্স, ভেতরে label $h_t^{(l)}$ ।
- **অনুভূমিক তীর** (একই layer-এর মধ্যে, বাম থেকে ডানে): $h_{t-1}^{(l)} \rightarrow h_t^{(l)}$, তীরের উপর label $W^{(l)}$, রঙ নীল। পাশে ছোট লেখা: “temporal / recurrent connection — same weight reused over time”।
- **উল্লম্ব তীর** (একই timestep-এর মধ্যে, নিচ থেকে উপরে): $h_t^{(l-1)} \rightarrow h_t^{(l)}$, তীরের উপর label $U^{(l)}$, রঙ সবুজ। পাশে লেখা: “spatial / feedforward connection — same weight reused across time but different per layer”।
- একদম নিচের সারিতে input $x_1, x_2, \dots, x_T$; একদম উপরের সারির উপরে output $y_1, y_2, \dots, y_T$ ।
- বাম পাশে একটি উল্লম্ব তীর label “increasing abstraction” এবং নিচে একটি অনুভূমিক তীর label “increasing time”।
- প্রতিটি layer-এর ডান পাশে সেই layer-এর hidden size লেখা থাকবে (যেমন Layer 1: 4 units, Layer 2: 2 units)।



8.৩ Mathematical Notation

8.৩.১ $h_t^{(l)}$ -এর অর্থ

$h_t^{(l)}$ = layer l -এর, timestep t -এর hidden state

- **Subscript** t = কোন সময়ে (temporal index)
- **Superscript** (l) = কোন স্তরে (layer index)

দুটি বিশেষ ক্ষেত্র সংজ্ঞায়িত করা প্রয়োজন:

$h_t^{(0)} \equiv x_t$ (layer 0 বলতে মূল input বোঝায়)

$h_0^{(l)} \equiv \mathbf{0}$ (প্রতিটি layer-এর প্রাথমিক state সাধারণত শূন্য vector)

8.৩.২ Master Equation

$$h_t^{(l)} = \tanh(W^{(l)}h_{t-1}^{(l)} + U^{(l)}h_t^{(l-1)} + b^{(l)})$$

প্রতিটি অংশের ব্যাখ্যা:

পদ	কোথা থেকে আসে	কী বোঝায়
$W^{(l)} h_{t-1}^{(l)}$	একই layer, আগের সময়	এই স্তরের নিজস্ব স্মৃতি – অতীতে এই স্তর কী বুঝেছিল
$U^{(l)} h_t^{(l-1)}$	নিচের layer, একই সময়	বর্তমান মুহূর্তে নিচের স্তর কী তথ্য পাঠাচ্ছে
$b^{(l)}$	—	এই স্তরের bias
$\tanh$	—	non-linearity; এটি না থাকলে একাধিক layer একটিমাত্র linear layer-এ ভেঙে পড়ত

লক্ষ করুন, $l = 1$ বসালে $h_t^{(0)} = x_t$ হওয়ায় সমীকরণটি সাধারণ Vanilla RNN-এ পরিণত হয়:

$$h_t^{(1)} = \tanh(W^{(1)} h_{t-1}^{(1)} + U^{(1)} x_t + b^{(1)})$$

অর্থাৎ Deep RNN হলো Vanilla RNN-এর একটি স্বাভাবিক সাধারণীকরণ (generalization)।

Note: এখানে Vanilla RNN cell ব্যবহার করে ব্যাখ্যা করা হয়েছে, কিন্তু একই stacking নীতি ছবছ LSTM ও GRU-এর ক্ষেত্রেও প্রযোজ্য। Stacked LSTM-এ প্রতিটি layer-এর নিজস্ব $C_t^{(l)}$ ও $h_t^{(l)}$ থাকে, এবং layer l -এর input হয় $h_t^{(l-1)}$ । বাস্তবে Deep LSTM ও Deep GRU-ই বেশি ব্যবহৃত হয়।

8.8 W এবং U -এর মৌলিক পার্থক্য

এই পার্থক্যটি Deep RNN-এর কেন্দ্রীয় ধারণা, তাই আলাদাভাবে স্পষ্ট করা দরকার।

	$W^{(l)}$	$U^{(l)}$
নাম	Recurrent weight	Input/feedforward weight
দিক	অনুভূমিক (horizontal)	উল্লম্ব (vertical)
কাকে গুণ করে	$h_{t-1}^{(l)}$ — একই layer, আগের timestep	$h_t^{(l-1)}$ — নিচের layer, একই timestep
Dimension	$n_l \times n_l$ (সর্বদা বর্গাকার)	$n_l \times n_{l-1}$ (সাধারণত আয়তাকার)
ভূমিকা	সময়ের সঙ্গে স্মৃতি বহন	স্তরে স্তরে বিমূর্ততা তৈরি
পুনর্ব্যবহার	একই layer-এর সব timestep-এ	একই layer-এর সব timestep-এ

এখানে n_l = layer l -এর hidden unit সংখ্যা।

কেন $W^{(l)}$ সর্বদা বর্গাকার? কারণ এটি $h_{t-1}^{(l)}$ (n_l মাত্রার) থেকে $h_t^{(l)}$ (n_l মাত্রার)-এ map করে — input ও output উভয়ই একই স্তরের, তাই একই আকারের।

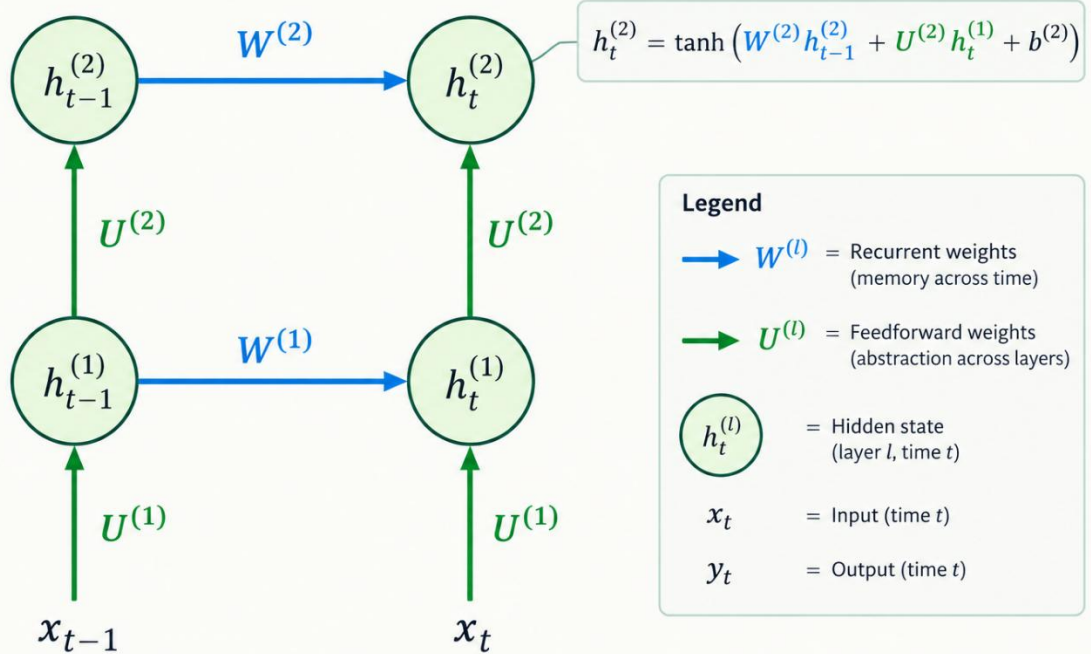
কেন $U^{(l)}$ আয়তাকার হতে পারে? কারণ এটি $h_t^{(l-1)}$ (n_{l-1} মাত্রার) থেকে $h_t^{(l)}$ (n_l মাত্রার)-এ map করে — দুটি ভিন্ন স্তর, তাই আকার ভিন্ন হতে পারে।

[DIAGRAM 6: Deep RNN — Horizontal vs Vertical Information Flow]

Purpose: W ও U -এর ভূমিকার পার্থক্য একটি ছোট, পরিষ্কার ছবিতে আলাদা করে দেখানো।

Elements:

- মাত্র চারটি node নিয়ে একটি 2×2 গ্রিড: $h_{t-1}^{(1)}$ (নিচ-বাম), $h_t^{(1)}$ (নিচ-ডান), $h_{t-1}^{(2)}$ (উপর-বাম), $h_t^{(2)}$ (উপর-ডান)।
- দুটি নীল অনুভূমিক তীর: $h_{t-1}^{(1)} \rightarrow h_t^{(1)}$ (label $W^{(1)}$) এবং $h_{t-1}^{(2)} \rightarrow h_t^{(2)}$ (label $W^{(2)}$)।
- দুটি সবুজ উল্লম্ব তীর: $h_{t-1}^{(1)} \rightarrow h_t^{(2)}$ (label $U^{(2)}$) এবং $h_t^{(1)} \rightarrow h_t^{(2)}$ (label $U^{(1)}$) — একই label দুবার, কারণ weight একই।
- নিচে x_{t-1}, x_t থেকে উল্লম্ব তীর layer 1-এ, label $U^{(1)}$ ।
- $h_t^{(2)}$ node-টিকে হাইলাইট করে পাশে একটি বাক্সে লেখা: $h_t^{(2)} = \tanh(W^{(2)}h_{t-1}^{(2)} + U^{(2)}h_t^{(1)} + b^{(2)})$, এবং সমীকরণের দুটি পদকে সংশ্লিষ্ট তীরের রঙে (নীল ও সবুজ) রঙ করা।
- দুটি key box: “Blue = memory across time (square matrix)”, “Green = abstraction across layers (may change dimension)”।



8.5 Dimension Tracking

একটি নির্দিষ্ট উদাহরণ নেওয়া যাক:

- **Input:** $n_0 = n_x = 3$
- **Layer 1:** hidden units $n_1 = 4$
- **Layer 2:** hidden units $n_2 = 2$

Layer 1-এর parameter:

$$U^{(1)} \in \mathbb{R}^{n_1 \times n_0} = \mathbb{R}^{4 \times 3}, \quad W^{(1)} \in \mathbb{R}^{n_1 \times n_1} = \mathbb{R}^{4 \times 4}, \quad b^{(1)} \in \mathbb{R}^{4 \times 1}$$

যাচাই:

$$(4 \times 3)(3 \times 1) = (4 \times 1), \quad (4 \times 4)(4 \times 1) = (4 \times 1)$$

$$U^{(1)} x_t$$

$$W^{(1)} h_{t-1}^{(1)}$$

দুটি (4×1) vector যোগ করা যায় ✓, ফলে $h_t^{(1)} \in \mathbb{R}^{4 \times 1}$ ।

Parameter সংখ্যা: $4 \times 3 + 4 \times 4 + 4 = 12 + 16 + 4 = 32$

Layer 2-এর parameter:

$$U^{(2)} \in \mathbb{R}^{n_2 \times n_1} = \mathbb{R}^{2 \times 4}, \quad W^{(2)} \in \mathbb{R}^{n_2 \times n_2} = \mathbb{R}^{2 \times 2}, \quad b^{(2)} \in \mathbb{R}^{2 \times 1}$$

যাচাই:

$$(2 \times 4)(4 \times 1) = (2 \times 1), \quad (2 \times 2)(2 \times 1) = (2 \times 1) \checkmark$$

$$U^{(2)} h_t^{(1)}$$

$$W^{(2)} h_{t-1}^{(2)}$$

Parameter সংখ্যা: $2 \times 4 + 2 \times 2 + 2 = 8 + 4 + 2 = 14$

মোট: $32 + 14 = 46$ parameter।

Note: layer 2-এর $U^{(2)}$ -এর column সংখ্যা (4) অবশ্যই layer 1-এর hidden size-এর সমান হতে হবে। এটিই stacking-এর dimension-সংক্রান্ত একমাত্র বাধ্যবাধকতা।

8.৬ Deep RNN Forward Pass — Numerical Example

সরল রাখার জন্য একটি ছোট configuration নিই: $n_x = 2, n_1 = 2, n_2 = 2$ ।

Parameters:

$$U^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.1 & 0.2 \\ -0.3 & 0.1 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}$$

$$U^{(2)} = \begin{bmatrix} 0.4 & 0.1 \\ -0.2 & 0.5 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.2 & -0.1 \\ 0.3 & 0.2 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

প্রাথমিক অবস্থা ও input:

$$h_{t-1}^{(1)} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}, \quad h_{t-1}^{(2)} = \begin{bmatrix} 0.1 \\ 0.3 \end{bmatrix}, \quad x_t = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

ধাপ ১ — Layer 1, timestep t (উল্লম্ব প্রবাহের শুরু)

$$\begin{aligned}U^{(1)}x_t &= \begin{bmatrix} (0.5)(1.0) + (-0.2)(0.5) \\ (0.3)(1.0) + (0.4)(0.5) \end{bmatrix} = \begin{bmatrix} 0.5 - 0.1 \\ 0.3 + 0.2 \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.5 \end{bmatrix} \\W^{(1)}h_{t-1}^{(1)} &= \begin{bmatrix} (0.1)(0.2) + (0.2)(-0.1) \\ (-0.3)(0.2) + (0.1)(-0.1) \end{bmatrix} = \begin{bmatrix} 0.02 - 0.02 \\ -0.06 - 0.01 \end{bmatrix} = \begin{bmatrix} 0.00 \\ -0.07 \end{bmatrix} \\a_t^{(1)} &= \begin{bmatrix} 0.4 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.00 \\ -0.07 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 0.43 \end{bmatrix} \\h_t^{(1)} &= \tanh \begin{bmatrix} 0.50 \\ 0.43 \end{bmatrix} = \boxed{\begin{bmatrix} 0.4621 \\ 0.4053 \end{bmatrix}}\end{aligned}$$

ধাপ ২ — Layer 2, timestep t

লক্ষ করুন, layer 2-এর input হলো এইমাত্র হিসাব করা $h_t^{(1)}$:

$$\begin{aligned}U^{(2)}h_t^{(1)} &= \begin{bmatrix} (0.4)(0.4621) + (0.1)(0.4053) \\ (-0.2)(0.4621) + (0.5)(0.4053) \end{bmatrix} = \begin{bmatrix} 0.1848 + 0.0405 \\ -0.0924 + 0.2027 \end{bmatrix} = \begin{bmatrix} 0.2254 \\ 0.1102 \end{bmatrix} \\W^{(2)}h_{t-1}^{(2)} &= \begin{bmatrix} (0.2)(0.1) + (-0.1)(0.3) \\ (0.3)(0.1) + (0.2)(0.3) \end{bmatrix} = \begin{bmatrix} 0.02 - 0.03 \\ 0.03 + 0.06 \end{bmatrix} = \begin{bmatrix} -0.01 \\ 0.09 \end{bmatrix} \\a_t^{(2)} &= \begin{bmatrix} 0.2254 \\ 0.1102 \end{bmatrix} + \begin{bmatrix} -0.01 \\ 0.09 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.2154 \\ 0.3002 \end{bmatrix} \\h_t^{(2)} &= \tanh \begin{bmatrix} 0.2154 \\ 0.3002 \end{bmatrix} = \boxed{\begin{bmatrix} 0.2121 \\ 0.2915 \end{bmatrix}}\end{aligned}$$

ধাপ ৩ — পরবর্তী timestep $t + 1$ (অনুভূমিক প্রবাহ)

নতুন input: $x_{t+1} = \begin{bmatrix} 0.0 \\ 1.0 \end{bmatrix}$

Layer 1:

$$\begin{aligned}U^{(1)}x_{t+1} &= \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix}, \quad W^{(1)}h_t^{(1)} = \begin{bmatrix} (0.1)(0.4621) + (0.2)(0.4053) \\ (-0.3)(0.4621) + (0.1)(0.4053) \end{bmatrix} = \begin{bmatrix} 0.1273 \\ -0.0981 \end{bmatrix} \\a_{t+1}^{(1)} &= \begin{bmatrix} -0.2 + 0.1273 + 0.1 \\ 0.4 - 0.0981 + 0.0 \end{bmatrix} = \begin{bmatrix} 0.0273 \\ 0.3019 \end{bmatrix} \Rightarrow h_{t+1}^{(1)} = \begin{bmatrix} 0.0273 \\ 0.2930 \end{bmatrix}\end{aligned}$$

Layer 2:

$$\begin{aligned}U^{(2)}h_{t+1}^{(1)} &= \begin{bmatrix} (0.4)(0.0273) + (0.1)(0.2930) \\ (-0.2)(0.0273) + (0.5)(0.2930) \end{bmatrix} = \begin{bmatrix} 0.0402 \\ 0.1410 \end{bmatrix} \\W^{(2)}h_t^{(2)} &= \begin{bmatrix} (0.2)(0.2121) + (-0.1)(0.2915) \\ (0.3)(0.2121) + (0.2)(0.2915) \end{bmatrix} = \begin{bmatrix} 0.0133 \\ 0.1220 \end{bmatrix}\end{aligned}$$

$$a_{t+1}^{(2)} = \begin{bmatrix} 0.0402 + 0.0133 + 0.0 \\ 0.1410 + 0.1220 + 0.1 \end{bmatrix} = \begin{bmatrix} 0.0535 \\ 0.3630 \end{bmatrix} \Rightarrow h_{t+1}^{(2)} = \begin{bmatrix} 0.0534 \\ 0.3479 \end{bmatrix}$$

গণনার ক্রম সম্পর্কে একটি গুরুত্বপূর্ণ পর্যবেক্ষণ

$h_t^{(2)}$ গণনা করতে $h_t^{(1)}$ আগে লাগে। আবার $h_{t+1}^{(1)}$ গণনা করতে $h_t^{(1)}$ লাগে। তাই বৈধ গণনাক্রম দুটি হতে পারে:

(ক) timestep-প্রধান: $h_t^{(1)} \rightarrow h_t^{(2)} \rightarrow h_{t+1}^{(1)} \rightarrow h_{t+1}^{(2)} \rightarrow \dots$

(খ) layer-প্রধান: $h_1^{(1)} \rightarrow h_2^{(1)} \rightarrow \dots \rightarrow h_T^{(1)}$ তারপর $h_1^{(2)} \rightarrow \dots \rightarrow h_T^{(2)}$

বাস্তব library (Keras, PyTorch) পদ্ধতি (খ) ব্যবহার করে — প্রথম layer-এর পুরো sequence একবারে হিসাব করে, তারপর পরের layer। এতে GPU-তে ভালো সমান্তরালীকরণ পাওয়া যায়।

8.9 Deep RNN Backpropagation — দুই দিকে Gradient

Deep RNN-এ gradient দুটি স্বাধীন দিকে প্রবাহিত হয়। এই দ্বৈত প্রবাহই এর training-কে কঠিন করে তোলে।

8.9.1 মোট Gradient-এর গঠন

একটি node $h_t^{(l)}$ দুটি জায়গায় ব্যবহৃত হয়:

1. **উপরের layer-এ:** $h_t^{(l)} \rightarrow h_t^{(l+1)}$ (একই সময়ে)
2. **পরের timestep-এ:** $h_t^{(l)} \rightarrow h_{t+1}^{(l)}$ (একই layer-এ)

তাই chain rule অনুযায়ী দুটি উৎস থেকে আসা gradient যোগ করতে হয়:

$$\frac{\partial L}{\partial h_t^{(l)}} = (U^{(l+1)})^\top \frac{\partial L}{\partial a_t^{(l+1)}} + (W^{(l)})^\top \frac{\partial L}{\partial a_{t+1}^{(l)}}$$

উল্লম্ব: উপরের layer থেকে অনুভূমিক: পরের timestep থেকে

(সর্বোচ্চ layer-এ প্রথম পদটির জায়গায় output layer থেকে আসা gradient বসবে; শেষ timestep-এ দ্বিতীয় পদটি শূন্য।)

8.9.2 Pre-activation-এ রূপান্তর

$h_t^{(l)} = \tanh(a_t^{(l)})$ হওয়ায়:

$$\frac{\partial L}{\partial a_t^{(l)}} = \frac{\partial L}{\partial h_t^{(l)}} \odot (1 - (h_t^{(l)})^2)$$

8.9.7 Temporal Backpropagation ($t \rightarrow t - 1$)

একই layer-এর মধ্যে সময়ের বিপরীতে:

$$\frac{\partial h_t^{(l)}}{\partial h_{t-1}^{(l)}} = \text{diag}\left(1 - \left(h_t^{(l)}\right)^2\right) W^{(l)}$$

k ধাপ পিছিয়ে যেতে:

$$\frac{\partial h_t^{(l)}}{\partial h_{t-k}^{(l)}} = \prod_{j=t-k+1}^t \text{diag}\left(1 - \left(h_j^{(l)}\right)^2\right) W^{(l)}$$

এটি ঠিক সেই vanishing/exploding gradient-এর সমস্যা যা আমরা ০.৪ অনুচ্ছেদে দেখেছি।

8.9.8 Vertical Backpropagation ($l \rightarrow l - 1$)

একই timestep-এ স্তরের বিপরীতে:

$$\frac{\partial h_t^{(l)}}{\partial h_t^{(l-1)}} = \text{diag}\left(1 - \left(h_t^{(l)}\right)^2\right) U^{(l)}$$

L টি layer পেরিয়ে নিচে নামতে:

$$\frac{\partial h_t^{(L)}}{\partial h_t^{(1)}} = \prod_{l=2}^L \text{diag}\left(1 - \left(h_t^{(l)}\right)^2\right) U^{(l)}$$

8.9.9 Parameter Gradient

$$\frac{\partial L}{\partial W^{(l)}} = \sum_{t=1}^T \frac{\partial L}{\partial a_t^{(l)}} \left(h_{t-1}^{(l)}\right)^\top$$

$$\frac{\partial L}{\partial U^{(l)}} = \sum_{t=1}^T \frac{\partial L}{\partial a_t^{(l)}} \left(h_t^{(l-1)}\right)^\top$$

$$\frac{\partial L}{\partial b^{(l)}} = \sum_{t=1}^T \frac{\partial L}{\partial a_t^{(l)}}$$

সব ক্ষেত্রেই t -এর উপর যোগফল নিতে হয়, কারণ একই parameter সব timestep-এ ব্যবহৃত হয়েছে।

Important Note — যৌগিক গভীরতার ঝুঁকি: T timestep ও L layer-এর একটি network-এ গভীরতম gradient path-এর দৈর্ঘ্য প্রায় $T + L$ । উদাহরণস্বরূপ $T = 100$ ও $L = 3$ হলে gradient-কে প্রায় ১০৩টি non-linearity পেরোতে হয়। তাই Deep RNN-এ

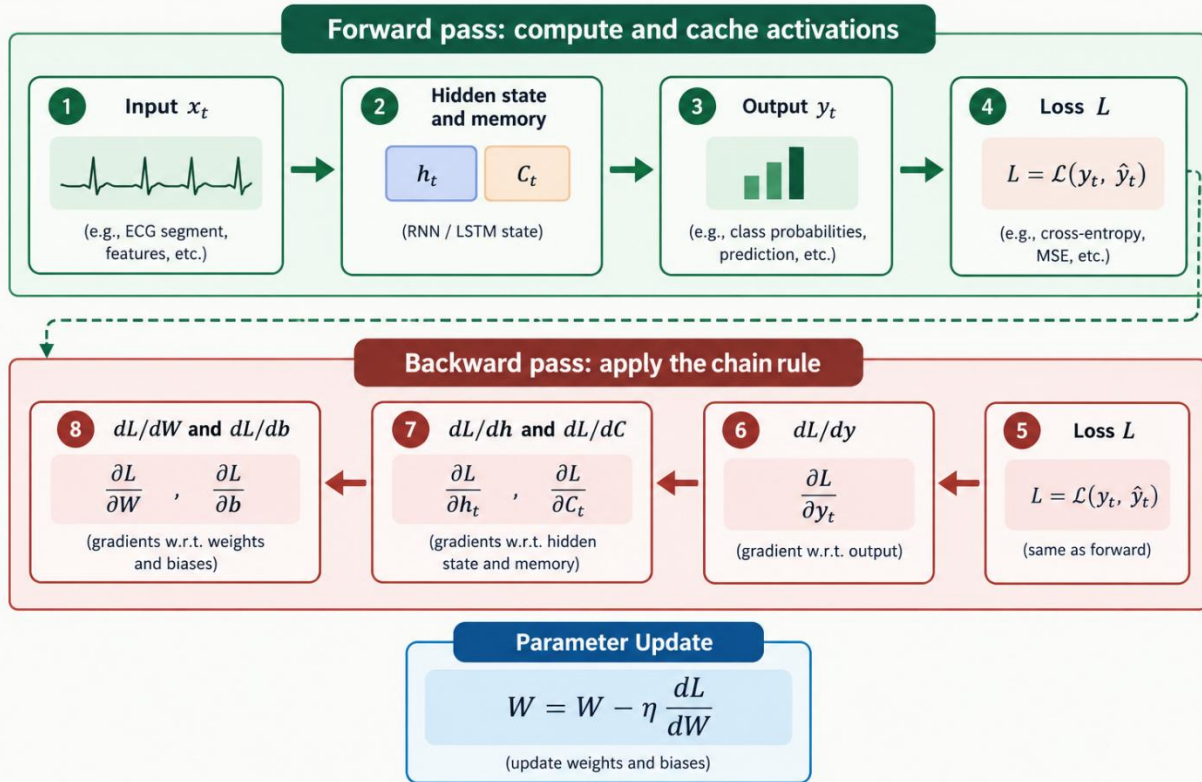
vanishing gradient সমস্যা **আরও তীব্র** হয়। এই কারণেই বাস্তবে Deep RNN প্রায় সবসময় LSTM বা GRU cell দিয়ে তৈরি হয়, এবং সঙ্গে ব্যবহৃত হয় residual/skip connection, layer normalization ও gradient clipping।

[DIAGRAM 10: Forward vs Backward Propagation Concept]

Purpose: Forward pass ও backward pass-এর দিক ও উদ্দেশ্যের পার্থক্য একটি ছবিতে স্পষ্ট করা (এই diagram-টি PART 8-এর সঙ্গেও প্রাসঙ্গিক)।

Elements:

- উপরের অর্ধেক **Forward Pass:** বাম থেকে ডানে চারটি বাক্স — Input $x \rightarrow$ Hidden/Memory $h, C \rightarrow$ Output $y \rightarrow$ Loss L ; তীর সবুজ, তীরের উপরে লেখা “compute & store intermediate values”।
- নিচের অর্ধেক **Backward Pass:** ডান থেকে বামে চারটি বাক্স — Loss $L \rightarrow \partial L / \partial y \rightarrow \partial L / \partial h, \partial L / \partial C \rightarrow \partial L / \partial W, \partial L / \partial b$; তীর লাল, তীরের উপরে লেখা “apply chain rule”।
- একদম নিচে একটি বাক্স: Parameter Update: $W \leftarrow W - \eta \partial L / \partial W$ ।
- ডান পাশে একটি উল্লম্ব ডটেড তীর যা forward-এর শেষ বিন্দু থেকে backward-এর শুরুতে যাবে।
- দুটি পাশে ছোট note: forward-এ “activations must be cached”, backward-এ “gradients accumulate over timesteps and layers”।



8.৮ Deep RNN Parameter Calculation — একটি সম্পূর্ণ উদাহরণ

একটি বাস্তব text model বিবেচনা করা যাক:

স্তর	বিবরণ
Embedding Layer	Vocabulary size $V = 10,000$, Embedding dimension $d = 32$
SimpleRNN Layer 1	5 units, return_sequences=True
SimpleRNN Layer 2	5 units

ধাপ ১ — Embedding Layer

Embedding হলো একটি lookup table: প্রতিটি শব্দের জন্য একটি 32-মাত্রার vector।

$$\text{Params}_{\text{emb}} = V \times d = 10,000 \times 32 = \boxed{320,000}$$

ধাপ ২ — SimpleRNN Layer 1

এই layer-এর input dimension = 32 (embedding থেকে), hidden units = 5।

$$\underbrace{32 \times 5}_{U^{(1)}: \text{input weight}} + \underbrace{5 \times 5}_{W^{(1)}: \text{recurrent weight}} + \underbrace{5}_{b^{(1)}} = 160 + 25 + 5 = \boxed{190}$$

ধাপ ৩ — SimpleRNN Layer 2

এই layer-এর input dimension = 5 (layer 1-এর hidden size), hidden units = 5।

$$\underbrace{5 \times 5}_{U^{(2)}} + \underbrace{5 \times 5}_{W^{(2)}} + \underbrace{5}_{b^{(2)}} = 25 + 25 + 5 = \boxed{55}$$

ধাপ ৪ — মোট

$$\text{Total} = 320,000 + 190 + 55 = \boxed{320,245}$$

পর্যবেক্ষণ: মোট parameter-এর ৯৯.৯% আছে embedding layer-এ, আর recurrent অংশে মাত্র 245টি। বড় vocabulary-যুক্ত NLP model-এ এটি খুবই সাধারণ চিত্র। এ কারণেই pre-trained embedding (Word2Vec, GloVe, FastText) ব্যবহার করে embedding layer freeze করে দিলে trainable parameter নাটকীয়ভাবে কমে যায়।

সাধারণ সূত্র

L টি layer-এর Deep RNN-এর জন্য (cell-ভেদে গুণক k ভিন্ন):

$$\text{Total} = \sum_{l=1}^L k (n_{l-1}n_l + n_l^2 + n_l)$$

$$k = 1 \text{ (SimpleRNN)}, \quad k = 3 \text{ (GRU)}, \quad k = 4 \text{ (LSTM)}$$

উদাহরণ: উপরের architecture-এ SimpleRNN-এর বদলে LSTM ব্যবহার করলে:

- Layer 1: $4(32 \times 5 + 25 + 5) = 4 \times 190 = 760$
- Layer 2: $4(5 \times 5 + 25 + 5) = 4 \times 55 = 220$
- মোট recurrent parameter: 980 (SimpleRNN-এর 245-এর ঠিক ৪ গুণ)

৪.৯ Deep RNN কখন ও কেন ব্যবহার করবেন?

৪.৯.১ সুবিধা (Advantages)

(১) **Hierarchical Representation:** নিচের layer স্থানীয় ও স্বল্পমেয়াদি pattern ধরে, উপরের layer দীর্ঘমেয়াদি ও বিমূর্ত কাঠামো ধরে। Speech recognition-এ এটি স্পষ্টভাবে দেখা যায়: নিচের layer ধ্বনি (phoneme), উপরের layer শব্দ ও ব্যাকরণগত কাঠামো শেখে।

(২) **Higher-level Feature Extraction:** একটি non-linear রূপান্তরে যে feature তৈরি করা সম্ভব নয়, একাধিক রূপান্তরে তা সম্ভব হতে পারে। দুটি layer-এর মধ্যে tanh থাকায় সম্মিলিত function কেবল linear থাকে না।

(৩) **Complex Sequence Modeling:** দীর্ঘ ও জটিল sequence (যেমন সম্পূর্ণ অনুচ্ছেদ, সংগীত, বহু-চ্যানেল biomedical signal)-এর জন্য একস্তরের ধারণক্ষমতা প্রায়ই অপ্রতুল।

(৪) **Encoder-Decoder Architecture:** Machine translation-এ encoder ও decoder উভয়ই সাধারণত বহুস্তরবিশিষ্ট stacked LSTM হয় (যেমন মূল Seq2Seq গবেষণায় ৪ স্তর ব্যবহৃত হয়েছিল)।

৪.৯.২ সীমাবদ্ধতা (Limitations)

(১) **More Parameters:** প্রতিটি নতুন layer নতুন $W^{(l)}, U^{(l)}, b^{(l)}$ যোগ করে $\rightarrow$ বেশি memory, বেশি overfitting-ঝুঁকি।

(২) **More Computation:** L layer ও T timestep মানে $L \times T$ বার cell গণনা। প্রশিক্ষণ ও inference উভয়ই ধীর হয়।

(৩) **Harder Optimization:** gradient-কে সময় ও স্তর — দুই দিকেই দীর্ঘ পথ পেরোতে হয়, ফলে training অস্থিতিশীল হতে পারে।

(৪) **Vanishing/Exploding Gradient তীব্রতর:** ইতিমধ্যে ৪.৭.৫-এ আলোচিত।

(৫) **Diminishing Returns:** বাস্তবে ২-৪ স্তরের বেশি গেলে উন্নতি সাধারণত কমে আসে, যদি না skip connection বা normalization ব্যবহার করা হয় এবং প্রচুর data থাকে।

ব্যবহারিক পরামর্শ: ১ স্তর দিয়ে শুরু করুন। যদি model **underfit** করে (training ও validation উভয় error বেশি), তবে স্তর বাড়ানো যুক্তিযুক্ত। যদি model **overfit** করে (training error কম, validation error বেশি), তবে স্তর বাড়ানো সমস্যা বাড়াবে — তখন dropout, regularization বা আরও data প্রয়োজন।

৫. PART 5 — Bidirectional RNN (BiRNN)

৫.১ কেন Bidirectional RNN প্রয়োজন?

এতক্ষণ আলোচিত সব architecture-এর একটি সাধারণ বৈশিষ্ট্য আছে: তারা সবাই **causal** বা কার্যকারণ-ক্রম মেনে চলে। অর্থাৎ timestep t -এ prediction করতে গিয়ে তারা কেবল x_1 থেকে x_t পর্যন্ত দেখতে পায়:

$$h_t = f(x_1, x_2, \dots, x_t)$$

$x_{t+1}, x_{t+2}, \dots, x_T$ — অর্থাৎ **ভবিষ্যৎ context** — তাদের কাছে অদৃশ্য।

অনেক কাজেই এই সীমাবদ্ধতা মারাত্মক। কারণ ভাষায় একটি শব্দের অর্থ প্রায়ই তার **পরে আসা শব্দ** দিয়ে নির্ধারিত হয়।

৫.১.১ NER উদাহরণ (Named Entity Recognition)

বিবেচনা করুন দুটি বাক্য:

বাক্য ১: “I love **Amazon**, it is a great website.”

বাক্য ২: “I love **Amazon**, it is a beautiful river.”

দুটিতেই “Amazon” শব্দটি ঠিক একই অবস্থানে, এবং তার **আগের সব শব্দ ছবছ অভিন্ন** (“I love”)।

একটি unidirectional RNN “Amazon” process করার মুহূর্তে কেবল “I love” দেখেছে। তার কাছে দুটি ক্ষেত্র সম্পূর্ণ অভেদ্য। ফলে সে বাধ্য হয়ে একই label দেবে। কিন্তু সঠিক উত্তর ভিন্ন:

বাক্য	সঠিক label	নির্ধারক তথ্য	অবস্থান
১	ORGANIZATION (প্রতিষ্ঠান)	“website”	৭ শব্দ পরে
২	LOCATION (স্থান)	“river”	৭ শব্দ পরে

নির্ধারক তথ্যটি সম্পূর্ণভাবে **ভবিষ্যতে**। তাই forward-only RNN দিয়ে এই কাজটি নীতিগতভাবেই সম্ভব নয়।

৫.১.২ আরও কিছু উদাহরণ

- **POS Tagging:** “He will **book** a flight” (verb) বনাম “He read the **book**” (noun) — পরের শব্দই নির্ধারক।
- **Speech Recognition:** একটি ধ্বনি কোন শব্দের অংশ তা প্রায়ই পরের ধ্বনি শুনে বোঝা যায় (coarticulation)।
- **Bioinformatics:** DNA-তে একটি অঞ্চলের কার্যকারিতা তার দুই পাশের ক্রম দিয়ে নির্ধারিত হয়।

সমাধানের ধারণা: যদি একটি RNN বাম থেকে ডানে পড়তে পারে, তবে আরেকটি RNN ডান থেকে বামে পড়তে দিলে কেমন হয়? তারপর প্রতিটি অবস্থানে দুটির তথ্য একত্র করলে সেই অবস্থানে **সম্পূর্ণ (past + future) context** পাওয়া যাবে। এটিই Bidirectional RNN (Schuster ও Paliwal, ১৯৯৭)।

৫.২ Architecture

BiRNN-এ দুটি সম্পূর্ণ স্বাধীন recurrent network থাকে, যারা একই input sequence-এর উপর বিপরীত দিকে চলে:

Network	প্রতীক	দিক	ক্রম
Forward	$\vec{h}_t$	বাম $\rightarrow$ ডান	$t = 1, 2, \dots, T$
Backward	$\overleftarrow{h}_t$	ডান $\rightarrow$ বাম	$t = T, T - 1, \dots, 1$

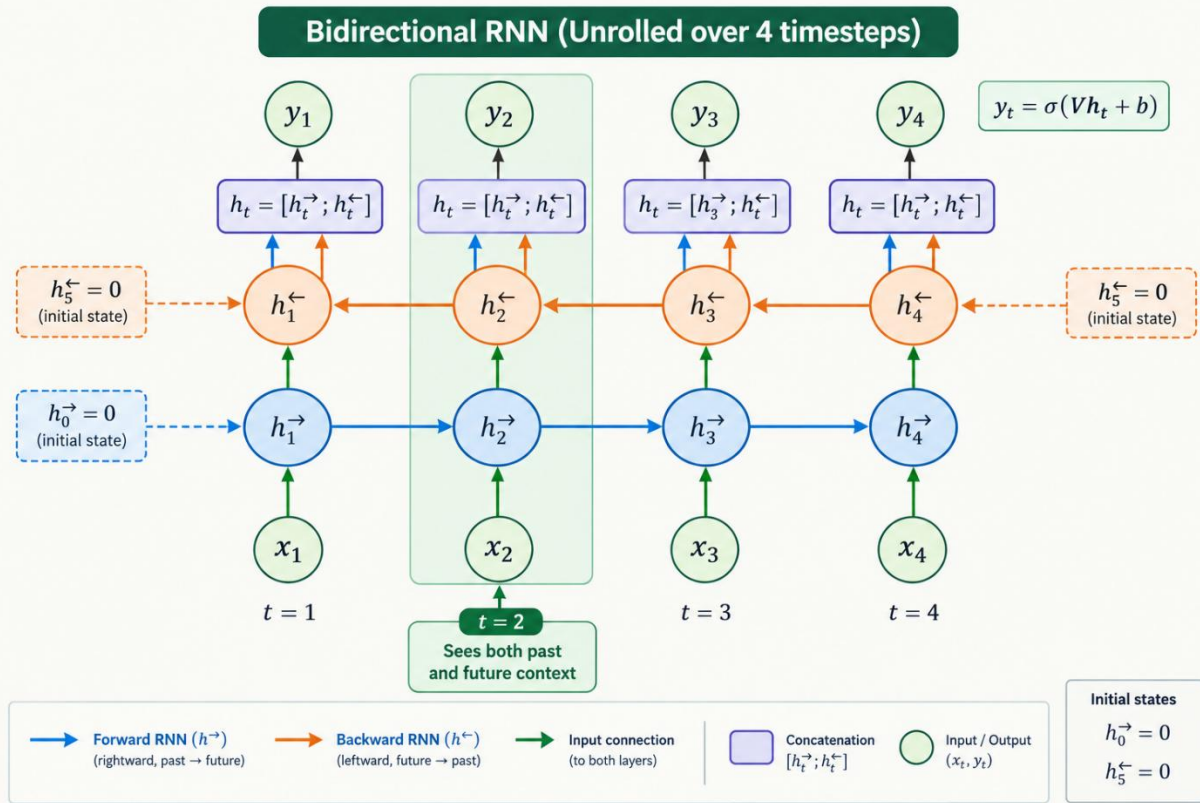
গুরুত্বপূর্ণ বিষয়সমূহ:

1. দুটি network-এর parameter সম্পূর্ণ আলাদা: $\vec{W}, \vec{U}, \vec{b}$ এবং $\overleftarrow{W}, \overleftarrow{U}, \overleftarrow{b}$ ।
2. দুটির মধ্যে কোনো সংযোগ নেই — forward network-এর hidden state backward network-এ যায় না এবং উল্টোটাও নয়। তারা স্বাধীনভাবে চলে এবং কেবল শেষে মিলিত হয়।
3. উভয়ই একই x_t input হিসেবে ব্যবহার করে।
4. দুটির ফল concatenate করে চূড়ান্ত representation তৈরি হয়।

Purpose: দুটি বিপরীতমুখী recurrent প্রবাহ এবং তাদের concatenation একটি ছবিতে দেখানো।

Elements:

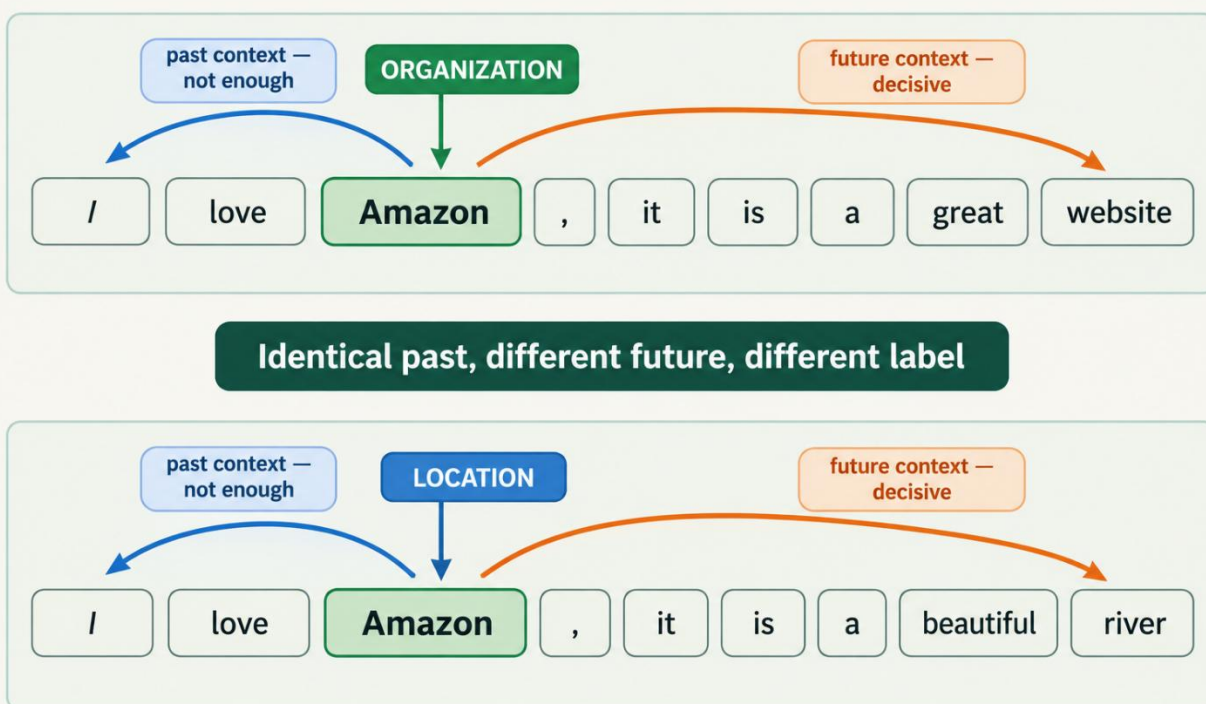
- একদম নিচের সারিতে input node: x_1, x_2, x_3, x_4 (বাম থেকে ডানে)।
- তার উপরে **Forward layer**: চারটি node $\vec{h}_1, \vec{h}_2, \vec{h}_3, \vec{h}_4$, এবং তাদের মধ্যে ডানমুখী তীর ($\rightarrow$), রঙ নীল। label: “Forward RNN: reads $x_1 \rightarrow x_T$ ”।
- তার উপরে **Backward layer**: চারটি node $\overleftarrow{h}_1, \overleftarrow{h}_2, \overleftarrow{h}_3, \overleftarrow{h}_4$, এবং তাদের মধ্যে বামমুখী তীর ($\leftarrow$), রঙ কমলা। label: “Backward RNN: reads $x_T \rightarrow x_1$ ”।
- প্রতিটি x_t থেকে দুটি উল্লম্ব তীর — একটি $\vec{h}_t$ -এ, আরেকটি $\overleftarrow{h}_t$ -এ।
- একদম উপরে **Concatenation layer**: প্রতিটি t -এর জন্য একটি বাক্স, যেখানে $h_t = [\vec{h}_t; \overleftarrow{h}_t]$; $\vec{h}_t$ ও $\overleftarrow{h}_t$ উভয় থেকে তীর এই বাক্সে আসবে।
- তার উপরে output node y_1, y_2, y_3, y_4 , label: $y_t = \sigma(Vh_t + b_y)$ ।
- একটি node (যেমন $t = 2$) হাইলাইট করে পাশে annotation: “ h_2 sees x_1, x_2 (from forward) and x_2, x_3, x_4 (from backward) $\rightarrow$ full sequence context”।
- দুই প্রান্তে শূন্য initial state দেখানো: $\vec{h}_0 = \mathbf{0}$ (বাম প্রান্তে) ও $\overleftarrow{h}_{T+1} = \mathbf{0}$ (ডান প্রান্তে)।



Purpose: “I love Amazon, it is a great website” উদাহরণে ভবিষ্যৎ context কীভাবে বর্তমান শব্দের label নির্ধারণে সাহায্য করে, তা concretely দেখানো।

Elements:

- একটি বাক্যকে শব্দে ভেঙে অনুভূমিকভাবে সাজানো: I | love | Amazon | it | is | a | great | website
- “Amazon” শব্দটির নিচে একটি বড় হাইলাইট বৃত্ত।
- “Amazon” থেকে বাম দিকে একটি নীল arc-তীর I ও love-এর দিকে, label: “past context (forward RNN) $\rightarrow$ not enough”।
- “Amazon” থেকে ডান দিকে একটি কমলা arc-তীর website পর্যন্ত, label: “future context (backward RNN) $\rightarrow$ decisive”।
- “Amazon”-এর উপরে একটি output বাক্স: ORGANIZATION, সবুজ রঙে।
- নিচে দ্বিতীয় একটি সারিতে একই ছবি দ্বিতীয় বাক্যের জন্য: I | love | Amazon | it | is | a | beautiful | river, এবং output বাক্স LOCATION, নীল রঙে।
- দুটি সারির মধ্যে একটি ভাগরেখা, পাশে লেখা: “Identical past, different future $\rightarrow$ different label”।



৫.৩ Forward Pass (বাম → ডান)

$$\vec{h}_t = \tanh(\vec{W} \vec{h}_{t-1} + \vec{U} x_t + \vec{b}), \quad t = 1, 2, \dots, T$$

প্রাথমিক শর্ত: $\vec{h}_0 = \mathbf{0}$ ।

Dimension: যদি n_x = input size ও n_h = forward hidden size হয়:

$$\vec{U} \in \mathbb{R}^{n_h \times n_x}, \quad \vec{W} \in \mathbb{R}^{n_h \times n_h}, \quad \vec{b} \in \mathbb{R}^{n_h \times 1}, \quad \vec{h}_t \in \mathbb{R}^{n_h \times 1}$$

$\vec{h}_t$ -এর অর্থ: “ x_1 থেকে x_t পর্যন্ত পড়ে আমি এই মুহূর্ত সম্পর্কে যা বুঝেছি।”

৫.৪ Backward Pass (ডান → বাম)

সতর্কতা – পরিভাষাগত বিভ্রান্তি এড়ানো: এখানে “Backward Pass” বলতে **gradient** গণনা বোঝানো হচ্ছে না। এটি BiRNN-এর দ্বিতীয় recurrent network-এর **forward computation**, যা কেবল sequence-কে উল্টো দিক থেকে পড়ে। Gradient-সংক্রান্ত আলোচনা ৫.৭ অনুচ্ছেদে আছে।

$$\vec{h}_t = \tanh(\vec{W} \vec{h}_{t+1} + \vec{U} x_t + \vec{b}), \quad t = T, T-1, \dots, 1$$

প্রাথমিক শর্ত: $\vec{h}_{T+1} = \mathbf{0}$ ।

লক্ষ করুন মূল পার্থক্যটি: recurrent পদে $\tilde{h}_{t-1}$ -এর বদলে $\tilde{h}_{t+1}$ — অর্থাৎ পরবর্তী timestep-এর state। এবং গণনা শুরু হয় sequence-এর শেষ থেকে।

$\tilde{h}_t$ -এর অর্থ: “ x_T থেকে x_t পর্যন্ত (পেছন থেকে) পড়ে আমি এই মুহূর্ত সম্পর্কে যা বুঝেছি।”

তথ্যপ্রবাহের ক্রম:

$$\tilde{h}_{T+1} = \mathbf{0} \rightarrow \tilde{h}_T \rightarrow \tilde{h}_{T-1} \rightarrow \dots \rightarrow \tilde{h}_1$$

৫.৫ দুই দিক একত্রীকরণ (Combining Both Directions)

$$h_t = [\vec{h}_t; \tilde{h}_t]$$

$$y_t = \sigma(Vh_t + b_y)$$

Dimension tracking:

যদি forward hidden size = n_h এবং backward hidden size = n_h হয়:

$$h_t \in \mathbb{R}^{2n_h \times 1}$$

$$V \in \mathbb{R}^{n_y \times 2n_h}, \quad b_y \in \mathbb{R}^{n_y \times 1}, \quad y_t \in \mathbb{R}^{n_y \times 1}$$

এখানে n_y = output class সংখ্যা (NER-এ entity ধরনের সংখ্যা)।

Important Note: Concatenation-ই সবচেয়ে প্রচলিত merge পদ্ধতি এবং এটি কোনো তথ্য হারায় না। বিকল্প হিসেবে যোগ (sum), গড় (average) বা গুণ (multiply) ব্যবহার করা যায় — সেক্ষেত্রে $h_t \in \mathbb{R}^{n_h}$ থাকে, output layer ছোট হয়, কিন্তু দুই দিকের তথ্য মিশে যাওয়ায় কিছু পার্থক্য হারিয়ে যেতে পারে।

৫.৬ সম্পূর্ণ Numerical BiRNN Example

একটি তিন-timestep sequence নিয়ে পুরো গণনা করা যাক।

৫.৬.১ Setup

$$n_x = 1, \quad n_h = 2, \quad T = 3$$

$$x_1 = 1.0, \quad x_2 = -1.0, \quad x_3 = 0.5$$

Forward network parameters:

$$\vec{U} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad \vec{W} = \begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}$$

Backward network parameters:

$$\vec{U} = \begin{bmatrix} -0.4 \\ 0.6 \end{bmatrix}, \quad \vec{W} = \begin{bmatrix} 0.3 & -0.2 \\ 0.1 & 0.2 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

৫.৬.২ Forward Direction গণনা ($t = 1 \rightarrow 3$)

$t = 1$ (শুরু: $\vec{h}_0 = [0,0]^T$):

$$\vec{a}_1 = \vec{W} \cdot \mathbf{0} + \vec{U}(1.0) + \vec{b} = \begin{bmatrix} 0.5 + 0.1 \\ -0.3 + 0.0 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -0.3 \end{bmatrix}$$

$$\vec{h}_1 = \tanh \begin{bmatrix} 0.6 \\ -0.3 \end{bmatrix} = \begin{bmatrix} 0.5371 \\ -0.2913 \end{bmatrix}$$

$t = 2$:

$$\vec{W}\vec{h}_1 = \begin{bmatrix} (0.2)(0.5371) + (0.1)(-0.2913) \\ (-0.1)(0.5371) + (0.3)(-0.2913) \end{bmatrix} = \begin{bmatrix} 0.1074 - 0.0291 \\ -0.0537 - 0.0874 \end{bmatrix} = \begin{bmatrix} 0.0783 \\ -0.1411 \end{bmatrix}$$

$$\vec{U}(-1.0) = \begin{bmatrix} -0.5 \\ 0.3 \end{bmatrix}$$

$$\vec{a}_2 = \begin{bmatrix} 0.0783 - 0.5 + 0.1 \\ -0.1411 + 0.3 + 0.0 \end{bmatrix} = \begin{bmatrix} -0.3217 \\ 0.1589 \end{bmatrix} \Rightarrow \vec{h}_2 = \begin{bmatrix} -0.3111 \\ 0.1576 \end{bmatrix}$$

$t = 3$:

$$\vec{W}\vec{h}_2 = \begin{bmatrix} (0.2)(-0.3111) + (0.1)(0.1576) \\ (-0.1)(-0.3111) + (0.3)(0.1576) \end{bmatrix} = \begin{bmatrix} -0.0622 + 0.0158 \\ 0.0311 + 0.0473 \end{bmatrix} = \begin{bmatrix} -0.0464 \\ 0.0784 \end{bmatrix}$$

$$\vec{a}_3 = \begin{bmatrix} -0.0464 + 0.25 + 0.1 \\ 0.0784 - 0.15 + 0.0 \end{bmatrix} = \begin{bmatrix} 0.3035 \\ -0.0716 \end{bmatrix} \Rightarrow \vec{h}_3 = \begin{bmatrix} 0.2946 \\ -0.0715 \end{bmatrix}$$

৫.৬.৩ Backward Direction গণনা ($t = 3 \rightarrow 1$)

$t = 3$ (শুরু: $\vec{h}_4 = [0,0]^T$):

$$\vec{a}_3 = \vec{W} \cdot \mathbf{0} + \vec{U}(0.5) + \vec{b} = \begin{bmatrix} -0.2 + 0.0 \\ 0.3 + 0.1 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix}$$

$$\vec{h}_3 = \tanh \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix} = \begin{bmatrix} -0.1974 \\ 0.3799 \end{bmatrix}$$

$t = 2$:

$$\vec{W}\vec{h}_3 = \begin{bmatrix} (0.3)(-0.1974) + (-0.2)(0.3799) \\ (0.1)(-0.1974) + (0.2)(0.3799) \end{bmatrix} = \begin{bmatrix} -0.0592 - 0.0760 \\ -0.0197 + 0.0760 \end{bmatrix} = \begin{bmatrix} -0.1352 \\ 0.0563 \end{bmatrix}$$

$$\vec{U}(-1.0) = \begin{bmatrix} 0.4 \\ -0.6 \end{bmatrix}$$

$$\vec{a}_2 = \begin{bmatrix} -0.1352 + 0.4 + 0.0 \\ 0.0563 - 0.6 + 0.1 \end{bmatrix} = \begin{bmatrix} 0.2648 \\ -0.4437 \end{bmatrix} \Rightarrow \vec{h}_2 = \begin{bmatrix} 0.2588 \\ -0.4167 \end{bmatrix}$$

$t = 1$:

$$\vec{W}\vec{h}_2 = \begin{bmatrix} (0.3)(0.2588) + (-0.2)(-0.4167) \\ (0.1)(0.2588) + (0.2)(-0.4167) \end{bmatrix} = \begin{bmatrix} 0.0776 + 0.0833 \\ 0.0259 - 0.0833 \end{bmatrix} = \begin{bmatrix} 0.1610 \\ -0.0575 \end{bmatrix}$$

$$\tilde{a}_1 = \begin{bmatrix} 0.1610 - 0.4 + 0.0 \\ -0.0575 + 0.6 + 0.1 \end{bmatrix} = \begin{bmatrix} -0.2390 \\ 0.6425 \end{bmatrix} \Rightarrow \tilde{h}_1 = \begin{bmatrix} -0.2346 \\ 0.5666 \end{bmatrix}$$

৫.৬.৪ সব hidden state একত্রে

t	x_t	$\vec{h}_t$ (forward)	$\tilde{h}_t$ (backward)
1	1.0	[0.5371, -0.2913]	[-0.2346, 0.5666]
2	-1.0	[-0.3111, 0.1576]	[0.2588, -0.4167]
3	0.5	[0.2946, -0.0715]	[-0.1974, 0.3799]

৫.৬.৫ $t = 2$ -এর চূড়ান্ত Output

ধাপ ১ — Concatenation:

$$h_2 = [\vec{h}_2; \tilde{h}_2] = \begin{bmatrix} -0.3111 \\ 0.1576 \\ 0.2588 \\ -0.4167 \end{bmatrix} \in \mathbb{R}^{4 \times 1}$$

ধাপ ২ — Output layer ($n_y = 1$, binary classification ধরে):

$$V = [0.5 \quad -0.2 \quad 0.3 \quad 0.4] \in \mathbb{R}^{1 \times 4}, \quad b_y = 0.1$$

$$\begin{aligned} Vh_2 &= (0.5)(-0.3111) + (-0.2)(0.1576) + (0.3)(0.2588) + (0.4)(-0.4167) \\ &= -0.1555 - 0.0315 + 0.0776 - 0.1667 = -0.2761 \\ u_2 &= Vh_2 + b_y = -0.2761 + 0.1 = -0.1761 \end{aligned}$$

ধাপ ৩ — Sigmoid:

$$y_2 = \sigma(-0.1761) = \frac{1}{1 + e^{0.1761}} = \frac{1}{1 + 1.1926} = \boxed{0.4561}$$

Dimension যাচাই: $(1 \times 4)(4 \times 1) = (1 \times 1) \checkmark$

৫.৬.৬ Interpretation

$y_2 = 0.4561$ — অর্থাৎ model timestep ২-এর জন্য ৪৫.৬% সম্ভাবনা দিচ্ছে (untrained random weight দিয়ে, তাই ফল ০.৫-এর কাছাকাছি হওয়াই প্রত্যাশিত)। গুরুত্বপূর্ণ বিষয় এটি নয়; গুরুত্বপূর্ণ হলো এই সিদ্ধান্তে x_1, x_2 ও x_3 — তিনটিরই অবদান আছে:

- $\vec{h}_2$ -এর মাধ্যমে এসেছে x_1, x_2 -এর তথ্য
- $\tilde{h}_2$ -এর মাধ্যমে এসেছে x_3, x_2 -এর তথ্য

একটি unidirectional RNN-এ x_3 -এর কোনো প্রভাবই থাকত না। এটিই BiRNN-এর মূল অর্জন।

৫.৬.৭ Parameter Count

Forward network: $\vec{U} (2 \times 1 = 2) + \vec{W} (2 \times 2 = 4) + \vec{b} (2) = 8$

Backward network: একই গঠন = 8

Output layer: $V (1 \times 4 = 4) + b_y (1) = 5$

$$\text{মোট} = 8 + 8 + 5 = 21$$

সাধারণ সূত্র: BiRNN-এর recurrent parameter সংখ্যা সমতুল্য unidirectional RNN-এর ঠিক দ্বিগুণ:

$$\text{Params}_{\text{Bi}} = 2 \times k(n_x n_h + n_h^2 + n_h)$$

($k = 1$ SimpleRNN, $k = 3$ GRU, $k = 4$ LSTM)। এছাড়া output layer-এর weight-ও দ্বিগুণ প্রশস্ত হয়, কারণ তার input এখন $2n_h$ মাত্রার।

উদাহরণ: $n_x = 100$, $n_h = 128$ -এর একটি BiLSTM-এ:

$$2 \times 4(100 \times 128 + 128^2 + 128) = 2 \times 117,248 = 234,496$$

৫.৭ BiRNN Backward Propagation (Gradient গণনা)

৫.৭.১ মূল নীতি

BiRNN-এর training আশ্চর্যরকম সরল, কারণ দুটি network স্বাধীন। Gradient তিনটি ধাপে প্রবাহিত হয়:

$$L \rightarrow y_t \rightarrow h_t = [\vec{h}_t; \vec{h}_t] \rightarrow \begin{cases} \vec{h}_t & \text{(forward network-এ, BPTT: } t \rightarrow t-1) \\ \vec{h}_t & \text{(backward network-এ, BPTT: } t \rightarrow t+1) \end{cases}$$

৫.৭.২ ধাপ ১ — Output থেকে Concatenated State-এ

$$\frac{\partial L}{\partial h_t} = V^\top \frac{\partial L}{\partial u_t}, \quad \frac{\partial L}{\partial u_t} = \frac{\partial L}{\partial y_t} \odot y_t(1 - y_t)$$

$$\frac{\partial L}{\partial V} = \sum_{t=1}^T \frac{\partial L}{\partial u_t} h_t^\top, \quad \frac{\partial L}{\partial b_y} = \sum_{t=1}^T \frac{\partial L}{\partial u_t}$$

৫.৭.৩ ধাপ ২ — Concatenation-এর gradient ভাগ করা

Concatenation একটি তথ্য-অবিকৃত operation। তাই তার gradient কেবল টুকরো করে ভাগ (split) হয়ে যায় — কোনো গুণ বা রূপান্তর হয় না:

$$\frac{\partial L}{\partial \vec{h}_t} = \left(\frac{\partial L}{\partial h_t} \right)_{1:n_h} \quad (\text{প্রথম } n_h \text{ উপাদান})$$

$$\frac{\partial L}{\partial \vec{h}_t} = \left(\frac{\partial L}{\partial \vec{h}_t} \right)_{n_h+1:2n_h} \quad (\text{শেষ } n_h \text{ উপাদান})$$

৫.৭.৪ ধাপ ৩ – দুই network-এ স্বাধীন BPTT

Forward network-এ (gradient সময়ের বিপরীতে, অর্থাৎ $t \rightarrow t - 1$):

$$\frac{\partial L}{\partial \vec{h}_t} \Big|_{\text{total}} = \underbrace{\left(\frac{\partial L}{\partial \vec{h}_t} \right)_{1:n_h}}_{\text{এই timestep-এর output থেকে}} + \underbrace{\vec{W}^\top \left[\frac{\partial L}{\partial \vec{h}_{t+1}} \Big|_{\text{total}} \odot (1 - \vec{h}_{t+1}^2) \right]}_{\text{পরের timestep থেকে}}$$

Backward network-এ (gradient সময়ের **অনুকূলে**, অর্থাৎ $t \rightarrow t + 1$ – কারণ forward computation উল্টো দিকে হয়েছিল):

$$\frac{\partial L}{\partial \vec{h}_t} \Big|_{\text{total}} = \underbrace{\left(\frac{\partial L}{\partial \vec{h}_t} \right)_{n_h+1:2n_h}}_{\text{এই timestep-এর output থেকে}} + \underbrace{\vec{W}^\top \left[\frac{\partial L}{\partial \vec{h}_{t-1}} \Big|_{\text{total}} \odot (1 - \vec{h}_{t-1}^2) \right]}_{\text{আগের timestep থেকে}}$$

Parameter gradient (উভয় network-এ একই কাঠামো):

$$\frac{\partial L}{\partial \vec{W}} = \sum_t \frac{\partial L}{\partial \vec{a}_t} \vec{h}_{t-1}^\top, \quad \frac{\partial L}{\partial \vec{W}} = \sum_t \frac{\partial L}{\partial \vec{a}_t} \vec{h}_{t+1}^\top$$

Important Note: যেহেতু দুটি network-এর মধ্যে কোনো সংযোগ নেই, তাই $\vec{W}$ -এর gradient-এ $\vec{W}$ -এর কোনো ভূমিকা নেই এবং উল্টোটাও সত্য। BiRNN-এর training কার্যত **দুটি স্বাধীন RNN-এর একযোগে training**, যাদের loss একটি সাধারণ output layer-এর মাধ্যমে যুক্ত। বাস্তব implementation-এ backward network-কে training করার সহজ কৌশল হলো – input sequence উল্টে দিয়ে একটি সাধারণ RNN চালানো, তারপর ফলাফল আবার উল্টে দেওয়া।

৫.৮ Applications

প্রয়োগক্ষেত্র	কেন BiRNN কার্যকর
Named Entity Recognition (NER)	একটি entity-র ধরন প্রায়ই পরের শব্দ দিয়ে নির্ধারিত (৫.১.১-এর উদাহরণ)
POS Tagging	শব্দের ব্যাকরণগত শ্রেণি দুই পাশের শব্দের উপর নির্ভরশীল
Speech Recognition	ধ্বনির উচ্চারণ পরবর্তী ধ্বনি দ্বারা প্রভাবিত (coarticulation); সম্পূর্ণ audio clip আগেই পাওয়া যায়
Sequence Labeling (সাধারণভাবে)	Chunking, slot filling, punctuation restoration – সব ক্ষেত্রেই প্রতিটি অবস্থানের জন্য পূর্ণ context প্রয়োজন
Bioinformatics	Protein secondary structure prediction, gene/splice-site সনাক্তকরণ –

	উভয় পাশের ক্রম গুরুত্বপূর্ণ
Seq2Seq Encoder	Machine translation-এ encoder সম্পূর্ণ উৎস-বাক্য আগেই পায়, তাই bidirectional encoder ব্যবহার করলে প্রতিটি শব্দের representation সমৃদ্ধ হয় (decoder অবশ্যই unidirectional থাকে)
Sentiment Analysis	একটি বাক্যের ভাব প্রায়ই শেষে উল্টে যায় (“...ভালোই ছিল, কিন্তু”)

৫.৯ Limitations

(১) সম্পূর্ণ sequence আগে থেকে থাকা আবশ্যিক

$\hat{h}_t$ গণনা করতে হলে x_T থেকে শুরু করতে হয়। অর্থাৎ শেষ element না জানা পর্যন্ত কোনো timestep-এর চূড়ান্ত output দেওয়া সম্ভব নয়।

(২) Real-time causal prediction-এর জন্য অনুপযুক্ত

Live speech transcription, real-time translation, stock price prediction, বা ECG-ভিত্তিক তাৎক্ষণিক সতর্কবার্তা — এসব ক্ষেত্রে ভবিষ্যৎ data বিদ্যমানই নয়। এখানে BiRNN নীতিগতভাবে ব্যবহারযোগ্য নয়। (আংশিক সমাধান: **chunked/latency-controlled BiRNN**, যেখানে ছোট ছোট window-এর উপর bidirectional প্রক্রিয়া চালানো হয়, কিছুটা বিলম্ব মেনে নিয়ে।)

(৩) প্রায় দ্বিগুণ recurrent computation

দুটি সম্পূর্ণ network চালাতে হয়, তাই সমতুল্য single-direction model-এর তুলনায় recurrent অংশের গণনা ও parameter প্রায় দ্বিগুণ।

(৪) উচ্চতর memory প্রয়োজন

Training-এ backpropagation-এর জন্য উভয় দিকের সব timestep-এর hidden state cache করে রাখতে হয়। দীর্ঘ sequence-এ এটি GPU memory-র উপর যথেষ্ট চাপ তৈরি করে।

(৫) Vanilla cell-এ gradient সমস্যা রয়েছে

Bidirectionality কেবল context-এর পরিধি বাড়ায়; এটি vanishing/exploding gradient-এর কোনো সমাধান নয়। উভয় দিকের network-ই দীর্ঘ sequence-এ একই সমস্যায় ভুগবে। এজন্যই বাস্তবে প্রায় সবসময় **BiLSTM** বা **BiGRU** ব্যবহার করা হয় — অর্থাৎ bidirectionality ও gating একসঙ্গে প্রয়োগ করা হয়।

সংযোগ: এখানেই এই নোটের চারটি architecture মিলিত হয়। একটি আধুনিক sequence labeling model প্রায়ই হয় **Stacked BiLSTM** — অর্থাৎ gating (LSTM) + spatial depth (Deep) + full context (Bidirectional), তিনটিই একসঙ্গে।

৬. PART 6 — সামগ্রিক স্থাপত্যগত অগ্রগতি (Architectural Progression)

এই অংশটি পুরো নোটের যুক্তিকাঠামো এক জায়গায় ধরে রাখে। প্রতিটি নতুন architecture আকাশ থেকে পড়েনি — প্রতিটিই আগেরটির একটি নির্দিষ্ট সমস্যার উত্তর।

৬.১ মূল ধারা: Memory নিয়ন্ত্রণের বিবর্তন

Vanilla RNN → LSTM → GRU

পর্যায় ১ → ২ : Vanilla RNN থেকে LSTM

Problem (সমস্যা): $h_t = \tanh(W_{hh}h_{t-1} + \dots)$ — প্রতিটি ধাপে পুরো state overwrite হয় এবং gradient-পথে বারবার W_{hh} ও $\tanh'$ গুণ হয়। ফলে vanishing gradient ও long-term dependency-র ব্যর্থতা।

Architectural Change (স্থাপত্যগত পরিবর্তন): একটি আলাদা Cell State C_t যোগ করা হলো, যার update হয় additive:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

এবং তিনটি শেখযোগ্য gate (f_t, i_t, o_t) যোগ করা হলো।

Consequence (ফলাফল): $\partial C_t / \partial C_{t-1} = \text{diag}(f_t)$ — একটি gradient highway তৈরি হলো, যার ক্ষয়ের হার এখন network নিজেই শেখে। **খরচ:** parameter ৪ গুণ, দুটি state বহন।

পর্যায় ২ → ৩ : LSTM থেকে GRU

Problem: LSTM কার্যকর, কিন্তু ৪টি weight matrix, ২টি state — অতিরিক্ত জটিল ও ব্যয়বহুল। পর্যবেক্ষণে দেখা গেল f_t ও i_t প্রায়শই পরিপূরকভাবে আচরণ করে।

Architectural Change: f_t ও i_t একীভূত করে একটিমাত্র update gate z_t ; C_t ও h_t একীভূত করে একটিমাত্র hidden state; এবং candidate-এ অতীতের প্রভাব নিয়ন্ত্রণে reset gate r_t ।

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Consequence: ২৫% কম parameter, কম computation, তুলনীয় কর্মক্ষমতা। **খরচ:** output নিয়ন্ত্রণ নেই, এবং ওজন পরিপূরক হওয়ায় স্মৃতি accumulate করা যায় না।

৬.২ সমান্তরাল সম্প্রসারণ: গভীরতা ও দিক

উপরের ধারাটি cell-এর ভেতরের উন্নয়ন। পরের দুটি সম্প্রসারণ cell-নিরপেক্ষ — এগুলো যেকোনো cell (RNN/LSTM/GRU)-এর সঙ্গে প্রয়োগযোগ্য।

সম্প্রসারণ ক : RNN → Deep RNN

Problem: একটিমাত্র recurrent layer একবারই non-linear রূপান্তর করে। ফলে জটিল, স্তরায়িত (hierarchical) pattern শেখার ক্ষমতা সীমিত।

Architectural Change: একাধিক recurrent layer উল্লম্বভাবে stack করা; layer l -এর input হয় layer $l - 1$ -এর output:

$$h_t^{(l)} = \tanh(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b^{(l)})$$

Consequence: নিচের স্তরে স্থানীয়, উপরের স্তরে বিমূর্ত feature। **খরচ:** gradient-পথ প্রায় $T + L$ দীর্ঘ, তাই optimization কঠিনতর; parameter ও computation বৃদ্ধি।

সম্প্রসারণ খ : RNN → Bidirectional RNN

Problem: Causal RNN কেবল অতীত দেখে। কিন্তু অনেক কাজে (t অবস্থানের label নির্ধারণে) ভবিষ্যৎ context অপরিহার্য।

Architectural Change: দুটি স্বাধীন network — একটি $1 \rightarrow T$, আরেকটি $T \rightarrow 1$ — এবং তাদের state concatenate করা:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Consequence: প্রতিটি অবস্থানে সম্পূর্ণ sequence-এর context। **খরচ:** পুরো sequence আগে থেকে লাগে (real-time অসম্ভব), computation ও memory প্রায় দ্বিগুণ।

৬.৩ সম্মিলিত রূপ

তিনটি অক্ষ পরস্পর-স্বাধীন হওয়ায় সেগুলো একসঙ্গে প্রয়োগ করা যায়:

অক্ষ	প্রশ্ন	পছন্দ
Cell type	স্মৃতি কীভাবে নিয়ন্ত্রিত হবে?	SimpleRNN / LSTM / GRU
Depth	কত স্তর বিমূর্ততা দরকার?	$L = 1, 2, 3, \dots$
Direction	ভবিষ্যৎ context কি পাওয়া যাবে?	Unidirectional / Bidirectional

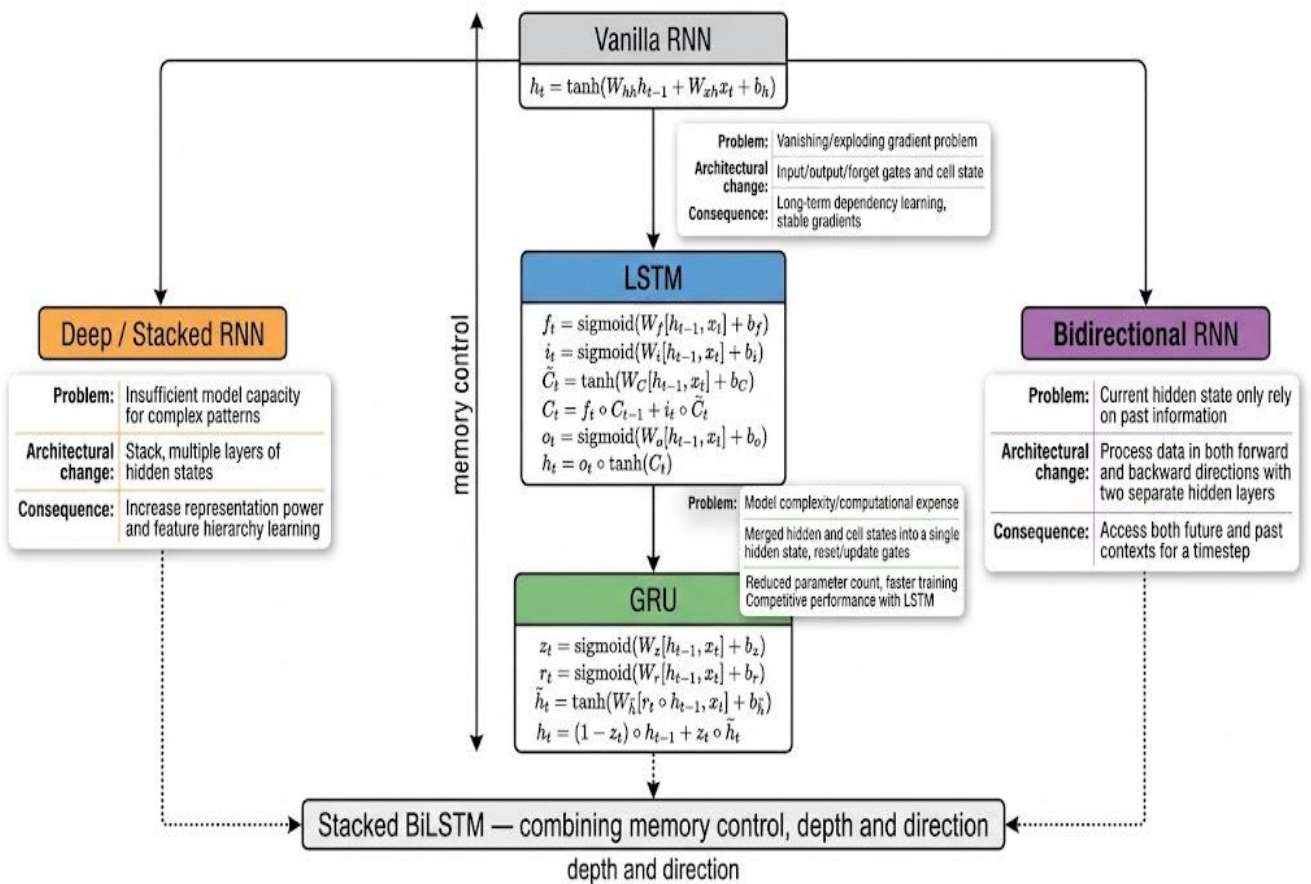
উদাহরণস্বরূপ একটি ২-স্তরের BiLSTM মানে: LSTM cell + $L = 2$ + bidirectional — তিনটি সিদ্ধান্ত একসঙ্গে।

[DIAGRAM 9: Evolution of Recurrent Neural Network Architectures]

Purpose: পুরো নোটের যুক্তিক্রম একটি ছবিতে ধরে রাখা — কোন সমস্যা থেকে কোন architecture এসেছে এবং তার ফল কী।

Elements:

- **কেন্দ্রীয় উল্লম্ব ধারা (মাঝখানে):** তিনটি বড় বাক্স উপর থেকে নিচে –
 1. Vanilla RNN (ধূসর) – নিচে ছোট লেখা: $h_t = \tanh(Wh_{t-1} + Ux_t + b)$
 2. LSTM (নীল) – নিচে: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
 3. GRU (সবুজ) – নিচে: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
- প্রতিটি বাক্সের মাঝে একটি নিচমুখী তীর, এবং তীরের পাশে তিন লাইনের annotation box:
 - RNN→LSTM: **Problem:** vanishing gradient, long-term dependency · **Change:** cell state + 3 gates (additive update) · **Consequence:** learnable gradient highway, 4× parameters
 - LSTM→GRU: **Problem:** too many parameters and two states · **Change:** merge forget/input into update gate, merge states · **Consequence:** 25% fewer parameters, no output gate
- **বাম দিকে একটি শাখা তীর** Vanilla RNN বাক্স থেকে বেরিয়ে একটি বাক্সে: Deep / Stacked RNN (কমলা), পাশে annotation: **Problem:** limited abstraction · **Change:** stack layers vertically · **Consequence:** hierarchical features, harder optimization |
- **ডান দিকে একটি শাখা তীর** Vanilla RNN বাক্স থেকে বেরিয়ে আরেকটি বাক্সে: Bidirectional RNN (বেগুনি), পাশে annotation: **Problem:** no future context · **Change:** two opposite-direction networks + concatenation · **Consequence:** full context, no real-time use |
- **একদম নিচে** একটি প্রশস্ত বাক্স: Stacked BiLSTM / BiGRU – combining all three axes, এবং LSTM/GRU, Deep, BiRNN – তিনটি বাক্স থেকেই এই বাক্সে ডটেড তীর আসবে।
- বাম পাশে উল্লম্বভাবে লেখা: “axis 1: memory control”; উপরে অনুভূমিকভাবে লেখা: “axis 2: depth ← → axis 3: direction” |



৭. PART 7 – সম্পূর্ণ তুলনামূলক সারণি

নিচের সারণিতে n_x = input dimension, n_h = hidden units, T = sequence length, L = layer সংখ্যা।

Feature	Vanilla RNN	LSTM	GRU	Deep RNN	BiRNN
Memory mechanism	একটি hidden state, প্রতিটি ধাপে সম্পূর্ণ overwrite	Cell state-এ additive update, gate-নিয়ন্ত্রিত	Hidden state-এ gated interpolation	ব্যবহৃত cell-এর উপর নির্ভরশীল; প্রতি layer-এ পৃথক state	ব্যবহৃত cell-এর উপর নির্ভরশীল; দুই দিকে পৃথক state
Gates	নেই	৩টি: f_t, i_t, o_t	২টি: r_t, z_t	cell-নির্ভর	cell-নির্ভর
State(s)	h_t	C_t ও h_t	কেবল h_t	$h_t^{(l)}$, প্রতি layer-এ একটি	$\vec{h}_t$ ও $\overleftarrow{h}_t$
Direction	একমুখী (past $\rightarrow$ present)	একমুখী	একমুখী	একমুখী (সাধারণত)	দ্বিমুখী
Depth	১ স্তর	১ স্তর	১ স্তর	L স্তর	১ স্তর (তবে stack করা যায়)
Core equation	$h_t = \tanh(W h_{t-1} + U x_t + b)$	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	$h_t^{(l)} = \tanh(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b^{(l)})$	$h_t = [\vec{h}_t; \overleftarrow{h}_t]$
Parameters	$n_x n_h + n_h^2 + n_h$	$4(n_x n_h + n_h^2 + n_h)$	$3(n_x n_h + n_h^2 + n_h)$	$\sum_l k (n_{l-1} n_l + n_l^2 + n_l)$	base cell-এর $2 \times$
Computation/timesteps	১ matrix mult.	৪	৩	L গুণ	২ গুণ
Gradient path (state-এ)	$\text{diag}(1 - h^2)W$	$\text{diag}(f_t)$	$\text{diag}(1 - z_t)$ (+ অন্য পথ)	সময় ও স্তর – দুই দিকে	দুই দিকে স্বাধীন BPTT
Long-term dependency	দুর্বল	শক্তিশালী	শক্তিশালী	cell-নির্ভর; গভীরতা	cell-নির্ভর; context

				optimization কঠিন করে	প্রশস্ত হয় কিন্তু gradient সমস্যা অপরিবর্তিত
Future context	নেই	নেই	নেই	নেই	আছে
প্রধান সুবিধা	সরল, দ্রুত, বোঝা সহজ	নিয়ন্ত্রিত ও দীর্ঘস্থায়ী স্মৃতি	LSTM-এর কাছাকাছি ফল, কম খরচে	স্তরায়িত/বিমূর্ত representation	প্রতিটি অবস্থানে সম্পূর্ণ context
প্রধান সীমাবদ্ধতা	Vanishing gradient	৪ গুণ parameter, ধীর	Output নিয়ন্ত্রণ নেই, accumulation নেই	Optimization কঠিন, খরচ বেশি	Real-time- এ অব্যবহার্য, দ্বিগুণ খরচ
সাধারণ প্রয়োগ	ছোট/সরল sequence, শিক্ষামূলক উদাহরণ	Translation, দীর্ঘ document , speech	Real- time/embedded, ছোট dataset	Speech recognition, Seq2Seq encoder- decoder	NER, POS tagging, sequence labeling, encoder

সতর্কতা: এই সারণির কোনো সারিই কোনো architecture-কে সার্বিকভাবে “সেরা” প্রমাণ করে না। প্রতিটি পছন্দই একটি **trade-off**। সঠিক নির্বাচন নির্ভর করে — sequence-এর দৈর্ঘ্য, dataset-এর আকার, real-time প্রয়োজনীয়তা, hardware সীমাবদ্ধতা এবং task-এর প্রকৃতির উপর। চূড়ান্ত সিদ্ধান্ত validation set-এর ফলাফল দিয়েই নেওয়া উচিত।

৮. PART 8 — সম্পূর্ণ Forward–Backward দৃষ্টিভঙ্গি

৮.১ সাধারণ কাঠামো

সব recurrent architecture-ই একই দুই-পর্বের চক্র অনুসরণ করে।

Forward Pass

Input → Hidden / Memory → Output → Loss

$$x_t \rightarrow h_t \text{ (এবং } C_t) \rightarrow \hat{y}_t \rightarrow L = \sum_{t=1}^T \ell(\hat{y}_t, y_t)$$

এই পর্বে প্রতিটি মধ্যবর্তী মান (activation, gate value, pre-activation) সংরক্ষণ করে রাখতে হয়, কারণ backward pass-এ সেগুলো লাগবে। এ কারণেই দীর্ঘ sequence-এর training-এ memory খরচ বেশি।

Backward Pass

Loss → Gradients → Parameters → Update

$$L \rightarrow \frac{\partial L}{\partial \hat{y}_t} \rightarrow \frac{\partial L}{\partial h_t}, \frac{\partial L}{\partial C_t} \rightarrow \frac{\partial L}{\partial W}, \frac{\partial L}{\partial b} \rightarrow W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

তিনটি সার্বজনীন নিয়ম

এই নোটের প্রতিটি derivation আসলে এই তিনটি নিয়মেরই প্রয়োগ:

(১) **Chain Rule:** একটি মান যদি একাধিক ধাপ পেরিয়ে loss-কে প্রভাবিত করে, তবে gradient হলো পথের প্রতিটি local derivative-এর গুণফল।

(২) **Multivariable Sum Rule:** একটি মান যদি একাধিক পথে loss-কে প্রভাবিত করে, তবে সব পথের gradient যোগ হয়। (যেমন $h_t \rightarrow \text{output}$ এবং $h_t \rightarrow h_{t+1}$)

(৩) **Shared Parameter Rule:** একই parameter যদি একাধিক জায়গায় ব্যবহৃত হয়, তবে তার gradient সব ব্যবহারের যোগফল:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L}{\partial W} \Big|_{\text{timestep } t}$$

৮.২ প্রতিটি architecture-এ এই প্রক্রিয়া কীভাবে পরিবর্তিত হয়

৮.২.১ Vanilla RNN

পর্ব	মূল সমীকরণ
Forward	$h_t = \tanh(W h_{t-1} + U x_t + b)$

State gradient	$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + W^T \left[\frac{\partial L}{\partial h_{t+1}} \odot (1 - h_{t+1}^2) \right]$
Backward path	$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2) W$
চরিত্র	বারবার matrix ও activation derivative-এর গুণফল $\rightarrow$ exponential ক্ষয়/বৃদ্ধি

৮.২.২ LSTM

পর্ব	মূল সমীকরণ
Forward	৪টি gate $\rightarrow C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \rightarrow h_t = o_t \odot \tanh(C_t)$
State gradient	$\delta C_t = \delta h_t \odot o_t \odot (1 - \tanh^2 C_t) + \delta C_{t+1} \odot f_{t+1}$
Backward path	$\frac{\partial C_t}{\partial C_{t-1}} = \text{diag}(f_t)$
চরিত্র	দুটি সমান্তরাল gradient পথ – C -পথ (বিশুদ্ধ, matrix-মুক্ত) ও h -পথ (gate-এর ভেতর দিয়ে)। C -পথই দীর্ঘ দূরত্বে gradient বহন করে।

৮.২.৩ GRU

পর্ব	মূল সমীকরণ
Forward	$r_t, z_t \rightarrow \tilde{h}_t \rightarrow h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
State gradient	তিনটি পথের যোগফল (সরাসরি + candidate + gate)
Backward path	$\frac{\partial h_t}{\partial h_{t-1}} \Big _{\text{direct}} = \text{diag}(1 - z_t)$
চরিত্র	LSTM-এর মতোই একটি matrix-মুক্ত সরাসরি পথ, তবে তা আলাদা state-এ নয়, একই hidden state-এর মধ্যেই; বাকি দুটি পথ gate-এর ভেতর দিয়ে যায়

৮.২.৪ Deep RNN

পর্ব	মূল সমীকরণ
Forward	$h_t^{(l)} = \tanh(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b^{(l)}), l = 1..L$
State gradient	$\frac{\partial L}{\partial h_t^{(l)}} = (U^{(l+1)})^T \frac{\partial L}{\partial a_t^{(l+1)}} + (W^{(l)})^T \frac{\partial L}{\partial a_{t+1}^{(l)}}$
Backward path	দুই দিক: temporal ($t \rightarrow t-1$, $W^{(l)}$ দিয়ে) ও vertical ($l \rightarrow l-1$, $U^{(l)}$ দিয়ে)
চরিত্র	প্রতিটি node-এ দুই উৎস থেকে gradient যোগ হয়; মোট পথদৈর্ঘ্য $\approx T + L$

৮.২.৫ Bidirectional RNN

পর্ব	মূল সমীকরণ
------	------------

Forward	$\vec{h}_t (1 \rightarrow T)$ ও $\bar{h}_t (T \rightarrow 1)$, তারপর $h_t = [\vec{h}_t; \bar{h}_t]$
State gradient	$\partial L / \partial h_t$ -কে দুই টুকরো করে ভাগ (split) করা হয়
Backward path	দুটি সম্পূর্ণ স্বাধীন BPTT — একটি $t \rightarrow t - 1$, অন্যটি $t \rightarrow t + 1$
চরিত্র	Concatenation-এ gradient কেবল বিভক্ত হয়, রূপান্তরিত হয় না; দুই network-এর মধ্যে কোনো gradient আদান-প্রদান নেই

৮.৩ একটি একীভূত পর্যবেক্ষণ

উপরের পাঁচটি ক্ষেত্র মিলিয়ে একটি সাধারণ সত্য দেখা যায়:

Gated architecture-গুলোর সাফল্যের একক গাণিতিক কারণ হলো — state-এর মধ্যে এমন একটি পথ তৈরি করা, যেখানে $\partial(\text{state}_t) / \partial(\text{state}_{t-1})$ একটি সরল diagonal matrix, কোনো weight matrix বা saturating activation-এর derivative নয়।

Vanilla RNN: $\text{diag}(1 - h_t^2) W$ (দুটি ক্ষয়কারী উপাদান)

LSTM: $\text{diag}(f_t)$ GRU: $\text{diag}(1 - z_t)$ (শেখযোগ্য, 1-এর কাছাকাছি হতে পারে)

এই একই ধারণা পরবর্তীকালে ResNet-এর skip connection ও Transformer-এর residual connection-এও ব্যবহৃত হয়েছে। অর্থাৎ LSTM-এর cell state একটি বৃহত্তর নীতির প্রাথমিক রূপ: **গভীর network-এ gradient-এর জন্য একটি বাধাহীন পথ রাখতে হয়।**

৯. প্রয়োজনীয় Diagram-সমূহের তালিকা

নিচে এই নোটে উল্লিখিত সব diagram-এর সারসংক্ষেপ এক জায়গায় দেওয়া হলো। প্রতিটির বিস্তারিত specification ও generation prompt সংশ্লিষ্ট অনুচ্ছেদে আছে।

#	Diagram	অবস্থান	মূল উদ্দেশ্য
1	Full LSTM Cell Architecture	ধারা ১.৩	চারটি gate ও দুটি state-এর সম্পূর্ণ তথ্যপ্রবাহ
2	LSTM Cell State + Hidden State Flow	ধারা ১.৩	সময়ের সঙ্গে দুটি স্মৃতিপথের চলাচল
3	Complete GRU Cell Architecture	ধারা ২.৩	reset/update gate ও একক state-এর গঠন
4	GRU Gate-by-Gate Information Flow	ধারা ২.৩	GRU-এর গণনার ক্রম ও নির্ভরতা
5	Deep RNN Stacked + Time-Unfolded	ধারা ৪.২	temporal ও spatial গভীরতা একসঙ্গে
6	Deep RNN Horizontal vs Vertical Flow	ধারা ৪.৪	$W^{(l)}$ ও $U^{(l)}$ -এর ভূমিকার পার্থক্য
7	Complete BiRNN Architecture	ধারা ৫.২	দুই দিকের network ও concatenation
8	BiRNN Context Flow for NER	ধারা ৫.২	ভবিষ্যৎ context-এর ব্যবহারিক প্রয়োজনীয়তা
9	Evolution of RNN Architectures	ধারা ৬.৩	পুরো নোটের Problem → Change → Consequence ধারা
10	Forward vs Backward Propagation	ধারা ৪.৭	দুই পর্বের দিক ও উদ্দেশ্যের পার্থক্য

সাধারণ Google Image Search keywords (সব diagram-এর জন্য উপযোগী):

LSTM architecture diagram · GRU architecture diagram · LSTM vs GRU comparison diagram · stacked RNN unfolded diagram · bidirectional LSTM architecture · backpropagation through time diagram · RNN family evolution chart · colah understanding LSTM networks

১০. Key Takeaways / Final Summary

১. মূল সমস্যা একটিই। Vanilla RNN-এ gradient-কে বহু timestep পেরোতে গিয়ে বারবার $\text{diag}(1 - h^2)W$ দিয়ে গুণ হতে হয়, যা exponential ক্ষয় বা বৃদ্ধি ঘটায়। এখান থেকেই বাকি সব architecture-এর জন্ম।

২. LSTM-এর সমাধান — additive memory with gates.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

মূল অর্জন: $\partial C_t / \partial C_{t-1} = \text{diag}(f_t)$ — একটি শেখযোগ্য gradient highway। খরচ: $4(n_x n_h + n_h^2 + n_h)$ parameter এবং দুটি state।

৩. GRU-এর সমাধান — একই নীতি, কম উপাদানে।

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Forget ও input gate একীভূত, দুটি state একীভূত। ফল: ঠিক ৭৫% parameter, $\partial h_t / \partial h_{t-1}|_{\text{direct}} = \text{diag}(1 - z_t)$ — একই ধরনের highway। খরচ: output নিয়ন্ত্রণ ও memory accumulation-এর সুযোগ হারানো।

৪. Deep RNN — গভীরতার অক্ষ। $h_t^{(l)} = \tanh(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b^{(l)})$ । এখানে $W^{(l)}$ সময়ের দিকে (বর্গাকার) এবং $U^{(l)}$ স্তরের দিকে (আয়তাকার) কাজ করে। Gradient দুই দিক থেকে এসে প্রতিটি node-এ যোগ হয়। সুবিধা: hierarchical feature। খরচ: দীর্ঘতর gradient পথ, কঠিনতর optimization।

৫. BiRNN — দিকের অক্ষ। $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ । দুটি স্বাধীন network, concatenation-এ মিলন, এবং backward pass-এ gradient কেবল দুই টুকরোয় বিভক্ত হয়। সুবিধা: প্রতিটি অবস্থানে সম্পূর্ণ context। খরচ: সম্পূর্ণ sequence আগে থেকে প্রয়োজন, তাই real-time causal prediction-এ অব্যবহার্য।

৬. তিনটি অক্ষ স্বাধীন। Cell type (RNN/LSTM/GRU), depth (L), এবং direction (uni/bi) — তিনটি পৃথক সিদ্ধান্ত, যেকোনো সমন্বয়ে ব্যবহার করা যায়। আধুনিক sequence labeling model প্রায়ই Stacked BiLSTM, অর্থাৎ তিনটিরই সমন্বয়।

৭. Parameter সূত্র মনে রাখার নিয়ম: ভিত্তি একটাই —

$$\text{base} = n_x n_h + n_h^2 + n_h$$

SimpleRNN = $1 \times \text{base}$, GRU = $3 \times \text{base}$, LSTM = $4 \times \text{base}$; bidirectional হলে আরও $2 \times$; L স্তর হলে প্রতিটি স্তরের জন্য আলাদা হিসাব (দ্বিতীয় স্তর থেকে n_x -এর জায়গায় আগের স্তরের n_h বসে)।

৮. একক সূত্রবাক্য। এই নোটের সব architecture একই প্রশ্নের ভিন্ন ভিন্ন উত্তর: “তথ্য ও gradient-কে দীর্ঘ পথ পেরিয়ে অক্ষত রাখব কীভাবে?” — LSTM ও GRU উত্তর দেয় gate ও additive update দিয়ে; Deep RNN বাড়ায় বিমূর্ততার স্তর; BiRNN বাড়ায় context-এর পরিধি।